

ALGORITMOS Y ESTRUCTURAS DE DATOS

MEMORIA DE PRÁCTICA II

ABRIL MAITENA BETTI

1. Análisis y descripción de la aplicación.	4
1.1.Cómo se han organizado y estructurado las clases y cuál es la responsabilidad de cada una.	4
1.2.Qué decisiones de diseño se han tomado.	12
1.3.Qué comprobaciones de integridad (y excepciones) se han implementado.	12
1.4.Qué técnicas de ordenación y búsqueda ha utilizado y por qué lo ha hecho.	18
1.5.Diagramas de clases UML.	20
2. Listado de fallos conocidos y funcionalidades definidas en el enunciado que no se han implementado en el código entregado.	20
3. Listado de todo el código fuente de la aplicación organizado por paquetes (si aplica) y clases.	21
A. Paquete Main	21
I. Clase MainAnimalLab	21
B. Paquete Laboratory	26
I. Clase Population	26
II. Clase Mouse	47
III. Clase Gender (Enumerado)	56
IV. Clase Chromosome (Enumerado)	57
C. Paquete InputOutput	57
I. Clase Input	57
II. Clase Output	66

D. Paquete CSV Administration	69
I. Clase CSV	69
E. Paquete Exceptions	74
I. Clase MouseExceptions	74
II. Clase PopulationExceptions	76
III. Clase CSVExceptions	77
IV. Clase InputExceptions	78
V. Clase FamilyExceptions	79
F. Paquete Families	80
I. Clase Family	80
II. Clase NormalFamily	84
III. Clase PolygamousFamily	86
IV. Clase SterileMaleFamily	88
V. Clase SterilePolygamousMaleFamily	90
G. Paquete Comparators	92
I. Clase ComparatorByWeight	92
II. Clase ComparatorByBirthdate	93
4. Listado de pruebas unitarias realizadas.	94
A. Paquete CSV Administration	94
I. CSVTest	94
B. Paquete Laboratory	95
I. GenderTest	95
II. MouseTest	96
III. PopulationTest	100
5. Conclusiones	105

1. Análisis y descripción de la aplicación.

1.1. Cómo se han organizado y estructurado las clases y cuál es la responsabilidad de cada una.

- Las clases que generé son las siguientes:
 - **Mouse:** la clase mouse consta de 9 atributos que definen el estado de las instancias de la clase y 26 métodos (4 constructores, 8 getters, 3 setters y 11 métodos) que definen su comportamiento. La responsabilidad de esta clase radica en establecer el "patrón" que tendrán todos los objetos (ratones) que sean creados para resolver la práctica y cumplir con su funcionalidad.
 - Respecto a los atributos, 5 son finales (la fecha de nacimiento, el sexo, los cromosomas y el código de referencia) ya que no quiero que el usuario pueda modificarlos una vez sean inicializados. La clase cuenta con 4 atributos no finales, 3 de los cuales pueden ser modificados por el usuario (peso, temperatura y descripción) y uno que es static ya que es utilizado para llevar la cuenta de cuántos ratones hemos generado, lo cual es útil a la hora de generar el código de referencia en el método que realiza esta función. Cabe mencionar que, ya que el atributo que lleva la cuenta de cuántos ratones fueron creados es estático, los códigos de referencia son únicos para cada ratón de la población (no se repetirán).
 - En cuanto a los métodos, hay un getter por cada atributo (menos del atributo estático "numMouse") mientras que sólo los atributos peso, temperatura y descripción tienen setters. La clase cuenta con un método que genera un código de referencia por cada ratón (lo hace de forma incremental, mediante el uso de la variable que lleva la cuenta de cuántos ratones han sido creados de tal forma que el código de referencia resulta igual al valor de dicho atributo estático), dos métodos que comprueban si la fecha de nacimiento es válida y si los cromosomas introducidos (mejor dicho, el segundo cromosoma, ya que es el que, en mi programa, determina el sexo del ratón) concuerda con el género que introdujo el usuario. También creé dos métodos cuya función es, basándose en los cromosomas del ratón, definir si el mismo es estéril o polígamo. Esta clase cuenta también con un método que modifica la información del ratón (sólo el peso, temperatura y descripción del ratón, ya que son los únicos atributos modificables de la clase).
 - Para la segunda entrega de la práctica sobrescribí los métodos "equals", "toString", "hashCode" y "compareTo" (ya que, hice que la clase Mouse implementara la interfaz 'comparable' para poder determinar, a través del método "compareTo" el orden natural con el que quiero que los ratones sean ordenados que, en mi práctica es en orden creciente del código de referencia del ratón). Además, incluí un método que genera una fecha de nacimiento random (dentro de los últimos 2,5 años) para poder inicializar el atributo 'birthday' de los ratones generados, con el objetivo de rellenar una población virtual. También agregué un método que analiza si un ratón es maduro sexualmente, calculando cuántos días lleva vivo y si supera la cantidad de días necesarios para poder comenzar a procrear.
 - **Population:** así como la clase mouse, la clase población establece el estado y comportamiento de los objetos (que en este caso son poblaciones) que serán creados al implementar el programa. Las clases población y ratón se encuentran fuertemente relacionadas a través de una relación de asociación en la cual la clase población es la

clase “fuerte”. Es por esto que, la clase población tiene un atributo que es un Array List en el cual se almacenan ratones (instancias de la clase ratón). En otras palabras, una población puede tener muchos ratones. Por otro lado, siendo la clase ratón la débil de la relación, un ratón sólo puede estar asignado a 0 o 1 población. Para la segunda entrega de la práctica, modifiqué el tipo de estructura que almacena ratones dentro de una población, cambiándola por un Array List (en la entrega anterior los ratones eran almacenados en un array estático de ratones). El motivo de esta modificación radica en las ventajas que estructuras de tipo dinámicas tales como una lista aportan, las cuales fomentan la simplicidad del programa ya que facilitan la gestión de los elementos contenidos en ella. Esto se debe a que en el programa la cantidad de ratones varía mucho, por lo que es útil almacenarlos en un tipo de estructura que no requiera que se le indique un tamaño fijo de elementos, si no que, a través de métodos como “add” o “remove”, su tamaño pueda ser modificado fácilmente. Cabe destacar que, debido al acceso directo a los elementos contenidos que caracterizan a los Array Lists, decidí emplear este tipo de estructura.

- La responsabilidad de la clase población es almacenar ratones en la lista mencionada y realizar distintas funciones especificadas en el enunciado de la práctica que involucran a los ratones (tanto como agregar un ratón a la población o eliminarlo, modificar la información de algún ratón, etc)
- Esta clase cuenta con 9 atributos:
 - I. Nombre de la población.
 - II. Nombre del responsable de la población.
 - III. La cantidad de días que la población pasará en el laboratorio para procrear
 - IV. La cantidad de días que lleva la población en el laboratorio.
 - V. Cuatro listas que contienen los ratones de la población: una lista para las hembras, otra para los machos, una tercera que almacena a todos los ratones y una cuarta en la que se almacenan ratones que son sexualmente inmaduros por lo que no están listos para formar familias.
 - VI. Un conjunto “Tree Set” en el que se almacenan las distintas familias relacionadas con la población (conformadas por ratones de la población).
- Ambos atributos de los nombres de la población y del responsable son de tipo “string” pero, el segundo, a diferencia del primero, sólo puede estar compuesto de letras. Esto se debe a que consideré que una población tranquilamente puede tener un nombre compuesto tanto de caracteres como de número pero no en el caso del nombre de una persona. Por último, el atributo de días en el laboratorio debe ser un número mayor que 0 y menor que 630 (inclusive), que a su vez debe ser múltiplo de 45 (ya que el ciclo de reproducción de un ratón dura 45 días).
- En cuanto a los métodos, la clase cuenta con 33 (2 constructores, 11 getters, 3 setters y 17 métodos). Los atributos que tienen setters son: la cantidad de días en el laboratorio y los nombres de la población y del responsable. En cuanto a los métodos, se encuentran: agregar ratón, obtener los códigos de referencia, identificar ratón, modificar la información de un ratón, eliminar un ratón, ver información (detallada) de un ratón. El resto de métodos fueron agregados para la segunda entrega de la práctica. Estos son:
 - I. “isListOfMiceEmpty”: comprueba si la lista de ratones está vacía.
 - II. “amountOfMiceInPopulation”: devuelve un entero que indica la cantidad de ratones que están almacenados en la lista de ratones de la población.
 - III. “fillVirtualPopulation”: es utilizado a la hora de rellenar la población virtual de ratones. Para esto, recibe por parámetro la información que ingresa el usuario para crear la población virtual (los porcentajes y la cantidad de ratones que quiere que tenga la población inicialmente) y crea, utilizando números ratones la cantidad

de ratones de cada tipo (machos estériles, hembras estériles, machos polígamos, machos normales, hembras normales o con una sola mutación). A su vez, este método llama, cada vez que genera un ratón, al método encargado de agregar los ratones a las listas de ratones de la población.

- IV. “isMouseInPopulation”: devuelve true o false dependiendo si el ratón que se le pasa por parámetro está almacenado en la lista que contiene todos los ratones de la población.
- V. “assignMiceToFamilies”: método que recorre las listas de ratones hembras y machos hasta encontrar uno que sea sexualmente maduro de cada sexo (motivo para el cual llama al método que determina si un ratón es o no sexualmente maduro) y llama al método encargado de asignarlos como los padres del tipo de familia que les corresponda (dependiendo los cromosomas del ratón macho).
- VI. “adjustFamilies”: este método se encarga de analizar si los ratones de
- VII. “adjustListsOfMice”: este método recibe una lista de ratones por parámetro, de la cual genera una copia que utiliza para recorrerla, determinar si cada uno de sus ratones es lo suficientemente joven como para seguir formando parte de la población y, si no lo es, lo elimina de la lista original recibida como parámetro y de la lista que engloba a todos los ratones de la población.
- VIII. “findSexuallyMatureMouse”: este método recibe una lista de ratones por parámetro y analiza cuáles de los ratones contenidos son sexualmente maduros. Aquellos que sean considerados inmaduros, son almacenados en la lista de ratones inmaduros y eliminados de la lista de ratones hembra o machos (dependiendo del sexo del ratón inmaduro). Un ratón es considerado sexualmente maduro pasados 70 días desde su nacimiento, ya que durante sus primeros 20 días de vida (que forman parte del ciclo en el que nace el ratón) el mismo se independiza y, en los siguientes 45 días (es decir, durante el siguiente ciclo), si bien el ratón es independiente, todavía no cuenta con la capacidad para formar una familia y generar crías.
- IX. “createFamily”: método que recibe por parámetro una instancia de ‘Mouse’ (el ratón padre) y un conjunto de ratones (en el que se almacenan las madres, que hasta el momento es una sola) y, dependiendo de si el padre es estéril o polígamo, asigna a la pareja de ratones a la familia que corresponda.
 - IX.I. Familia normal: el método crea la familia compuesta por el padre y la madre. Estos tipos de familias producirán crías a menos que cuenten con una madre estéril.
 - IX.II. Familia con macho estéril: el método se encarga de buscar un macho fértil que se utiliza para construir la familia. Si se encuentra un macho que no sea polígamo y que sea fértil, la familia producirá crías (siempre y cuándo la madre no sea estéril). Si no se encuentra ningún macho que reúna las características necesarias, la familia se crea igualmente, pero no producirá crías ya que no cuenta con un macho fértil.
 - IX.III. Familia con macho polígamo: mediante el uso de números random, se siguen agregando hembras en el conjunto o no. Una vez se haya terminado de agregar hembras, se crea la familia, compuesta por el padre y las n hembras que hayan sido agregadas al conjunto (como mínimo, puede ser una hembra). Este tipo de familias producirá distintas cantidades de crías por cada hembra, a menos que todas las hembras de una familia polígama sean estériles.
 - IX.IV. Familia con macho polígamo y estéril: el método realiza los procedimientos explicados para las familias estériles y para las familias polígamas. La diferencia radica en que para este tipo de familias, el macho fértil debe ser,

además de no estéril, polígamo. Si no se encuentra un macho que reúna estas características, la familia procede a ser creada sin un macho fértil por lo que, no producirá crías. Otro escenario en el que podría producir cero crías es si todas las madres fueran estériles.

- X. “addBabyMiceToPopulation”: método encargado de recorrer el conjunto de familias y llamar al método (perteneciente a la clase ‘Family’) que hace que las mismas se reproduzcan. Luego, recibe el conjunto que contiene a las crías producidas por la familia y, lo recorre con el objetivo de agregar las crías a las listas de ratones de la población. A su vez, vuelve a agregar a la lista de ratones de la población aquellos que, en el ciclo anterior fueron considerados inmaduros (es decir, los que se encontraban almacenados en la lista de ratones inmaduros) y vacía dicha lista.
- XI. Por último, sobrescribí los métodos “toString”, “equals” y “ hashCode” con el objetivo de que los objetos de la clase población fueran representados de manera legible y comparados correctamente.

- **CSV:** esta clase es la que se encarga de manejar las opciones del programa que involucran el uso de ficheros. Consta de 5 funciones estáticas: una que abre un archivo con extensión ‘CSV’, dos que verifican si la línea leída del archivo abierto representan una población y un ratón, otra que guarda un archivo y finalmente una función encargada ‘guardar como...’ el archivo.
- **Input:** es la clase encargada de leer los datos de consola que ingrese el usuario. No tiene atributos pero sí diversas funciones (13 en total, todas static).
 - Funciones encargadas de leer datos de consola: son 7 las funciones que son utilizados en el main y otras funciones también de la clase Input. Si bien sus responsabilidades son muy similares, difieren en el tipo de dato que devuelve cada una. Es decir, se ha creado una función por cada tipo de dato que se le ordena introducir al usuario, con el objetivo de poder corroborar que los ‘inputs’ concordaran con el tipo de dato pedido. Hay una función para leer: enteros, número reales, arrays de caracteres, fechas (tipo de dato “LocalDate”), enumerados (cromosomas y sexo).
 - Las funciones restantes son: crear población, crear población virtual, crear ratón, ordenar ratones, modificar la información de un ratón y ver información de un ratón. Estas funciones le piden al usuario la información que requieran y llaman a los métodos de las clases población o ratón que sean necesarios para cumplir con su propósito.
 - Para la segunda entrega de la práctica, las funciones “createVirtualPopulation” y “orderMiceOfPopulation” fueron creados. El primero, es el encargado de llamar a los métodos correspondientes de ‘Population’ que crean y rellenan una población (virtual). El segundo método, por el otro lado, ordena, dependiendo el orden que haya elegido el usuario, por código de referencia, en orden cronológico o por peso los ratones de la población. Esto es conseguido utilizando los métodos 'equals' y 'hashCode' que fueron sobrescritos en la clase ‘Mouse’ en el caso de que se desee ordenar los ratones en orden creciente de código de referencia. Si los ratones son ordenados en orden cronológico o por peso, utilizamos las clases comparadoras que fueron generadas para éste propósito.

- **Output:** al igual que la clase Input, la clase Output no cuenta con ningún atributo pero sí con 5 funciones estáticas. Una de ellas es la encargada de imprimir el menú de opciones para que el usuario pueda verlo en consola. Otra de sus funciones, imprime los datos de la población (y sus ratones, si es que tiene) guardada al leer el archivo abierto en la opción 1 del menú.
- Las funciones restantes fueron incluidos para llevar a cabo la segunda parte de la práctica. En primer lugar, agregué la función “showOrdinationCriteria” que, como su nombre indica, imprime las diversas opciones con las que el usuario puede ordenar los ratones de la población. Como consecuencia incluí a su vez el método “printPopulationOpened” que se encarga de imprimir los ratones de la población (utiliza el método sobrescrito “toString” de ‘Mouse’). Por último, incluí el método “simulatePopulationsEvolution” que es el encargado de calcular los porcentajes de ratones (hembras normales, machos normales, machos estériles, etc.) y de imprimirlos.
- **Main:** la clase main se genera con el objetivo de contar con una clase que contenga un método (el método main) que le indique al programa dónde comenzar la ejecución del programa (sin este método, el programa no sería ejecutable). En esta clase es dónde se generan instancias de las clases ratón y población que luego son utilizadas por distintos motivos, dependiendo de lo que desee hacer el usuario (de la opción que elija de las ofrecidas por el menú). En definitiva, gracias al método main podemos ejecutar el programa y, de esta manera, probar nuestros métodos, funciones y clases.
- La clase main cuenta con 4 atributos estáticos: uno que guarda el “file path” utilizado al guardar un archivo, otro en el que se almacena la población que ha creado el usuario (o null si no ha creado ninguna todavía), un tercero de tipo boolean que almacena true si se ha abierto un archivo (y, por ende, podemos guardarlo o guardarlo como) o false si ningún archivo ha sido abierto. El último atributo es también de tipo boolean y almacena true si la población ya fue guardada como o false en el caso contrario; este atributo es útil para saber, a la hora de guardar un archivo, si el mismo fue o no guardado como y viceversa.
- Además, generé dos funciones cuyo objetivo es verificar si se ya se creó una población o no y si la lista de ratones de la población se encuentra vacía o no. Excepciones serán lanzadas en los casos de que no se haya creado una población o la lista de ratones se encuentre vacía. Estas funciones fueron creadas para reutilizar código, ya que antes de realizar la mayoría de las operaciones del menú principal, es necesario corroborar que se haya creado una población y que la misma cuente con al menos un ratón.
- Estos atributos y funciones son muy útiles para establecer bajo qué condiciones se puede, por ejemplo, eliminar un ratón de una población (necesitamos que el usuario cree una población y le agregue ratones antes de eliminar alguno) o guardar un archivo (primero necesitamos que el archivo se guarde como)
- Dentro del método main, realicé un switch con un case por cada opción del menú y un default en el que se imprime un mensaje de error indicando que el usuario ingresó un número de operación inexistente.
- **ComparatorByWeight:** esta clase implementa la interface ‘Comparator’ y, por ende, el método ‘compare’. Esta clase fue generada con el objetivo de proveer una forma alternativa de ordenar los ratones dependiendo de su peso.

- **ComparatorByBirthdate:** esta clase implementa la interface 'Comparator' y, por ende, el método 'compare'. Esta clase fue generada para establecer como deben ser ordenados los ratones dependiendo de su fecha de nacimiento.
- **Family:** clase abstracta que está compuesta por 5 atributos que definen el estado de las instancias de la clase y 15 métodos (1 constructor, 3 getters, 1 setter y 10 métodos). La responsabilidad de esta clase radica en establecer el "patrón" que tendrán todos los objetos (familias) que sean creados para resolver la práctica y cumplir con su funcionalidad.
 - Los atributos que tiene esta clase son dos objetos de la clase 'Mouse' (el ratón padre y el ratón macho fértil, que será null en las familias donde no se lo necesite y almacenará un ratón macho y fértil en las familias donde el ratón padre sea estéril), un conjunto de ratones (las madres de la familia), un entero final 'familyID' y un contador estático de tipo entero que es utilizado para asignar un ID a cada familia. Cabe destacar que, de la misma forma que se generan los código de referencia se generan los ID para las familias. Esto es, de manera incremental, sumando 1 a una variable estática que lleva la cuenta de cuántas familias fueron creadas. Es por esto que, los IDs de las familias serán únicos. En cuanto al conjunto de ratones madres, decidí almacenarlos en un 'Tree Set' ya que un conjunto no admite elementos repetidos, lo cual coincide con mi intención de almacenar ratones diferentes en la estructura que contuviera a las madres.
 - En cuanto a los métodos, dos de ellos, "amountOfBabyMice" y "reproduce" son abstractos ya que cada clase hija de familia debe implementarlos y cada una lo hará a su forma. El primero se encarga de determinar la cantidad de crías que producirá una pareja de ratones, mediante un número random. Luego, "reproduce" generará el número obtenido de crías (es decir, crea los ratones con fecha de nacimiento actual). Entre los demás métodos de 'Family' encontramos "inheritChromosome" cuyo fin es, mediante un número random, definir qué cromosoma (el primero o el segundo) heredará el ratón bebé del ratón que recibe por parámetro (la madre o el padre). Debido a que, mi programa no admite 'Y' o 'Ymut' como el primer cromosoma del ratón, las crías, de manera fija, siempre heredarán su primer cromosoma de la madre y el segundo cromosoma del padre. Además, cabe destacar que se realizó en el orden explicado ya que, si fuese al revés (es decir, que las crías heredaran el primer cromosoma del padre y el segundo de la madre), las crías formadas serían siempre hembras. A su vez, generé un método llamado "retrieveMouseFromMothersSet" que recibe una posición (un entero) por parámetro y devuelve el ratón almacenado en la posición recibida del conjunto de ratones madres. Este método es útil para, por ejemplo establecer de qué ratón hembra debe heredar su cromosoma una cría. Luego agregué un método llamado "generateFamilyID" que genera el ID de la familia, el cual devuelve como un entero. Además, para comprobar si los ratones que componen una familia son suficientemente jóvenes para seguir formando parte de la familia y de la población, generé el método "shouldBeAlive" que devuelve un boolean dependiendo de si el ratón que recibe como parámetro, no cumplió todavía los 2,5 años (devuelve true) o si sí los cumplió (devuelve false). Este método es útil para simular la muerte de aquellos ratones que hayan alcanzado la edad estipulada (2,5 años).
 - La clase 'Family' tiene 4 clases hijas: todas ellas implementan los métodos abstractos de 'Family'.
 - **SterileMaleFamily:** la característica de esta clase es que su macho (el ratón padre) es estéril. A pesar de esto, hay ciertas posibilidades de que la hembra quede embarazada de otro ratón macho (que, por supuesto debe ser no estéril). En mi programa, el constructor de esta clase recibe 3 parámetros: el padre y un macho fértil (que son dos

objetos de 'Mouse') y un conjunto de ratones hembras (las madres, que en este caso será una sola). El macho fértil puede ser null (si no se encontró un ratón macho sin mutaciones; es decir, no estéril y no polígamo) o puede almacenar un ratón. En el primer caso, las crías producidas serán igual a 0, lo cual ocurre también si la madre es estéril (independientemente de si se encontró o no un ratón macho fértil). En el segundo escenario, siempre y cuando la hembra no sea estéril, la familia producirá la cantidad de crías que devuelva el método "amountOfBabyMice" (entre 2 y 6 crías). A su vez, los ratones bebés heredarán su primer cromosoma de la madre y el segundo cromosoma del padre biológico (el ratón fértil).

- **NormalFamily:** a este tipo de familia se compone por un ratón macho que no presenta ninguna mutación y una hembra. En los casos en los que la hembra sea estéril, la familia producirá 0 crías. En el caso contrario, entre 2 y 9 crías serán producidas.
- **PolygamousMaleFamily:** el macho de esta clase, el ratón padre, presenta la mutación 'Ymut', por lo que tiende a la poligamia. En este caso, el conjunto de ratones madres contendrá como mínimo 1 ratón. Como consecuencia, dentro de la implementación del método "reproduce" de esta clase encontramos un for que recorre el conjunto de ratones, ya que cada hembra se reproduce con el macho. Además, el método "amountOfBabyMice" es llamado tantas veces como madres almacenadas haya en el conjunto, ya que las madres pueden producir distintas cantidades de crías. En este caso, los ratones bebés heredarán el primer cromosoma de la madre que corresponda y el segundo cromosoma del padre.
- **SterilePolygamousMaleFamily:** en estos casos, el macho presenta ambas mutaciones; es decir, es estéril y tiende a la poligamia. Los constructores de estas familias reciben también por parámetro un objeto de 'Mouse' que almacena null (si no se encontró un ratón macho sólo con la mutación de la poligamia; es decir, no estéril y polígamo) o un ratón (en el caso de que se haya encontrado un "sustituto"). Si el ratón fértil es null, la familia no producirá ninguna cría mientras que, si almacena un ratón, la familia producirá entre 2 y 6 crías por cada hembra fértil. El único caso en el que una familia de este tipo no generaría crías implica que el macho fértil sea null y las hembras contenidas (o la hembra contenida) en el conjunto de ratones fueran estériles (o estéril). En este tipo de familia, volvemos a tener un for dentro del método "reproduce" que recorre el conjunto de hembras, llamando tantas veces (como hembras haya en el conjunto) al método "amountOfBabyMice", "inheritChromosome" y generando los ratones bebés. En este caso, las crías heredarán su primer cromosoma de la madre que corresponda y el segundo del macho fértil.
- **Gender:** la clase 'Gender' es un enumerado que define los posibles valores que el atributo 'gender' de los objetos de la clase 'Mouse' pueden tomar. Las opciones de valores son: 'MALE' y 'FEMALE'. La responsabilidad de esta clase es definir los únicos valores que son considerados como válidos para una variable de tipo 'Gender'. A su vez, esta clase implementa dos métodos cuya finalidad es verificar que los atributos género y cromosomas de los ratones tengan sentido. Estos métodos son "isGenderConcordant", que verifica que el género y los cromosomas que introdujo el usuario para el ratón concuerden (es decir, si el segundo cromosoma introducido es 'Y' o 'Ymut', que el género sea masculino o femenino si el segundo cromosoma es 'X' o 'Xmut') y "assignGenderToVirtualMouse" que, basándose en el segundo cromosoma que se le haya asignado al ratón, asigna su género como femenino o masculino, según corresponda. Este último método fue generado para establecer el sexo de los ratones virtuales.

- **Chromosome:** la clase 'Chromosome' es un enumerado que define los posibles valores que los atributo 'chromosome1' y 'chromosome2' de los objetos de la clase 'Mouse' pueden tomar. Las opciones de valores son: 'X', 'Xmut', 'Y' e 'Ymut'. La responsabilidad de esta clase es definir los únicos valores que son considerados como válidos para una variable de tipo 'Chromosome'.
- **MouseExceptions:** clase hija de 'RuntimeException' que cuenta con un enum en el que se definen los 6 distintos tipos de excepciones que pueden ocurrir al operar con la clase Mouse. La responsabilidad de esta clase radica en manejar errores y situaciones inesperadas que puedan llegar a ocurrir al manipular objetos de dicha clase. Esta clase cuenta con un atributo cuyo tipo es el enumerado mencionado, un constructor en el que se inicializa el atributo, un get para su único atributo y el método 'toString' sobrescrito, en el que se imprimen mensajes especiales por cada tipo de excepción.
- **PopulationExceptions:** clase hija de 'RuntimeException' que cuenta con un enum en el que se definen los 14 distintos tipos de excepciones que pueden ocurrir al operar con la clase 'Population' y sus objetos. Esta clase cuenta con un atributo cuyo tipo es el enumerado mencionado, un constructor en el que se inicializa su atributo, un método get para dicho atributo y un método 'toString' sobrescrito en el que se imprimen mensajes especiales por cada tipo de excepción. Al igual que las excepciones de la clase ratón, la responsabilidad de esta clase es el manejo controlado de errores que puedan ocurrir al ejecutar el programa.
- **InputExceptions:** clase hija de 'RuntimeException' que cuenta con un enum en el que se definen los 2 distintos tipos de excepciones que pueden ocurrir al operar con la clase 'Input' (que no se haya creado una población o que la lista de ratones de la población esté vacía). Esta clase cuenta con un atributo cuyo tipo es el enumerado mencionado, un constructor en el que se inicializa su atributo, un método get para dicho atributo y un método 'toString' sobrescrito en el que se imprimen mensajes especiales por cada tipo de excepción. Al igual que las excepciones de la clase ratón y población, la responsabilidad de esta clase es el manejo controlado de errores que puedan ocurrir al ejecutar el programa.
- **CSVExceptions:** clase hija de 'RuntimeException' que cuenta con un enum en el que se definen los 5 distintos tipos de excepciones que pueden ocurrir al operar con la clase 'CSV'. Esta clase cuenta con un atributo cuyo tipo es el enumerado mencionado, un constructor en el que se inicializa su atributo, un método get para dicho atributo y un método 'toString' sobrescrito en el que se imprimen mensajes especiales por cada tipo de excepción. La responsabilidad de esta clase, al igual que las demás clases de excepciones, radica en proveer un manejo controlado de errores al manipular métodos de, en este caso, la clase 'CSV'.
- **FamilyExceptions:** clase hija de 'RuntimeException' que cuenta con un enum en el que se definen los 8 distintos tipos de excepciones que pueden ocurrir al operar con la clase 'Family' y sus objetos. Esta clase cuenta con un atributo cuyo tipo es el enumerado mencionado, un constructor en el que se inicializa su atributo, un método get para dicho atributo y un método 'toString' sobrescrito en el que se imprimen mensajes especiales por cada tipo de excepción. La responsabilidad de esta clase no difiere del mencionado en los casos de excepciones para las clases ratón, población, input y CSV.

1.2. Qué decisiones de diseño se han tomado.

- En cuanto al diseño del programa, el mismo se encuentra dividido en 7 paquetes con el objetivo de ordenar el código, de tal modo que sea entendible y se pueda acceder a cada clase con facilidad. Es decir, intenté evitar tener muchas clases juntas con propósitos diferentes ya que puede resultar perjudicial.
- **CSVAdministration:** este paquete contiene la clase “CSV” que, como fue explicado en el apartado anterior, es la encargada de abrir y guardar archivos.
- **Exceptions:** este paquete contiene las clases hijas de “RuntimeException”: “PopulationExceptions”, “MouseExceptions”, “CSVExceptions”, “FamilyExceptions” e “InputExceptions”. El propósito de separar las excepciones en un paquete distinto se debe a organizar el código de la forma más ordenada posible. Al tener un paquete que reúne las clases de excepciones, es más fácil identificarlas a la hora de querer agregar algún tipo de excepción o simplemente al querer acceder a ellas.
- **InputOutput:** este paquete contiene dos clase cuyos nombres son “Input” y “Output”. Esto se debe a que me pareció útil separar la clase encargada de leer datos ingresados por consola y aquella encargada de imprimir en la consola, del resto del programa.
- **Laboratory:** este paquete contiene lo que se define en el enunciado como la “lógica del negocio”, es decir, contiene las clases: “Population”, “Mouse” y los enumerados “Chromosome” y “Gender”.
- **Main:** este paquete contiene la clase “MainAnimalLab” que es la que contiene el método que hace que el programa sea ejecutable. Este paquete, como se indicaba en el enunciado de la practica, se encarga de la interfaz del usuario.
- **Comparators:** dicho paquete contiene dos clases que, como se mencionó anteriormente, implementan la interfaz “Comparator”: “ComparatorByBirthdate” y “ComparatorByWeight”. Generé este paquete ya que la responsabilidad de esta clase difiera con las responsabilidades de las clases contenidas por los demás paquetes.

1.3. Qué comprobaciones de integridad (y excepciones) se han implementado.

- En esta práctica, se han realizado diversas pruebas de integridad que están fuertemente ligadas con las clases de excepciones que fueron creadas con el objetivo de controlar y manejar errores. Se han implementado 5 clases de excepciones, todas clases hijas de “RuntimeException”. Entre ellas encontramos “MouseExceptions”, que contiene los distintos tipos de excepciones necesarias para generar correctamente instancias de ‘Mouse’; “PopulationExceptions” que también contiene distintos tipos de excepciones (declarados en un enumerado) que se precisan a la hora de crear objetos de la clase ‘Population’; “InputExceptions”, que contiene dos tipos de excepción que podemos encontrar cuando queremos realizar ciertas acciones que requieren que una población haya sido creada y que cuente con al menos un ratón; “CSVExceptions”, que incluye

varias excepciones que pueden ser lanzadas cuando utilizamos las opciones de 'abrir', 'guardar' y 'guardar como' un archivo; y "FamilyExceptions", que contiene 8 excepciones distintas que precisamos para asegurar una correcta generación de objetos de la clase.

- **Clase 'Input':**

- Funciones que crean una población y un ratón: estas funciones comprueban, respectivamente, que la población y el ratón se creen correctamente. Es decir, contemplan la posibilidad de que se lancen excepciones al intentar instanciar estas clases con información inválida introducida por el usuario.
- Función que ordena los ratones: comprueba que el usuario haya introducido un número válido; esto es, entre 1 y 3. Esto es comprobado incluyendo un 'default' en el switch case que implementa dicha función.
- Función que modifica la información de un ratón: contempla la posibilidad de que se lancen excepciones del tipo 'MouseExceptions' o 'PopulationExceptions' al intentar cambiar la información de un ratón por valores inválidos o si el ratón con el código de referencia que insertó el usuario no se encuentra en la población, respectivamente.
- Función que muestra la información de un ratón recibiendo su código de referencia: contempla la posibilidad de que no exista un ratón con ese código de referencia y se lance una excepción por ello.
- Funciones que leen datos de consola: contemplan que se pueden lanzar excepciones de tipo "IOException" si hay algún inconveniente a la hora de leer una línea de la consola. A su vez, una excepción del tipo "IllegalArgumentExceptions" puede ser lanzada si el input está vacío, por lo que el usuario no podrá ingresar ningún caracter en blanco para inicializar la variable que se le pida. También se contempla, en la función en la que se lee una fecha, que si la fecha es inválida (como lo son por ejemplo el 30 de febrero o el 31 de abril) se lanza una excepción de tipo "DateTimeException".

- **Clase 'Mouse' y su respectiva clase de excepciones 'MouseExceptions':**

- Cromosomas: se comprobó que el primer cromosoma solo pudiera tomar los valores 'X' o 'Xmut' y que el segundo cromosoma, el cual determiné como el que define el sexo del ratón, tomara alguno de los siguientes valores 'X', 'Xmut', 'Y' o 'Ymut'. Además se comprobó que el segundo cromosoma (ingresado por el usuario) concordara con el sexo que ingresó el usuario. De no cumplirse alguna de estas condiciones, el ratón no será creado.
 - Si el valor introducido para el cromosoma número 1 no es 'X' ni 'Xmut', se lanza una excepción del tipo "INVALID_CHROMOSOME_1" de la clase 'MouseExceptions' y se imprime un mensaje indicando el problema.
 - Por otro lado, si el valor introducido para el cromosoma número 2 no es 'X', 'Xmut', 'Y' ni 'Ymut', se lanza una excepción del tipo "INVALID_CHROMOSOME_2" de la clase 'MouseExceptions' y se imprime un mensaje indicando el problema.

- En cambio, si ambos cromosomas son válidos pero si el sexo determinado por el segundo cromosoma no concuerda con el ingresado por el usuario, se lanza una 'MouseExceptions' del tipo "NON_CONCORDANT_GENDER" y se imprime un mensaje indicando el problema.
- Fecha de nacimiento: en el caso de generar ratones virtuales o a través de la información introducida por el usuario como inputs, se comprobó que la fecha de nacimiento fuera válida, es decir, que no sea una fecha futura. En este caso, se lanza una 'MouseExceptions' del tipo "INVALID_BIRTHDAY" y se imprime su mensaje correspondiente.
- En el caso de las crías, esta comprobación fue eliminada ya que, al estar simulando la evolución de la población, las fechas de nacimiento de las crías deben ser futuras.
- Peso: se comprobó que el peso estuviese dentro de un rango que establecí con un mínimo de 10 gramos y un máximo de 500 gramos. Establecí un rango para evitar que el usuario pudiera ingresar un valor demasiado alto ya que se trata del peso de un ratón. Si el peso introducido por el usuario no reúne estas condiciones, el mismo no es creado, se lanza una 'MouseExceptions' del tipo "INVALID_WEIGHT" y se imprime el mensaje que corresponda.
- Temperatura: al igual que con el peso del ratón, establecí un rango de temperaturas válidas cuyo mínimo es 1 y su máximo es 45 grados. Si esto no se cumple, el ratón no es creado, se lanza una 'MouseExceptions' del tipo "INVALID_TEMPERATURE" y se imprime el mensaje que corresponda.
- **Clase 'Population' y su respectiva clase de excepciones 'PopulationExceptions':**
 - Nombre del responsable: se comprobó que el nombre de la persona no estuviese compuesto por caracteres de espacio en blanco. Además, sólo puede estar compuesto por letras, por lo que, si el usuario ingresa algún número o algún carácter especial o de espacio en blanco, la población no será creada. En el caso de que el usuario introduzca un nombre inválido, se lanza una 'PopulationExceptions' de tipo "INVALID_NAME" y se imprime su mensaje correspondiente.
 - Nombre de la población: se comprobó que el nombre de la persona no fuesen caracteres de espacio en blanco. El nombre de la población, a diferencia del nombre del responsable, sí puede estar compuesto por número, caracteres o una mezcla.
 - Días en el laboratorio: se comprobó que el número ingresado en carácter de la cantidad de días que la población pasará en el laboratorio fuese mayor que 0 (sin incluir el 0) y menor o igual que 630. A su vez, se comprobó que el mismo fuera múltiplo de 45 (duración del ciclo de reproducción de los ratones). De no cumplir estas condiciones, la población tampoco será creada. En el caso en el que la cantidad de días introducida sea inválida ya que no se encuentra entre el rango pedido (mayor que 0 y menor que 631), se lanza una 'PopulationExceptions' del tipo "INVALID_PROCREATION_DAYS". Por otro lado, si el dato introducido se encuentra en el rango solicitado pero no es múltiplo de 45, se lanza una 'PopulationExceptions' del tipo "NON_MULTIPLE_45_PROCREATION_DAYS".

- Ratones agregados a la población: antes de agregar ratones a una población, se comprueba si el ratón que se quiere añadir es null. En el caso de que lo sea, se lanza una 'PopulationExceptions' del tipo "FAILED_TO_ADD_MOUSE", ya que no añadiremos ratones null a la lista de ratones de la población. Además, se verifica que el ratón que se quiere agregar no se encuentre en la población de ratones, para evitar tener elementos repetidos en la lista. Por último, se comprueba, en el caso de que el ratón no sea null, el sexo del mismo para agregarlo a la lista que corresponda (la lista de machos o la de hembras).
- Método de identificar ratones: se comprueba si existe un ratón con el código de referencia que introdujo el usuario. En el caso de que no exista, se lanza una 'PopulationExceptions' de tipo "MOUSE_NOT_FOUND" y se imprime el mensaje correspondiente.
- Borrar un ratón de la población: el método borrar ratón llama al método que identifica un ratón por lo que, se contempla el caso en el que el usuario intente remover de la lista un ratón inexistente (en ese caso, como fue explicado en el método anterior, se lanza una 'PopulationsExceptions' del tipo "MOUSE_NOT_FOUND"). En el caso de que se encuentre el ratón que el usuario quiere eliminar, se comprueba si el ratón fue eliminado correctamente. En caso contrario, se lanza otra 'PopulationExceptions' del tipo "FAILED_TO_REMOVE_MOUSE".
- Método que muestra información detallada del ratón: llama al método que identifica ratones por lo que, como se explico anteriormente, se comprueba que el ratón exista en la población.
- Método que rellena una población virtual con ratones virtuales: llama al método que agrega ratones a la lista de ratones de la población. Como consecuencia, se contempla que el ratón no sea null y que el mismo no haya sido agregado anteriormente (para evitar tener elementos repetidos).
- Método "adjustFamilies": verifica la edad de los ratones, eliminando de las familias a aquellos mayores de 2,5 años. Luego, en el caso de que los otros ratones que componen la familia del ratón fallecido sean menores de 2,5 años, los agrega a las listas llamando al método que cumple con esta funcionalidad (se comprueba que el ratón no sea null, que no se encuentre en la lista y su sexo).
- Método "adjustMiceLists": al igual que el método anterior, verifica que los ratones contenidos en la lista que recibe por parámetro sean menores de 2,5 años.
- Método "findSexuallyMatureMouse": comprueba si los ratones de la lista que recibe como parámetro son o no sexualmente maduros. En el caso de que no lo sean, son eliminados de la lista que fue recibida por parámetro y agregados a la lista de ratones inmaduros.
- Método que crea familias: este método llama a los constructores de las familias que correspondan según cada caso y, si alguno de los ratones no reúne las condiciones que debería para formar una familia del tipo cuyo constructor invocamos, una 'FamilyException' será lanzada. Esta excepción, dependiendo del problema en específico, podrá ser de diversos tipos.

- **Clase 'CSV':**

- Se implementan funciones que, en el caso de no ser satisfactorias, lanzan excepciones tales como "IOException" o "NullPointerException". Por ejemplo, al crear una nueva instancia de "FileWriter" e inicializarla en un file path introducido por el usuario, una "IOException" puede ser lanzada. Esto mismo ocurre al utilizar operaciones de "BufferedWriter". En el caso de "NullPointerException", esto puede ocurrir si el file path introducido por el usuario es null. Además, en la función de abrir un archivo CSV, utilizamos funciones 'parse' para convertir la información leída en tipos de datos que sean de nuestro interés. Es por esto que incluí, en el catch, la excepción de tipo "DateTimeParseException", la cual es lanzada cuando la función parse falla al convertir la información leída en una variable de tipo 'LocalDate'. Lo mismo ocurre en el caso de convertir la información leída en un número entero o uno flotante ya que, en el caso de fallar, lanza una excepción de tipo "NumberFormatException".
- Método "isLineAPopulation": lanza una excepción personalizada de la clase 'CSVExceptions' y del tipo "INVALID_FORMAT_OF_LINE_POPULATION" en el caso de que la línea leída del archivo abierto no sea una población. Es decir, tanto si la línea contiene más o menos elementos de los esperados, o si estos elementos son de tipos distintos de los que deberían, la excepción será lanzada.
- Método "isLineAMouse": al igual que el método anterior, este lanza una excepción de la clase 'CSVExceptions' y del tipo "INVALID_FORMAT_OF_LINE_MOUSE" en el caso de que la línea leída del archivo abierto no represente un ratón. Además, en el caso de que haya un problema al crear el ratón, se atrapa la excepción que el constructor de ratón haya lanzado y se lanza una "IllegalArgumentException".
- Método "savePopulation": lanza 'CSVExceptions' de los tipos "NULL_POPULATION", "NO_FILE_OPENED" y "POPULATION_WAS_NEVER_SAVED" para los casos en los que todavía no se haya creado una población, no se haya abierto un archivo o no se haya guardado como, respectivamente.
- Método "savePopulationAs": lanza 'CSVExceptions' de los tipos "NULL_POPULATION", "NO_FILE_OPENED" y "POPULATION_WAS_ALREADY_SAVED_AS" para los casos en los que todavía no se haya creado una población, no se haya abierto un archivo o ya se haya guardado como, respectivamente.

- **Clase 'Family':**

- En el constructor, se comprueba si el conjunto de madres está vacío y, de ser verdadero, se lanza una 'FamilyExceptions' del tipo "NO_MOTHER_ASSIGNED". Además, se comprueba si el sexo del padre y de la/s madre/s es correcto y, en caso contrario se lanza otra 'FamilyExceptions' de los tipos "INVALID_GENDER_FATHER" e "INVALID_GENDER_MOTHER", respectivamente.
- Las clases hijas de 'Family', al heredar de la misma, llamarán en sus respectivos constructores al constructor de la clase padre por lo que, las pruebas realizadas en el constructor de 'Family' aplicarán para todos los objetos de las clases hijas. Es por esto que, las pruebas realizadas en el constructor de 'Family' son aquellas que son

comunes a todos los casos, independientemente de si una familia es estéril o polígama.

- Todas las clases hijas de esta clase, en el método “reproduce” comprueban si la madre es estéril y, si lo es, determinan que la cantidad de crías producidas será de 0.

- **Clase ‘NormalFamily’:**

- En el constructor de esta clase se comprueba si el macho asignado es fértil y no polígamo. En caso de no serlo, se lanzan ‘FamilyExceptions’ de los tipos “NO_FERTILE_MALE_ASSIGNED” y “MUTATION_MALE_HAS_POLYGAMY”, respectivamente.

- **Clase ‘PolygamousFamily’:**

- En el constructor se comprueba que el padre sea fértil y polígamo. En caso de no serlo, se lanzan ‘FamilyExceptions’ de los tipos “MUTATION_MALE_HAS_STERILITY” y “MUTATION_MALE_LACKS_POLYGAMY”, respectivamente.

- **Clase ‘SterileMaleFamily’:**

- En el constructor se comprueba que el ratón fértil no sea nulo. En el caso de que no lo sea, se comprueba a su vez que sea fértil, no polígamo y macho, lanzando, según corresponda, ‘FamilyExceptions’ de los tipos “NO_FERTILE_MALE_ASSIGNED”, “MUTATION_MALE_HAS_POLYGAMY” e “INVALID_GENDER_FATHER”. Además, se comprueba que el padre estéril sea estéril y no polígamo, lanzando respectivamente las excepciones de los tipos “MUTATION_MALE_LACKS_STERILITY” y “MUTATION_MALE_HAS_POLYGAMY”. En el caso de que el ratón fértil fuera null, la familia se construye igual pero, la/s hembra/s no producirá/n crías. Es decir, las posibilidades de que una familia con padre estéril produjera crías, en mi programa, dependen de si se encontró o no un macho fértil que pueda embarazar a la hembra.
- El método “reproduce” de esta clase, además de comprobar si la hembra es estéril, vuelve a comprobar si el ratón “fértil” (el macho sustituto) es null ya que, en cualquiera de los dos casos, la cantidad de crías producidas por la familia sería 0.

- **Clase ‘SterilePolygamousMaleFamily’:**

- Las pruebas realizadas en esta clase son las mismas que en el caso de la clase ‘SterileMaleFamily’. Es decir, si el macho “fértil” no es null, se comprueba que sea no estéril, polígamo y macho, lanzando las excepciones que correspondan en el caso de que no cumpla alguno de estos requisitos; en este caso, si el macho sustituto no es polígamo se lanza la excepción “MUTATION_MALE_LACKS_POLYGAMY”. Al igual que el caso anterior, si el ratón macho “fértil” es null, la familia se construirá ya que, esto implica que la madre no quedará embarazada.
- En esta clase, el método “reproduce” también comprueba si la madre es estéril y el macho sustituto es null, estableciendo el número de crías producidas a 0 en el caso de que cualquiera de las dos condiciones fueran verdaderas.

- **Clase ‘Main’: AGREGAR MÉTODOS QUE COMPRUEBAN COSAS**

- Continuidad del programa: establecí una variable de tipo boolean que es true siempre y cuando el usuario seleccione una opción del menú distinta a 0 (que indica su deseo

de salir de la aplicación). Esa variable se establece a false cuando el usuario ingresa un 0 y el programa deja de funcionar.

- En los casos (que especifican las distintas opciones de operaciones que se pueden realizar), se comprueba, cuando es necesario, que el usuario haya creado una población y/o que le haya agregado al menos un ratón. Esto determina la continuidad del programa ya que, si el usuario desea eliminar un ratón antes de crear una población y/o agregarle al menos un ratón, no podrá hacerlo y se le pedirá que vuelva a ingresar el número correspondiente a la operación que desee hacer.
- Con este objetivo, cree dos funciones “isListOfMiceEmpty” y “isPopulationNull” que comprueban, respectivamente, si la lista de ratones de una población está vacía (lanzando una ‘InputException’ del tipo “EMPTY_LIST_OF_MICE” en caso de que estuviera vacía) y si se creó una población (lanzando otra ‘InputException’ del tipo “NULL_POPULATION”). Estas funciones son llamadas en los casos que implementan determinado código dependiendo de la opción del menú que el usuario haya escogido, tales como cuando se desea agregar un ratón (el usuario debe haber creado o abierto una población), listar los ratones, eliminar un ratón, modificar la información de un ratón, ver su información o simular la evolución de la población (casos para los cuales ambas funciones son llamadas ya que, para seguir adelante es necesario que el usuario haya creado o abierto una población y haya agregado al menos un ratón).
- A su vez, el main cuenta con algunos atributos estáticos cuyo objetivo es fomentar la continuidad del programa correctamente. Uno de ellos es de tipo ‘Population’ y sirve para almacenar la población que el usuario haya creado/abierto o la población virtual que se haya generado. En caso de que fuera null, implica que no se ha creado ni abierto ninguna por lo que, el usuario sólo podrá utilizar las opciones 1, 2 y 3. Además generé un string que almacena el filePath que almacena el filePath introducido para abrir el archivo y luego el introducido para guardar el archivo como, que luego es utilizado en el método “save”. Otro atributo involucrado en el funcionamiento de archivos es uno de tipo boolean que es true una vez se ha abierto un archivo y false cuando no se ha abierto ninguno. Este atributo es muy útil para los casos “save as” y “save” ya que, si no se ha abierto un archivo (si el atributo es falso) no se deben implementar los métodos de guardado. Por último, generé otro atributo de tipo boolean con el objetivo de identificar si un archivo ha sido “guardado como” o no; este atributo es falso hasta que se guarda como, caso en el cual se establece como ‘true’. Esto es útil en el caso 10 (guardar el archivo), ya que compruebo si el atributo ‘savedAs’ es verdadero o falso, invocando al método “savePopulationAs” cuando es falso y siguiendo adelante (guardando el archivo) si el atributo es true. Lo contrario ocurre en el método “savePopulationAs”, donde si el atributo es falso se sigue adelante guardando como el archivo pero, si el mismo es verdadero, se invoca al método “savePopulation”.

1.4. Qué técnicas de ordenación y búsqueda ha utilizado y por qué lo ha hecho.

- **Técnicas de búsqueda:**

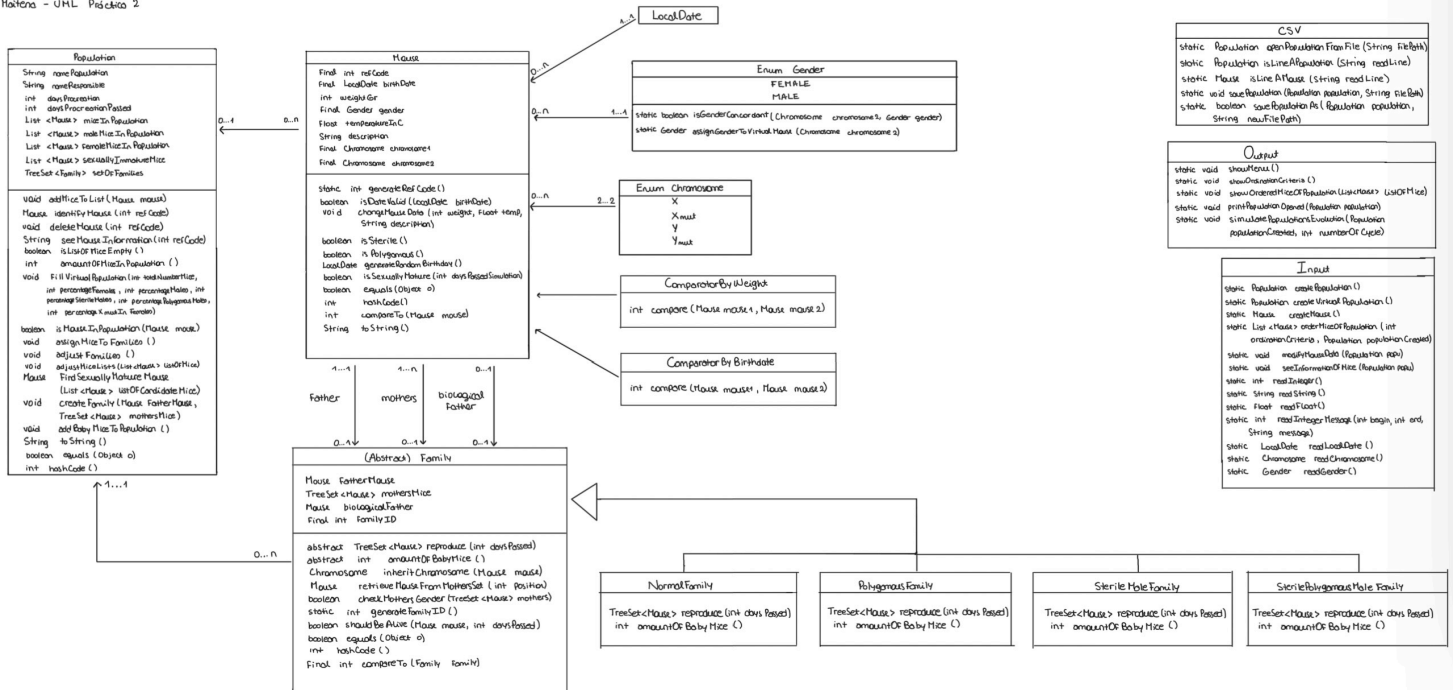
- Método “identifyMouse” de ‘Population’: en el caso de identificar a un ratón dentro de un array de ratones de una población a través de su código de referencia, implementé la

técnica de búsqueda binaria, ya que creí que era la que mejor se aplicaba a los objetivos de mi método. Ya que mi lista de ratones se encuentra ordenada naturalmente dependiendo los códigos de referencia de los ratones (los cuales son únicos por lo cual, no podemos tener más de un ratón con un mismo código de referencia), llegué a la conclusión de que la técnica mencionada sería la menos costosa y más efectiva al utilizar listas grandes. Con este fin, realicé un while que ajustara los intervalos ('start', 'end' y 'middle') dentro de los cuales se sitúa el ratón cuyo código de referencia recibimos por parámetro. Una vez encontrado, el ratón es devuelto. En cambio, si no se encuentra quiere decir que ese ratón no forma parte de la población por lo que se lanza una 'PopulationException' del tipo "MOUSE_NOT_FOUND".

- Método "isMouseInPopulation" de 'Population': este método utiliza una búsqueda secuencial para encontrar un ratón en la lista 'miceInPopulation' comparando cada ratón con el que recibe por parámetro. Esto se hace mediante el método "equals" de 'Mouse'. En el caso de que los ratones fueran iguales, se devuelve 'true' (lo cual indica que el ratón ya está presente en la lista). Si el ratón no está en la lista, se devuelve 'false'. Si bien es más costosa que otros tipos de búsquedas, consideré que para su finalidad era la más sencilla de aplicar.

- **Técnicas de ordenación:**

- Orden al eliminar un ratón de 'Population': al almacenar los ratones en un 'Array List' y utilizar el método de dicha clase "remove", cada vez que elimino un ratón de la lista, los elementos que le seguían son desplazados hacia la izquierda. Como consecuencia, la lista sigue estando ordenada.
- Método "orderMicePopulation" de 'Input': este método utiliza "Collections.sort()" para ordenar la lista que contiene los ratones de la población, según diferentes criterios. El criterio de ordenación es determinado según el número que haya introducido el usuario como input.
 - En el caso de que el usuario haya introducido un 1, los ratones serán ordenados según el orden natural que establecí. Es decir, en orden creciente de sus códigos de referencia. Para obtener esto, hice que la clase 'Mouse' implementara la interfaz 'Comparable' y, por ende, su método "compareTo()" el cual a su vez implementa el método que sobrescribí "equals()" (así como el "hashCode()").
 - Si el número introducido es un 2, entonces los ratones serán ordenados según su fecha de nacimiento. Para este fin, un comparador personalizado ("ComparatorByBirthdate") es utilizado. El comparador compara las fechas de nacimiento de los ratones y los ordena de mayor a menor.
 - Finalmente, si el número es 3, los ratones serán ordenados según su peso. En este caso también se utiliza un comparador personalizado ("ComparatorByWeight") pero, éste compara los pesos de ratones, ordenándolos en orden creciente.
 - Cabe destacar que, cuando se ordena los ratones según su fecha de nacimiento o peso y dos ratones presentan el mismo valor, se recurre al orden natural de ordenación de los ratones. Es decir, el ratón con un código de referencia menor figurará exactamente arriba (en la lista se lo almacena una posición antes) del ratón con el código de referencia.
 - Los comparadores fueron generados para poder establecer una forma personalizada de ordenar los ratones de la manera más sencilla.



2. Listado de fallos conocidos y funcionalidades definidas en el enunciado que no se han implementado en el código entregado.

- El enunciado indica que por cada ratón que forme parte de una población, deberá poder indicarse un código de referencia. En mi programa, el código de referencia no es indicado, es decir, el usuario no puede ingresar o decidir cuál será el código de referencia del ratón, si no que a cada ratón se le asigna un código único en la función "generateRefCode" de la clase ratón. Decidí hacerlo de esta manera para evitar que los códigos de referencia pudieran repetirse.
- No todos los métodos y funciones tienen tests. Sé que en ciertos casos es un poco redundante realizar tests y es por esto que algunas funciones no los tienen.
- A la hora de representar los porcentajes de ratones en la población, no realicé un gráfico de barras como era recomendado, si no que imprimí los porcentajes separados por el número de ciclo de reproducción del que se tratase en la consola.
- Adicionalmente, si bien no formaba parte del enunciado, en clase se definió un sexto tipo de familia caracterizada por ratones hembras estériles. Sin embargo, decidí no incluir este tipo de familia en mi práctica y, que el tipo de familia al que se asignaran ratones dependiera exclusivamente del ratón macho. Es decir, analicé las mutaciones que los ratones machos tenían y, en base a eso, los asigné a una u otra familia. De esta manera, las hembras eran asignadas a las familias que les tocara y su fertilidad era analizada a la hora de determinar cuántas crías produciría la familia.

3. Listado de todo el código fuente de la aplicación organizado por paquetes (si aplica) y clases.

A. Paquete Main

I. Clase MainAnimalLab

```
package aed.Main;

import aed.Exceptions.CSVExceptions;
import aed.Exceptions.FamilyExceptions;
import aed.Exceptions.InputExceptions;
import aed.Exceptions.PopulationExceptions;
import aed.InputOutput.Input;
import aed.InputOutput.Output;
import aed.Laboratory.Mouse;
import aed.Laboratory.Population;
import aed.CSVAdministration.CSV;

import java.util.List;

public class MainAnimalLab {
    //I create a static instance of the population class which I set to null
    private static Population populationCreated = null;
    public static String filePath = null;
    private static boolean openedFile = false;
    private static boolean savedAs = false;

    public static void main(String[] args) {
        //I create an integer variable that stores the input the user introduces when asked what
        option of the menu wishes to perform
        int menuOption;
        //Boolean variable I create in order to stop the program when the user wants to exit it
        boolean keepRunning = true;
        //The while runs until the user chooses the option 0 of the menu, which is when the
        "keepRunning" boolean variable is set to false and the while loop is exited
        while (keepRunning) {
            Output.showMenu(); //I call the function that shows the menu to the user
            menuOption = Input.readInteger(); //read the input introduced by the user
            switch (menuOption) {
                case 1:
                    //I ask the user to introduce the path of the CSV file
                    System.out.println("Please introduce the path of the file you want to open: ");
                    filePath = Input.readString();
                    //To use as an example in my computer "/Users/abrilbetti/Desktop/Algoritmos/
                    Programas/Practica/populationTextFile.CSV";
                    //I call the method that opens CSV files and pass as a parameter the path of my file
                    populationCreated = CSV.openPopulationFromFile(filePath);
                    if (populationCreated != null) {
                        Output.printPopulationOpened(populationCreated);
                    }
                }
            }
        }
    }
}
```

```

        //If the population is != null it is because we successfully opened the file. Therefore,
        I set the variable 'openedFile' to true.
        openedFile = true;
        //I print a message showing the operation was successful
        System.out.println("The population has been successfully opened and loaded.");
    }
    break;
case 2: //Create a new population
    populationCreated = Input.createPopulation();
    if (populationCreated != null) {
        System.out.println("The population has been created successfully.");
        Output.printPopulationOpened(populationCreated);
    }
    break;
case 3: //Create a virtual population
    populationCreated = Input.createVirtualPopulation();
    if (populationCreated != null) {
        System.out.println("The virtual population has been created successfully.");
        Output.printPopulationOpened(populationCreated);
    }
    break;
case 4: //Add a new mouse to an existing population
    try {
        //I check that the user has created a population
        isPopulationNull(populationCreated);
        Mouse mouseCreated = Input.createMouse();
        populationCreated.addMiceToList(mouseCreated);
        System.out.println("The mouse has been created and added successfully to the
population.");
    } catch (PopulationExceptions | InputExceptions pe) {
        System.out.println("ERROR: " + pe);
    }
    break;
case 5: //List the mice of the population
    try{
        //I check that the user has created a population and added at least one mouse to t.
        isPopulationNull(populationCreated);
        isListOfMiceEmpty(populationCreated);
        //If the user has already created a population and at least one mouse has been
added to it , we list the reference codes of the mice/mouse.
        //I call the function that outputs the different ways in which the mice can be
ordered and read the number the user introduces by calling the appropriate function of the input
class
        Output.showOrdinationCriteria();
        int ordinationCriteria = Input.readInteger();
        //Once I read the type of ordination, I call the function that orders the mice of the
population and returns a copy of the array list which is ordered according to the chosen criteria.
        List<Mouse> listOfMice = Input.orderMiceOfPopulation(ordinationCriteria,
populationCreated);
        System.out.println("Sorted mice of the population: ");

```

```

        //I call the function that prints the mice of the population sorted with the type of
order the user chose
        Output.showOrderedMiceOfPopulation(listOfMice);
    } catch (InputExceptions ie){
        System.out.println("ERROR: " + ie);
    }
    break;
case 6: //Delete a mouse from a population
    /*We will only delete mice (or try to, because the mouse the user wants to delete may
not exist) if the user has already created a population
and if it has added at least one mouse to the population*/
    try{
        isPopulationNull(populationCreated);
        isLisOfMiceEmpty(populationCreated);
        System.out.println("Introduce the reference code of the mouse you wish to
eliminate from the population (integer): ");
        int refCodeOfMouseToEliminate = Input.readInteger();
        try {
            //I call the method that eliminates mice from a population and pass as parameter
the reference code the user introduced
            populationCreated.deleteMouse(refCodeOfMouseToEliminate);
            System.out.println("The mouse has been successfully eliminated from the
population.");
        } catch (PopulationExceptions pe) {
            System.out.println("ERROR: " + pe);
        }
    } catch (InputExceptions ie){
        System.out.println("ERROR: " + ie);
    }
    break;
case 7: //Modify a mouse's data
    //If the population exists and mice has been added to it, we go on to modify a
mouse's data
    try {
        isPopulationNull(populationCreated);
        isLisOfMiceEmpty(populationCreated);
        //I call the function that modifies the mouse's data
        Input.modifyMouseData(populationCreated);
    } catch (InputExceptions ie){
        System.out.println("ERROR: " + ie);
    }
    break;
case 8: //View detailed information about a mouse
    //If the user has created a population and at least 1 mouse has been added to it, we
go on
    try {
        isPopulationNull(populationCreated);
        isLisOfMiceEmpty(populationCreated);
        //I call the function that prints the information of the mouse
        Input.seelInformationOfMice(populationCreated);
    } catch (InputExceptions ie){

```

```

        System.out.println("ERROR: " + ie);
    }
    break;
case 9: //Simulate the evolution of the population over time
    //If the user has created a population and at least 1 mouse has been added to it, we
go on
    try {
        isPopulationNull(populationCreated);
        isListOfMiceEmpty(populationCreated);
        //For that runs as many times as cycles the introduced days in lab represent (by
dividing the amount of days by 45)
        for (int i = 0; i < populationCreated.getDaysProcreation() / 45; i++) {
            try {
                populationCreated.assignMiceToFamilies();
                populationCreated.addBabyMiceToPopulation();
            } catch (FamilyExceptions fe) {
                System.out.println("ERROR: " + fe);
            }
            //I call the method that computes and prints the statistics of the population's
mice.
            Output.simulatePopulationsEvolution(populationCreated, i);
        }
    } catch (InputExceptions ie) {
        System.out.println("ERROR: " + ie);
    }
    break;
case 10: //Save
    try {
        CSV.savePopulation(populationCreated, filePath, openedFile, savedAs);
        System.out.println("The population was correctly saved in the file you indicated!");
    } catch (CSVExceptions | InputExceptions ie) {
        System.out.println("ERROR: " + ie);
    }
    if (!savedAs && openedFile) {
        System.out.println("Introduce the file path where you want to save your population:
");
        String readPath = Input.readString();
        CSV.savePopulationAs(populationCreated, readPath, openedFile, savedAs);
        savedAs = true;
        System.out.println("The population was correctly saved in the file you indicated!");
    }
    break;
case 11: //Save as
    System.out.println("Enter the new file path: ");
    String newFilePath = Input.readString();
    try {
        CSV.savePopulationAs(populationCreated, newFilePath, openedFile, savedAs);
        System.out.println("The population was correctly saved in the file you indicated!");
        //If function 'save as' was performed successfully, I update the value of the 'current
path' variable by setting it to the new file path the user introduced.

```

```

        //I also print a message in order to let the user know that the operation was
        performed correctly, and set 'savedAs' as true.
        filePath = newFilePath;
        savedAs = true;
    } catch (CSVExceptions | InputExceptions ie) {
        System.out.println("ERROR: " + ie);
    }
    if(savedAs){
        CSV.savePopulation(populationCreated, newFilePath, openedFile, savedAs);
        System.out.println("The population was correctly saved in the file you indicated!");
    }
    break;
case 0: //EXIT (end programme)
    System.out.println("Closing the program...");
    // I set the boolean "keepRunning" variable to false in order to stop the while loop
and finish the program
    keepRunning = false;
    break;
default:
    //If the integer introduced by the user is none of the cases presented before, I print a
message
    System.out.println("The option you chose is invalid. Try again, now entering a valid
number (from 0 to 9).");
}
}
}

/**
 * Function that checks if any mice had been added to the population it receives as parameter.
 * @param population object of Population.
 * @throws InputExceptions when the list of mice of the population is empty.
 */
private static void isListOfMiceEmpty(Population population) throws InputExceptions {
    if(population.isListOfMiceEmpty()){
        throw new InputExceptions(InputExceptions.ErrorTypeInput.EMPTY_LIST_OF_MICE);
    }
}

/**
 * Function that checks if a population has been created, by analysing if it is equal or not to null.
 * @param population object of Population.
 * @throws InputExceptions when the population is equal to null (it has not been created).
 */
private static void isPopulationNull(Population population) throws InputExceptions {
    if(population == null){
        throw new InputExceptions(InputExceptions.ErrorTypeInput.NULL_POPULATION);
    }
}
}
}

```

B. Paquete **Laboratory**

I. Clase **Population**

```
package aed.Laboratory;

import aed.Exceptions.FamilyExceptions;
import aed.Exceptions.MouseExceptions;
import aed.Exceptions.PopulationExceptions;
import aed.Families.*;

import java.time.LocalDate;
import java.util.*;

/**
 * Population class defines the "blueprint" that its objects (populations) will have in this program.
 * The class is composed of 5
 * attributes that define the class' state and several methods that define its behaviour.
 * @author abrilbetti
 */
public class Population{
    private String namePopulation;
    private String nameResponsible;
    private int daysProcreation;
    private int daysProcreationPassed = 0;
    private List<Mouse> micelInPoupulation = new ArrayList<>(); //I create an array list of mice
    private List<Mouse> femaleMiceInPopulation = new ArrayList<>();
    private List<Mouse> maleMiceInPopulation = new ArrayList<>();
    private List<Mouse> sexuallyImmatureMice = new ArrayList<>();
    private TreeSet<Family> setOfFamilies = new TreeSet<>();

    //Constructor of Population class
    /**
     * Constructor of the population class, which receives as parameter a value per each attribute
     * of the class which are used in order to initialise them.
     * The constructor checks that the values of certain variables received as parameters (such as
     * the days of procreation) are valid before initialising the
     * attributes of the object.
     * @param namePopulation string variable that stores the name of the population.
     * @param nameResponsible string variable that stores the name of the responsible for the
     * population.
     * @param daysProcreation integer variable that stores the amount of days a population can
     * spend in a laboratory in order to procreate.
     * @throws PopulationExceptions an exception is rethrown if any of the called methods throws
     * one
     */
    public Population(String namePopulation, String nameResponsible, int daysProcreation) throws
    PopulationExceptions {
```

Abril Maitena Betti

```

        setNamePopulation(namePopulation);
        setNameResponsible(nameResponsible);
        setDaysProcreation(daysProcreation);
    }

```

```

/**

```

* Constructor of the population class, which is used in order to create a virtual population. The constructor receives as parameter the name of the population and its responsible, the days the population will spend in the laboratory and many percentages which describe

* the composition of the virtual population.

* @param namePopulation string that stores the introduced name of the population.

* @param nameResponsible string that stores the introduced name of the responsible for the population.

* @param daysProcreation int that stores the amount of days that the population will spend in the laboratory in order to procreate.

* @param numberOfMice integer that stores the total amount of mice that the population has to have.

* @param percentageMales integer that stores the percentage of male mice that the population has to have.

* @param percentageSterileMales integer that stores the percentage of sterile male mice that the population has to have.

* @param percentagePolygamousMales integer that stores the percentage of polygamous male mice that the population has to have.

* @param percentageFemales integer that stores the percentage of female mice that the population has to have.

* @param percentageXmutInFemales integer that stores the percentage of mutated X chromosomes that there have to be among the female mice of the population.

* @throws PopulationExceptions is any of the percentages or the total number of mice is invalid. It also rethrows the exceptions the setters throw, if any.

```

*/

```

```

    public Population(String namePopulation, String nameResponsible, int daysProcreation, int
    numberOfMice, int percentageMales, int percentageSterileMales, int
    percentagePolygamousMales, int percentageFemales, int percentageXmutInFemales) throws
    PopulationExceptions {

```

```

        setNamePopulation(namePopulation);
        setNameResponsible(nameResponsible);
        setDaysProcreation(daysProcreation);

```

```

        if(numberOfMice <= 0)
        {

```

```

            throw new
            PopulationExceptions(PopulationExceptions.ErrorType.INVALID_NUMBER_OF_MICE);
        }

```

```

        if((percentageMales <= 0 || percentageMales >= 100)
        {

```

```

            throw new
            PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PERCENTAGE_MALES);
        }

```

```

        if((percentageFemales <= 0 || percentageFemales >= 100){
            throw new

```

```

            PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PERCENTAGE_FEMALES);
        }

```

Abril Maitena Betti

```

        if(percentagePolygamousMales < 0 || percentagePolygamousMales >= 100){
            throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PERCENTAGE_POLYGAMOUS_
MALES);
        }
        if(percentageSterileMales < 0 || percentageSterileMales >= 100){
            throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PERCENTAGE_STERILE_MALES
);
        }
        if(percentageXmutInFemales < 0 || percentageXmutInFemales >= 100){
            throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PERCENTAGE_XMUT);
        }
        if(percentageMales + percentageFemales != 100){
            throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_MALE_FEMALE_PERCENTAGES)
;
        }
    }

//Getters
/**
 * Getter for the name of the population.
 * @return a string variable that stores the name of the population.
 */
public String getNamePopulation() {
    return namePopulation;
}

/**
 * Getter for the days the population will be in the lab.
 * @return an integer variable that stores the amount of days the population will spend in the
lab.
 */
public int getDaysProcreation() {
    return daysProcreation;
}

/**
 * Getter for the name of the responsible for the population.
 * @return a string variable that stores the name of the responsible for the population.
 */
public String getNameResponsible() {
    return nameResponsible;
}

/**
 * Getter of a copy of the array list that stores the mice present in the population.
 * @return the copy of the array list that contains the mice of the population.
 */

```

Abril Maitena Betti

```

public List<Mouse> getCopyOfMicelnPopulation() {
    //I create the copy of the array list that contains the mice of the population
    ArrayList<Mouse> copiedListOfMicelnPopulation = new ArrayList<>();
    //I add the objects stored in the original list of mice in the population to the copy list
    copiedListOfMicelnPopulation.addAll(micelnPoupulation);
    //I return the copy of the array list in order tp avoid violating the encapsulation principle of
the original array list that contains the mcie
    return copiedListOfMicelnPopulation;
}

```

/**

* Method that returns the amount of female mice in the population by going over the list that stores all the mice in the population and counting how many of them are female.

* @return the counter that is incremented each time a female mouse is found in the list.

*/

```

public int getAmountFemaleMice(){
    int counterFemales = 0;
    for(int i = 0; i < micelnPoupulation.size(); i++){
        if (micelnPoupulation.get(i).getGender() == Gender.FEMALE){
            counterFemales++;
        }
    }
    return counterFemales;
}

```

/**

* Method that returns the amount of male mice in the population by going over the list that stores all the mice in the population and counting how many of them are male.

* @return the counter that is incremented each time a male mouse is found in the list.

*/

```

public int getAmountMaleMice(){
    int counterMales = 0;
    for(int i = 0; i < micelnPoupulation.size(); i++){
        if (micelnPoupulation.get(i).getGender() == Gender.MALE){
            counterMales++;
        }
    }
    return counterMales; }
}

```

/**

* Method that returns the amount of sterile male mice in the population by going over the list that stores the mice of the population and counting how many of them are male and sterile.

* @return an integer that represents how many sterile mice there are among the male population.

*/

```

public int getAmountSterileMaleMice(){
    int counterSterileMales = 0;
    for(int i=0; i<micelnPoupulation.size(); i++){
        if(micelnPoupulation.get(i).getGender() == Gender.MALE &&
micelnPoupulation.get(i).isSterile()){
            counterSterileMales++;
        }
    }
}

```

Abril Maitena Betti

```

    return counterSterileMales;
}
/**

```

* Method that returns the amount of polygamous male mice in the population by going over the list that stores the mice and counting how many of them are polygamous and male.

* @return an integer that represents how many polygamous mice there are among the male population.

```

*/
public int getAmountPolygamousMaleMice(){
    int counterPolygamousMales = 0;
    for(int i=0; i<micelnPoupulation.size(); i++){
        if(micelnPoupulation.get(i).isPolygamous()){
            counterPolygamousMales++;
        }
    }
    return counterPolygamousMales;
}
/**

```

* Method that returns the amount of normal male mice in the population by going over the list that stores the mice of the population and counting how many of them are non-sterile, not polygamous and male.

* @return an integer that represents how many normal mice there are among the male population.

```

*/
public int getAmountNormalMaleMice(){
    int counterNormalMales = 0;
    for(int i=0; i<micelnPoupulation.size(); i++){
        if(micelnPoupulation.get(i).getGender() == Gender.MALE && !
micelnPoupulation.get(i).isPolygamous() && !micelnPoupulation.get(i).isSterile()){
            counterNormalMales++;
        }
    }
    return counterNormalMales;
}

```

```

/**

```

* Method that returns the amount of sterile female mice in the population by going over the list that stores the mice of the population and counting how many of them are sterile and female.

* @return an integer that represents how many sterile mice there are among the female population.

```

*/
public int getAmountSterileFemaleMice(){
    int counterSterileFemales = 0;
    for(int i=0; i<micelnPoupulation.size(); i++){
        if(micelnPoupulation.get(i).getGender() == Gender.FEMALE &&
micelnPoupulation.get(i).isSterile()){
            counterSterileFemales++;
        }
    }
    return counterSterileFemales;
}

```

```

/**
 * Method that returns the amount of fertile female (that have one mutated chromosome) mice
 in the population by going over the list that stores the mice of the population and counting how
 many of them fulfill this.
 * @return an integer that represents how many fertile mice that have one mutation there are
 among the female population.
 */
public int getAmountFemaleMiceWithOneMutation(){
    int counterXmutFemales = 0;
    for(int i=0; i<micelnPoupulation.size(); i++){
        int counterMutations = 0;
        if(micelnPoupulation.get(i).getGender() == Gender.FEMALE &&
micelnPoupulation.get(i).getChromosome1() == Chromosome.Xmut){
            counterMutations++;
        } else if(micelnPoupulation.get(i).getChromosome2() == Chromosome.Xmut){
            counterMutations++;
        }
        if(counterMutations == 1){
            counterXmutFemales++;
        }
    }
    return counterXmutFemales;
}

```

//Setters for the name of the population and its responsible, and the days of procreation of the population.

```

/**
 * Setter for the name of the responsible. In order for the value stored in the variable received as
 a parameter to be set as the name of
 * the responsible, it needs to be composed only by letters. If it includes numbers or other
 invalid characters, an exception is thrown.
 * @param nameResponsible variable that stores the string we want to set as the number of the
 responsible.
 * @throws PopulationExceptions an exception of type "invalid name" is thrown if the string
 received as parameter is not only composed of letters.
 */
public void setNameResponsible(String nameResponsible) throws PopulationExceptions {
    //I check that the string introduced by the user is composed only by letters (upper or lower
case)
    // ^ and $ state that this applies to all the string introduced by the user (these symbols mark
the beginning and end of the string)
    //The + indicates that the patter must match at least one character (in this way, empty strings
will be rejected too)
    if(nameResponsible.matches("[a-zA-Z]+$")) {
        this.nameResponsible = nameResponsible;
    }
    else{
        throw new PopulationExceptions(PopulationExceptions.ErrorType.INVALID_NAME);
    }
}

```

Abril Maitena Betti

```

/**
 * Setter for the name of the population.
 * @param namePopulation variable of string type that stores the new name of the population.
 */
public void setNamePopulation(String namePopulation) {
    this.namePopulation = namePopulation;
}

/**
 * Setter for the days in the lab that a population will spend. In order to set a value to this
attribute, it needs to fulfill certain conditions such as:
 * being greater than 0, lower than 630 and a multiple of 45.
 * @param daysProcreation integer variable that stores the value we want to set as the
'daysProcreation'.
 * @throws PopulationExceptions an exception of type 'invalid procreation days' or 'non-
multiple of 45 procreation days'
 * is thrown if the value we want to set for this attribute does not meet the aforementioned
requirements.
 */
public void setDaysProcreation(int daysProcreation) throws PopulationExceptions {
    //I create an if-else in order to check that the days of procreation value introduced by the
user is among the valid range of numbers (days)
    if(daysProcreation > 0 && daysProcreation < 631 && daysProcreation%45==0){
        this.daysProcreation = daysProcreation;
    }
    //If the value is not between the expected range or is not a number, then I throw an exception
of type "invalid procreation days"
    else if(daysProcreation <= 0 || daysProcreation > 630){

        throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PROCREATION_DAYS);
    }
    //If the introduced number is not a multiple of 45 then I throw the appropriate type of
exception.
    else if(daysProcreation%45 != 0){
        throw new
PopulationExceptions(PopulationExceptions.ErrorType.NON_MULTIPLE_45_PROCREATION_DAY
S);
    }
    //In any other case, I throw an exception that states that the data type introduced is invalid
(not an int)
    else{
        throw new
PopulationExceptions(PopulationExceptions.ErrorType.INVALID_PROCREATION_DAYS);
    }
}

/**
 * Method that adds the mouse (passed as parameter) to the population by adding it to the
appropriate lists, which is determined by the gender of the mouse.

```

```

    * @param mouse instance of Mouse which represents a virtual mouse which was created in
    the 'fillVirtualPopulation' method.
    * @throws PopulationExceptions if the mouse we are trying to add is equal to null.
    */
    public void addMiceToList(Mouse mouse) throws PopulationExceptions {
        if(mouse == null){
            throw new
PopulationExceptions(PopulationExceptions.ErrorType.FAILED_TO_ADD_MOUSE);
        }
        if(!isMouseInPopulation(mouse)) {
            //I add the created mice to the array list that contains them all (regardless of their sex) only
            if it is not present in it.
            micelnPoupulation.add(mouse);
        }
        //I add the created mouse to the appropriate list depending on their gender
        if (mouse.getGender() == Gender.MALE) {
            maleMicelnPopulation.add(mouse);
        }
        if (mouse.getGender() == Gender.FEMALE) {
            femaleMicelnPopulation.add(mouse);
        }
    }

    /**
    * Method that runs the ArrayList and looks for the mouse with the reference code it receives as
    parameter. If it is found it returns
    * the respective mouse and, if it is not, it throws an exception. This method implements binary
    search.
    * @param refCode of the mouse the user wants to look for.
    * @return the mouse if the reference code is associates with one.
    * @throws PopulationExceptions if a mouse with the reference code is not found.
    */
    public Mouse identifyMouse(int refCode) throws PopulationExceptions{
        //I define 3 integer variables: start, end (the size of the list of mice) and middle.
        int start = 0;
        int end = micelnPoupulation.size();
        int middle = (start + end) / 2;
        //While the start number is smaller than the 'end' one and the reference code of the mouse
        stored in the position 'middle' of the list is not the one we are looking for, we continue the search.
        while((start <= end) && micelnPoupulation.get(middle).getRefCode() != refCode){
            //If the reference code we are looking for is smaller than that of the mouse stored in
            position 'middle', we set the 'end' to one position before the middle.
            if(refCode < micelnPoupulation.get(middle).getRefCode()){
                end = middle - 1;
            } else if(refCode > micelnPoupulation.get(middle).getRefCode()){
                //I set the 'start' position to one position after the middle if the reference code we are
                looking for is not smaller than the one stored in the 'middle' position.
                start = middle + 1;
            }
            //I update the value of 'middle' to the middle number between the new end and start.
            middle = (start + end) / 2;

```

Abril Maitena Betti

```

    }
    //I check that the reference code is equal to that stored in the 'middle' position and, if it is, I
return the mouse stored in that position.
    if(refCode == micelnPoupulation.get(middle).getRefCode()){
        return micelnPoupulation.get(middle);
    }
    //If the mouse was not found (because it doesn't exist a mouse with the introduced reference
code), an exception is thrown.
    throw new PopulationExceptions(PopulationExceptions.ErrorType.MOUSE_NOT_FOUND);
}

/**
 * Method that removes a mouse from the population by removing it from the dynamic list. This
method receives a reference code, calls the method
 * that identifies the mouse whose reference code is the one provided by the user and
eliminates the mouse from the list of mice of the population.
 * @param refCode integer that stores the reference code of the mouse that the user wishes to
delete.
 * @throws PopulationExceptions if the mouse is not found and the method 'identifyMouse'
throws an exception of this type; then this method rethrows the exception.
 * Also, if the mouse is found but not removed from the population, an exception is thrown.
 */
public void deleteMouse(int refCode) throws PopulationExceptions{
    Mouse mouseToRemove = identifyMouse(refCode);
    boolean removed = micelnPoupulation.remove(mouseToRemove);
    //If the mouse was found but not removed from the population (when removed is false), I
throw an exception
    if(!removed){
        throw new
PopulationExceptions(PopulationExceptions.ErrorType.FAILED_TO_REMOVE_MOUSE);
    }
}

/**
 * Method that prints all the information about the mouse that has the reference code that was
introduced as parameter.
 * @param refCode of the mouse whose information is displayed.
 * @return all the information of the mouse.
 * @throws PopulationExceptions if an exception is thrown in the identify mouse method.
 */
public String seeMouseInformation(int refCode) throws PopulationExceptions{
    //I call the method identify mouse in order to look for the mouse whose reference code is the
one the user introduced
    Mouse mouse = identifyMouse(refCode);
    return "Reference code: " + mouse.getRefCode() + "\nBirthday: " + mouse.getBirthDate() +
"\nWeight: " + mouse.getWeightGr()
        + "\nGender: " + mouse.getGender() + "\nTemperature: " + mouse.getTemperatureInC()
+ "\nDescription: " + mouse.getDescription()
        + "\nIs polygamous: " + mouse.isPolygamous() + "\nIs sterile: " + mouse.isSterile();
}

```

```

/**
 * Method that checks whether the array list that contains the mice of the population is empty
or not.
 * @return a boolean which is true when the list is empty and false when it is not.
 */
public boolean isListOfMiceEmpty(){
    return micelnPoupulation.isEmpty();
}

/**
 * Method that returns the size of the array list that stores the mice tht were added to the
population.
 * @return the length of the list that contains the mice of the population, which is dynamic and
therefore coincides with the amount of mice the population has.
 */
public int amountOfMicelnPopulation(){
    return micelnPoupulation.size();
}

/**
 * Method that receives the total number of mice in the population and the percentages and
computes how many mice we will have in the population with each condition, generates random
numbers which
 * are analysed in order to obtain the gender and chromosomes of the virtual mice (making sure
that we are creating the accurate amount of each 'type' of mouse (male and sterile, male and
polygamous, female, etc.)
 * Finally, the method calls another method which adds the mice to the appropriate array list.
 * @param totalNumberOfMice integer that stores the introduced value of the total amount of
mice in the population.
 * @param percentageFemale integer that stores the percentage of female mice that the
population has.
 * @param percentageMale integer that stores the percentage of male mice that the population
has.
 * @param percentageSterileMales integer that stores the percentage of sterile male mice that
the population has.
 * @param percentagePolygamousMales integer that stores the percentage of polygamous
male mice that the population has.
 * @param percentageXmutlnFemales integer that stores the percentage of mutated X
chromosomes among the female mice of the population.
 * @throws MouseExceptions if an exception is thrown when we call the constructor of Mouse
and the object is created (this method rethrows it)
 * @throws PopulationExceptions is the method 'addMiceToList' throws an exception, then this
method rethrows it.
 */
public void fillVirtualPopulation(int totalNumberOfMice, int percentageFemale, int
percentageMale, int percentageSterileMales, int percentagePolygamousMales, int
percentageXmutlnFemales) throws MouseExceptions, PopulationExceptions{
    //I compute the amount of mice with each condition present in the population
    int amountOfFemaleMice = totalNumberOfMice * percentageFemale / 100;
    int amountOfMaleMice = totalNumberOfMice * percentageMale / 100;
    int amountOfSterileMales = amountOfMaleMice * percentageSterileMales / 100;

```

```

    int amountOfPolygamousMales = amountOfMaleMice * percentagePolygamousMales / 100;
    int amountXmutInFemales = (amountOfFemaleMice * 2 * percentageXmutInFemales) / 100;
    // create as many counter as I need to make sure that I am creating the appropriate amount
of mice depending on the percentages
    int counterCreatedMales = 0;
    int counterCreatedFemales = 0;
    int counterCreatedSterileMales = 0;
    int counterCreatedPolygamousMales = 0;
    int counterUsedXmutInFemales = 0;
    int counterNonSterileMales = 0;
    int counterNonPolygamousMales = 0;
    int counterUsedXInFemales = 0;
    // create an instance of the Random class, which I need in order to generate 3 random
numbers between 0 and 99
    Random random1 = new Random();
    boolean keepGeneratingRandom1;
    boolean keepGeneratingRandom2;
    boolean keepGeneratingRandom3;

    //For that runs as many times as mice the population will have
    for(int i = 0; i<totalNumberOfMice; i++) {
        // create chromosomes that will store certain values depending on the random numbers I
generated
        Chromosome chromosome1 = null;
        Chromosome chromosome2 = null;
        // generate 3 random numbers between 0 and 99
        int randNum1 = random1.nextInt(100); //I am generating 3 random numbers between 0
and 99
        int randNum2 = random1.nextInt(100);
        int randNum3 = random1.nextInt(100);
        keepGeneratingRandom1 = true;
        keepGeneratingRandom2 = true;
        keepGeneratingRandom3 = true;

        //I analyse the random numbers I got, increasing the appropriate counters and initialising
the gender and chromosomes accordingly.
        /*If the first random number is smaller than the percentage of male mice and the amount
of male mice created is smaller than the
        total amount of male mice that the percentage represents, we enter the 'if' and create
another male mouse.
        */
        //While loop that runs until we have created either a female or male mouse.
        while(keepGeneratingRandom1) {
            if (randNum1 < percentageMale && counterCreatedMales < amountOfMaleMice) {
                //The mouse will be a male; I increase by one the counter of male mice already
created.
                counterCreatedMales++;
                keepGeneratingRandom1 = false;
                //I create a while loop that will iterate until a value is assigned to the first chromosome
of the mouse.
                while (keepGeneratingRandom2) {

```

```

        /*If the second random number is smaller than the percentage of sterile male mice
in the population and if the amount of sterile male mice
        I have already created is smaller than the total amount of sterile male mice I need
to have in my population, we enter the 'if' and set the
        first chromosome of the mouse as mutated (Xmut) in order for it to be sterile.
        */
        if (randNum2 < percentageSterileMales && counterCreatedSterileMales <
amountOfSterileMales) {
            //I set the first chromosome of the mouse as mutated (Xmut) and add one to the
counter of sterile males created
            chromosome1 = Chromosome.Xmut;
            counterCreatedSterileMales++;
            keepGeneratingRandom2 = false; //I set it to false in order to exit the while
        }
        /*If the second random number is equal or greater than the percentage of sterile
male mice, or if we have reached the amount of sterile male mice
        we had to build, we create non-sterile male mice. In addition, I check that I'm not
creating more non-sterile male mice than I should as the consequences
        would involve having less sterile male mice than asked. Because of that, I check
that the created amount of non-sterile males is smaller than the wanted amount
        sterile males (which is equal to the total amount of male mice minus the wanted
amount of sterile males)*/
        else if (randNum2 >= percentageSterileMales && counterNonSterileMales <
amountOfMaleMice - amountOfSterileMales) {
            //The mouse will not be sterile. chromosome1 = X
            chromosome1 = Chromosome.X;
            counterNonSterileMales++;
            keepGeneratingRandom2 = false;
        }
        /*If the type of male mice I should build according to the value of the random
number has reached the limit (for instance we've already built the wanted amount
        of sterile males) I should build a non-sterile male. However, this won't happen
because the random number does not meet the condition of the 'if'; therefore, I
        created the while and the else in which the random number's value is updated.*/
        else {
            //I update the value of the random number and keep trying to assign values to
the chromosomes of the male mouse
            randNum2 = random1.nextInt(100);
        }
    }
    /*If the third random number is smaller than the percentage of polygamous male mice
in the population and if the amount of polygamous male mice
        I have already created is smaller than the total amount of polygamous male mice I
need to have in my population, we enter the 'if' and set the
        second chromosome of the mouse as mutated (Ymut) in order for it to have a
tendency of polygamy.
        */
    while (keepGeneratingRandom3) {
        if (randNum3 < percentagePolygamousMales &&
counterCreatedPolygamousMales < amountOfPolygamousMales) {
            //The mouse is polygamous. The male chromosome is mutated (Ymut)

```

```

        chromosome2 = Chromosome.Ymut;
        //I add one to the counter of polygamous male mice created
        counterCreatedPolygamousMales++;
        keepGeneratingRandom3 = false;
    }
    /*If the third random number is equal or greater than the percentage of polygamous
male mice, or if we have reached the amount of polygamous male mice
we had to build, we create non-polygamous male mice.*/
    else if (randNum3 >= percentagePolygamousMales &&
counterNonPolygamousMales < amountOfMaleMice - amountOfPolygamousMales) {
        //The mouse is not polygamous. chromosome2 = Y
        chromosome2 = Chromosome.Y;
        counterNonPolygamousMales++;
        keepGeneratingRandom3 = false;
    } else {
        //I update the value of the third random number
        randNum3 = random1.nextInt(100);
    }
}
}
}
/*If the first random number is equal or greater than the percentage of males, the mice
will be female. I create the female mice only if I have
created less than the total amount of females I have to create according to the
percentage of female mice.*/
    else if (randNum1 >= percentageMale && counterCreatedFemales <
amountOfFemaleMice) {
        //The mouse will be female. I add one to the counter of created female mice.
        counterCreatedFemales++;
        keepGeneratingRandom1 = false;
        while (keepGeneratingRandom2) {
            /*If the second random number is smaller than the percentage of mutated X
chromosomes among the female population and if the amount of used Xmut
is smaller than the "available" amount of Xmut, we enter the 'if' and set the first
chromosome of the mouse as mutated (Xmut).*/
            if (randNum2 < percentageXmutInFemales && counterUsedXmutInFemales <
amountXmutInFemales) {
                //The first chromosome of the female mouse will be mutated. chromosome1 =
Xmut
                chromosome1 = Chromosome.Xmut;
                keepGeneratingRandom2 = false;
                //I increase by one the counter of how many mutated X chromosomes we have
assigned to female mice in the created population.
                counterUsedXmutInFemales++;
            }
            /*If the second random number is equal or greater than the percentage of mutated
X chromosomes present among the female population, or if we have
used all the mutated X chromosomes we had available, we create female mice with
normal a normal (X) 'chromosome1'.*/
            else if (randNum2 >= percentageXmutInFemales && counterUsedXInFemales < (2 *
amountOfFemaleMice - amountXmutInFemales)) {
                //The first chromosome of the female mouse will be normal. chromosome1 = X

```

```

        chromosome1 = Chromosome.X;
        counterUsedXInFemales++;
        keepGeneratingRandom2 = false;
    } else {
        randNum2 = random1.nextInt(100);
    }
}
while (keepGeneratingRandom3) {
    /*If the third random number is smaller than the percentage of mutated X
chromosomes among the female population and if the amount of used Xmut
is smaller than the "available" amount of Xmut, we enter the 'if' and set the second
chromosome of the mouse as mutated (Xmut).*/
    if (randNum3 < percentageXmutInFemales && counterUsedXmutInFemales <
amountXmutInFemales) {
        // The second chromosome of the female mouse will be mutated. chromosome2
= Xmut.
        chromosome2 = Chromosome.Xmut;
        //I increase by one the counter of how many mutated X chromosomes we have
assigned to female mice in the created population.
        counterUsedXmutInFemales++;
        keepGeneratingRandom3 = false;
    }
    /*If the third random number is equal or greater than the percentage of mutated X
chromosomes present among the female population, or if we have
used all the mutated X chromosomes we had available, we create female mice with
normal a normal (X) 'chromosome2'.*/
    else if (randNum3 >= percentageXmutInFemales && counterUsedXInFemales < (2 *
amountOfFemaleMice - amountXmutInFemales)) {
        //The second chromosome of the female mouse will be normal. chromosome2 =
X.
        chromosome2 = Chromosome.X;
        counterUsedXInFemales++;
        keepGeneratingRandom3 = false;
    } else {
        randNum3 = random1.nextInt(100);
    }
}
}
else{
    randNum1 = random1.nextInt(100);
}
}
//I make sure that the chromosomes are not null in order to build appropriate mice.
if (chromosome1!=null && chromosome2!=null) {
    //I create a mouse instance with the obtained gender and chromosomes
    Mouse mouse = new Mouse(chromosome1, chromosome2);
    addMiceToList(mouse);
}
}
}

```

```

/**
 * Method that checks if a mouse is present in a list, by calling the equals method and going
over the list of mice.
 * @param mouse Mouse instance we want to check if it has been added or not to the list.
 * @return true if the mouse is in the list or false otherwise.
 */
protected boolean isMouseInPopulation(Mouse mouse){
    for(int i=0; i<micelInPoupulation.size(); i++) {
        if(micelInPoupulation.get(i).equals(mouse)) {
            return true;
        }
    }
    return false;
}

/**
 * Method that assigns the mice stored in the array lists of female, male and sexually immature
mice in the population to the appropriate families.
 * The appropriate family for each mouse is determined by the method 'createFamily'.
 * @throws FamilyExceptions rethrows exceptions thrown when creating objects of Family in
the method 'createFamily'.
 * @throws PopulationExceptions if the method 'addMiceToList' throws an exception, this
method rethrows it.
 */
public void assignMiceToFamilies() throws FamilyExceptions, PopulationExceptions{
    daysProcreationPassed = daysProcreationPassed + 45;
    adjustFamilies();
    adjustMiceLists(maleMicelInPopulation);
    adjustMiceLists(femaleMicelInPopulation);
    //While there are mice left in the lists of female and male mice, I keep assigning them to
families.
    while(!maleMicelInPopulation.isEmpty() && !femaleMicelInPopulation.isEmpty()) {
        //I create the father mouse which will be the mouse stored in position 0 of the array list
that contains the male mice of the virtual population.
        Mouse fatherMouse = findSexuallyMatureMouse(maleMicelInPopulation);
        Mouse motherMouse = findSexuallyMatureMouse(femaleMicelInPopulation);
        //I create a set that will store 1 or more female mice according to his couple's tendency to
polygamy.
        TreeSet<Mouse> mothersMice = new TreeSet<>();
        if (fatherMouse != null && motherMouse != null) {
            //I add the female mouse stored in the position 0 of the list of female mice within the
virtual population to the tree set.
            mothersMice.add(motherMouse);
            //I call the method that creates the families.
            createFamily(fatherMouse, mothersMice);
        } else{
            //If the family couldn't be created because either the female or male mouse were null, I
add the one that wasn't null to the lists of the population.
            if(fatherMouse != null){
                addMiceToList(fatherMouse);
            } else if (motherMouse != null) {

```

```

        addMiceToList(motherMouse);
    }
}
}

/**
 * Method that checks that the mice present in the families are younger than 2.5 years. If they
 * are not, I make them die by eliminating them from the lists and removing the families from the
 * set of families, if necessary (if the dead mouse was the father or if there are no females left in
 * the family).
 * @throws PopulationExceptions if the method 'addMiceToList' throws an exception, then this
 * method rethrows it.
 */
private void adjustFamilies() throws PopulationExceptions{
    //I create a clone of the set of families because inside the for I'm modifying the set of
    families.
    TreeSet<Family> clonedSetOfFamilies = (TreeSet<Family>) setOfFamilies.clone();
    //For each that runs the cloned set of families.
    for (Family family : clonedSetOfFamilies) {
        //I obtain the mothers, the father and the biological father of the family by calling the
        getters of Family.
        TreeSet<Mouse> mothers = family.getMothersMice();
        Mouse father = family.getFatherMouse();
        Mouse biologicalFather = family.getBiologicalFather();
        //I store the set of mothers in a list, so I can have direct access to the elements it contains.
        List<Mouse> listOfMothers = new ArrayList<>(mothers);
        //For that runs the list of female mice (mothers of the family)
        for(int i = 0; i < listOfMothers.size(); i++) {
            //If the mouse in position 'i' is not young enough to continue living, I remove it from the
            list that contains the mice in the population and the list that contains the mothers of the family.
            if (!family.shouldBeAlive(listOfMothers.get(i), daysProcreationPassed)) {
                micelnPoupulation.remove(listOfMothers.get(i));
                listOfMothers.remove(listOfMothers.get(i));
            }
        }
        if(listOfMothers.isEmpty()) {
            if(family.shouldBeAlive(father, daysProcreationPassed)) {
                addMiceToList(father);
            }
            if(biologicalFather != null && family.shouldBeAlive(biologicalFather,
daysProcreationPassed)){
                addMiceToList(biologicalFather);
            }
            setOfFamilies.remove(family);
        }
        if (!family.shouldBeAlive(father, daysProcreationPassed)) {
            micelnPoupulation.remove(father);
            maleMicelnPopulation.remove(father);
            //If the younger male is null it is because we didn't find a mouse to replace the dead
            one. Therefore, I run the list of mothers (the ones in there will have already

```

```

        //passed the 'youth' check and are young enough to continue living) and call the
method that adds them to the list of mice of the population (I re-insert them so they can
        //form another family).
        for(int k=0; k<listOfMothers.size(); k++) {
            addMiceToList(listOfMothers.get(k));
        }
        //Also, if the biological father is not null and is young enough to continue living, I call the
method that re-inserts mice in the mice of the population that can form families.
        if(biologicalFather != null && family.shouldBeAlive(biologicalFather,
daysProcreationPassed)) {
            addMiceToList(biologicalFather);
        }
        //Finally, I remove the family from the set of families.
        setOfFamilies.remove(family);
    }
    if (biologicalFather != null && !family.shouldBeAlive(biologicalFather,
daysProcreationPassed)) {
        micelnPoupulation.remove(biologicalFather);
        maleMicelnPopulation.remove(biologicalFather);
        //I set the male mouse as the fertile male of the family. If we did not find a biological
father to replace the dead one, the biological father will be null and the family
        //will continue to work. However, the family will produce 0 baby mice as biological
fathers are needed in sterile families (if one male is null and the other is sterile,
        //the family lacks a fertile male mouse)
    }
}
}

/**
 * Method that receives as parameter the list whose mice longevity we wish to analyse. If any
mouse is older than 2.5 years, I remove
 * it from the list received as parameter and from the general list 'micelnPopulation'.
 * @param listOfMice list whose mice we want to analyse.
 */
private void adjustMiceLists(List<Mouse> listOfMice){
    if(!listOfMice.isEmpty()) {
        //I create a copy of the receives list in order to use it to iterate (so I can change the original
one)
        List<Mouse> copyListOfMice = new ArrayList<>(listOfMice);
        for (Mouse mouse : copyListOfMice){
            LocalDate birthday = mouse.getBirthDate();
            //LocalDate eldestDate = birthday.minusDays((long) (2.5 * 365));
            LocalDate simulatedDate = LocalDate.now().plusDays(daysProcreationPassed);
            LocalDate eldestDate = simulatedDate.minusDays((long) (2.5 * 365));
            if (birthday.isBefore(eldestDate)){
                listOfMice.remove(mouse);
                micelnPoupulation.remove(mouse);
            }
        }
    }
}
}

```

```

/**
 * Method that receives by parameter a list whose elements are checked to be sexually mature
or not. Either way, the element is removed from the list received as parameter
 * and returned or added to another list that stores the immature mice.
 * @param listOfCandidateMice list that contains the mice whose maturity we will check.
 * @return the mouse. If the mouse is sexually mature the returned object will contain the
object. If it is not, the returned object will contain null.
 */
private Mouse findSexuallyMatureMouse(List<Mouse> listOfCandidateMice){
    Mouse mouse = null;
    while(mouse == null && !listOfCandidateMice.isEmpty()) {
        mouse = listOfCandidateMice.get(0);
        listOfCandidateMice.remove(0);
        if (mouse.isSexuallyMature(daysProcreationPassed)) {
            break;
        } else {
            sexuallyImmatureMice.add(mouse);
            mouse = null;
        }
    }
    return mouse;
}

/**
 * Method that determines the appropriate family for each male mouse by analysing its sterility
or tendency to polygamy.
 * Also, the created families are added to the set of families of the population.
 * @param fatherMouse Mouse object that represents a sexually mature male mouse to be
assigned as the father of the appropriate family.
 * @param mothersMice Mouse object that represents a sexually mature female mouse to be
assigned as the mother of the appropriate family.
 * @throws FamilyExceptions if an exception is thrown when creating an object of any family
class, this method rethrows it.
 */
protected void createFamily(Mouse fatherMouse, TreeSet<Mouse> mothersMice) throws
FamilyExceptions {
    //I create an instance of the Random class which I will use in order to generate random
numbers.
    Random rand = new Random();
    Mouse fertileMale = null; //I create another mouse for the cases in which the father of a
family is sterile.
    //If the male mouse is sterile, we assign it as the father of the family with a sterile male.
    if (fatherMouse.isSterile() && !fatherMouse.isPolygamous()) {
        //I create a for that goes over the list of male mice in the population looking for a fertile
male mouse.
        for(int i=0; i<maleMiceInPopulation.size(); i++){
            //If we find a non-sterile and sexually mature male mouse, we store it in 'fertileMale' and
remove it from the list of male mice in the population.
            if(!maleMiceInPopulation.get(i).isSterile() && !maleMiceInPopulation.get(i).isPolygamous()
&& maleMiceInPopulation.get(i).isSexuallyMature(daysProcreationPassed)){

```

```

        fertileMale = maleMiceInPopulation.get(i);
        maleMiceInPopulation.remove(fertileMale);
        break;
    }
}

//I call the constructor of the sterile family with a sterile dad, a fertile male and the female
and add the family to the set of families.
SterileMaleFamily family = new SterileMaleFamily(fatherMouse, fertileMale, mothersMice);
//The 'fertile male' may store a male mouse or null, depending on whether a fertile male
has been found or not. If it is null, the family will be created anyway but will produce 0 babies.
setOfFamilies.add(family);
}

//If the male mouse is polygamous, I assign him as the father of the polygamous family.
else if (fatherMouse.isPolygamous()) {
    //I create an integer variable that stores the random number generated between 0 and 9.
    int keepAddingMothers = rand.nextInt(10);
    //If the random number is greater or equal to 5, we add one more female to the set of
    mothers of the polygamous family.
    while(keepAddingMothers >= 5) {
        //I call the method that looks for sexually mature mice
        Mouse motherMouseFound = findSexuallyMatureMouse(femaleMiceInPopulation);
        if(motherMouseFound == null){
            //There are no more sexually mature female mice in the list. Therefore, I stop looking
            for females by setting 'keepAddingFemales' to a number smaller than 5.
            keepAddingMothers = 4;
        } else {
            //A sexually mature female mouse was found. I add it to the set of mothers.
            mothersMice.add(motherMouseFound);
            //And I update the random number by re-generating it between 0 and 9.
            keepAddingMothers = rand.nextInt(10);
        }
    }
}

//When I get a random number smaller than 5, I call the constructor of the polygamous
family and create it with the father mouse and the set of females.
if(!fatherMouse.isSterile()){
    //If the father mouse is non-sterile, I create a polygamous family and add it to the set of
    families of the population.
    PolygamousFamily family = new PolygamousFamily(fatherMouse, mothersMice);
    setOfFamilies.add(family);
}
else{
    //If the father mouse is sterile, I create a polygamous family with a sterile father and add
    it to the set of families of the population.
    //I create a 'for' that goes over the list of male mice in the population looking for a fertile
    male mouse
    for(int i=0; i<maleMiceInPopulation.size(); i++){
        //If we find a non-sterile and sexually mature male mouse, we store it in 'fertileMale'
        and remove it from the list of male mice in the population.
        if(!maleMiceInPopulation.get(i).isSterile() &&
        maleMiceInPopulation.get(i).isPolygamous() &&
        maleMiceInPopulation.get(i).isSexuallyMature(daysProcreationPassed)){

```

```

        fertileMale = maleMiceInPopulation.get(i);
        maleMiceInPopulation.remove(fertileMale);
        //I create the family only if we have found a fertile male.
        SterilePolygamousMaleFamily family = new
SterilePolygamousMaleFamily(fatherMouse, fertileMale, mothersMice);
        setOfFamilies.add(family);
        break;
    }
}
}
}
else {
    //If the male mouse is neither sterile nor polygamous, I assign him as the father of a normal
family.
    //I call the constructor of the normal family and add it to the set of families of the
population.
    NormalFamily family = new NormalFamily(fatherMouse, mothersMice);
    setOfFamilies.add(family);
}
}

/**
 * Method that calls the reproduce method of the Family class and adds the offspring to the
lists of the population, by calling the method 'addMiceToList'. We pass an integer
 * to the reproduce method which is a variable that stores how many cycles have been
simulated (it increases by 45 its value every time this method is implemented).
 */
public void addBabyMiceToPopulation(){
    for(Family family : setOfFamilies){
        TreeSet<Mouse> offspring = family.reproduce(daysProcreationPassed);
        for(Mouse mouse : offspring){
            addMiceToList(mouse);
        }
    }
    //I add the mice that in the previous cycle were sexually immature to the lists of female and
male mice of the population (by calling the appropriate method)
    for(int i = 0 ; i < sexuallyImmatureMice.size() ; i++) {
        addMiceToList(sexuallyImmatureMice.get(i));
    }
    //I remove all the elements from the array list
    sexuallyImmatureMice.clear();
}

/**
 * Overridden toString method that establishes how the population objects are outputted.
 * @return the lines to be outputted.
 */
@Override
public String toString() {
    return "Population '" + namePopulation+ "'"+
        "\n - Days in lab to procreate = " + daysProcreation +

```

```

        "\n - Name of the responsible person = " + nameResponsible;
    }
    /**
     * Overridden equals method that defines the conditions under which two population objects
     * will be considered as equals. This is, only when their names, the names of their responsible
     * person, the
     * amount of days they will spend in the laboratory to procreate, the amount of mice and the
     * amount of each type of mouse the population has, are the same.
     * @param o Population instance to be compared.
     * @return true if all the attributes analysed of the 2 populations are equal, or false otherwise.
     */
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Population that)) return false;
        return daysProcreation == that.daysProcreation && Objects.equals(namePopulation,
            that.namePopulation) && Objects.equals(nameResponsible, that.nameResponsible) &&
            Objects.equals(this.amountOfMiceInPopulation(), that.amountOfMiceInPopulation())
            && Objects.equals(this.getAmountFemaleMice(), that.getAmountFemaleMice()) &&
            Objects.equals(this.getAmountMaleMice(), that.getAmountMaleMice()) &&
            Objects.equals(this.getAmountNormalMaleMice(), that.getAmountNormalMaleMice())
            && Objects.equals(this.getAmountPolygamousMaleMice(),
            that.getAmountPolygamousMaleMice()) && Objects.equals(this.getAmountSterileMaleMice(),
            that.getAmountSterileMaleMice()) && Objects.equals(this.getAmountSterileFemaleMice(),
            that.getAmountSterileFemaleMice())
            && Objects.equals(this.getAmountFemaleMiceWithOneMutation(),
            that.getAmountFemaleMiceWithOneMutation());
    }
    /**
     * Overridden hashCode method that generates a hash code for every population depending on
     * its name, the name of its responsible, the amount of days it will spend in the laboratory to
     * procreate, the amount of mice and the amount of each type of mouse the population has.
     * @return the hash code of the population.
     */
    @Override
    public int hashCode() {
        return Objects.hash(namePopulation, nameResponsible, daysProcreation,
            this.amountOfMiceInPopulation(), this.getAmountFemaleMice(), this.getAmountMaleMice(),
            this.getAmountNormalMaleMice(), this.getAmountPolygamousMaleMice(),
            this.getAmountSterileMaleMice(),
            this.getAmountSterileFemaleMice(), this.getAmountFemaleMiceWithOneMutation());
    }
}

```

II. Clase **Mouse**

```
package aed.Laboratory;

import aed.Exceptions.MouseExceptions;
import java.time.LocalDate;
import java.util.Objects;
import java.util.Random;

/**
 * Mouse class defines the "blueprint" that its objects, the mice that this program uses, will have.
 * The class is composed of 9
 * attributes that define the class' state and several methods that define its behaviour.
 * @author abrilbetti
 */
public class Mouse implements Comparable<Mouse> {

    private final int refCode;
    private final LocalDate birthDate;
    private int weightGr;
    private final Gender gender;
    private float temperatureInC;
    private String description;
    private final Chromosome chromosome1;
    private final Chromosome chromosome2;
    private static int numMouse = 0; //Variable that counts how many mice we have in the array
    of the population

    /**
     * Constructor of the mouse class, which receives as parameter a value per each attribute of
     * the class which are used in order to initialise them. The constructor checks that the values of
     * certain variables received as parameters
     * are valid before initialising the attributes of the object.
     * @param _birthDate variable of type local date which stores the date in which the mouse was
     * born.
     * @param _weightGr integer variable that stores the weight in grams of the mouse.
     * @param _chromosome1 variable of type chromosome (enumerate) which stores X or Xmut
     * (value of the first chromosome of the mouse).
     * @param _chromosome2 variable of type chromosome (enumerate) which stores X, Xmut, Y
     * or Ymut (value of the second chromosome of the mouse, which is the one that determines the
     * gender of the mouse).
     * @param _gender variable of type gender (enumerate) which stores 'MALE' or 'FEMALE'
     * (value of the gender of the mouse).
     * @param _temperature float variable that stores the temperature of the mouse in Celsius
     * degrees.
     * @param _description string variable that stores a description of the mouse.
     * @param _refCode integer value that stores the reference code of the mouse, which is unique
     * of every mouse.
    */
}
```

* @throws MouseExceptions if any of the values stored in the received variables does not fulfill the required conditions in order to be appropriate to determine the state of the mouse instances, such as an invalid

* value for the chromosomes or the gender or a future or invalid birthdate.

*/

```
public Mouse(LocalDate _birthDate, int _weightGr, Chromosome _chromosome1, Chromosome
_chromosome2, Gender _gender, float _temperature, String _description, int _refCode) throws
MouseExceptions{
    if(_chromosome1!= Chromosome.X && _chromosome1!= Chromosome.Xmut){
        throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_1);
    }
    if(_chromosome2!=Chromosome.Xmut && _chromosome2!= Chromosome.X &&
_chromosome2!= Chromosome.Ymut && _chromosome2!= Chromosome.Y){
        throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_2);
    }
    if(!isDateValid(_birthDate)){
        throw new MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_BIRTHDAY);
    }
    if(!Gender.isGenderConcordant(_chromosome2, _gender)){
        throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.NON_CONCORDANT_GENDER);
    }
    this.birthDate = _birthDate;
    setWeightGr(_weightGr);
    setTemperatureInC(_temperature);
    this.description = _description;
    this.chromosome1 = _chromosome1;
    this.chromosome2 = _chromosome2;
    this.gender = _gender;
    this.refCode = _refCode;
}
/**
```

* Overloaded constructor for the class 'Mouse'. This constructor receives one variable in order to initialise the mouse's attributes by calling the

* previous constructor. The difference is that this constructor does not receive a reference code as parameter.

* @param _birthday local date type of variable that stores the date in which the mouse was born.

* @param _weightGr integer variable that stores the weight of the mouse.

* @param _chromosome1 chromosome type of variable (enumerate) that stores the value of the first chromosome (either X or Xmut).

* @param _chromosome2 chromosome type of variable (enumerate) that stores the value of the second and gender determining chromosome (X, Xmut, Y or Ymut).

* @param _gender gender type of variable (enumerate) that stores the value of the gender of the mouse (either MALE or FEMALE).

* @param _temperatureInC float variable that stores the value of the temperature in Celsius degrees of a mouse.

* @param _description string variable that stores a description of the mouse.

* @throws MouseExceptions an exception is rethrown if the called constructor throws one.

Abril Maitena Betti

```

    */
    public Mouse(LocalDate _birthday, int _weightGr, Chromosome _chromosome1, Chromosome
_chromosome2, Gender _gender, float _temperatureInC, String _description)throws
MouseExceptions{
        this(_birthday, _weightGr, _chromosome1, _chromosome2, _gender, _temperatureInC,
_description, generateRefCode());
    }

    /**
     * Constructor of the mouse class that is used to create a virtual population of mice in order to
perform the "Montecarlo" simulation.
     * This constructor receives as parameter 2 variables, which are the 2 chromosomes of the
mouse. In order to initialise the other attributes of the mice, it generates random
     * numbers and a date within the last 2.5 years.
     * @param chromosome1 stores the value of the first chromosome the mice has according to
the random numbers (used in Population) and their relationship with the percentages.
     * @param chromosome2 stores the value of the second chromosome the mice has according
to the random numbers (used in Population) and their relationship with the percentages.
    */
    public Mouse(Chromosome chromosome1, Chromosome chromosome2) throws
MouseExceptions{
        if(chromosome1!= Chromosome.X && chromosome1!= Chromosome.Xmut){
            throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_1);
        }
        if(chromosome2!=Chromosome.Xmut && chromosome2!= Chromosome.X && chromosome2!
= Chromosome.Ymut && chromosome2!= Chromosome.Y){
            throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_2);
        }
        Random random = new Random();
        this.chromosome1 = chromosome1;
        this.chromosome2 = chromosome2;
        this.gender = Gender.assignGenderToVirtualMouse(chromosome2);
        this.refCode = generateRefCode();
        this.birthDate = generateRandomBirthday();
        this.weightGr = random.nextInt(491)+10; //I create a random number between 10 and 500 for
the weight of the virtual mouse.
        //I do this by computing a random number between 0 and 490 and shifting the result by 10,
so I will get a number between 10 and 500 and not between 0 and 490.
        this.temperatureInC = Math.round((1 + (random.nextFloat()*(45-1)))*10.0f)/10.0f; //I create a
random float number between 1 and 45 for the temperature of the mouse.
        /*I do it by multiplying the random float by the range of possible temperatures (which is 44)
and shift the result by one in order to obtain a number between 1 and 45 instead of 0 and 44.
        I also round the obtained value multiplied by 10.0f, which is then divided by the same value
in order to obtain only 1 digit after the comma.
    */
        this.description = "Mouse " + this.refCode + " weights: " + this.weightGr + " gr.";
    }

    /**

```

* Constructor of the mouse class that is used in order to create virtual offspring of the families of mice in order to perform the simulation of the evolution of the mice population.

* This constructor receives as parameter 3 variables, which are the gender of the mouse and its 2 chromosomes. In order to initialise the other attributes of the mice, it generates random

* numbers and uses the present date.

* @param chromosome1 stores the value of the first chromosome the mice has according to the random numbers (used in Population) and their relationship with the percentages.

* @param chromosome2 stores the value of the second chromosome the mice has according to the random numbers (used in Population) and their relationship with the percentages.

* @param birthday stores the date in which the baby mice were born (the present date).

*/

```
public Mouse(Chromosome chromosome1, Chromosome chromosome2, LocalDate birthday)
throws MouseExceptions{
    if(chromosome1!= Chromosome.X && chromosome1!= Chromosome.Xmut){
        throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_1);
    }
    if(chromosome2!=Chromosome.Xmut && chromosome2!= Chromosome.X && chromosome2!
= Chromosome.Ymut && chromosome2!= Chromosome.Y){
        throw new
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_CHROMOSOME_2);
    }
    Random random = new Random();
    this.chromosome1 = chromosome1;
    this.chromosome2 = chromosome2;
    this.birthDate = birthday;
    this.gender = Gender.assignGenderToVirtualMouse(chromosome2);
    this.refCode = generateRefCode();
    this.weightGr = random.nextInt(491)+10; //I create a random number between 10 and 500 for
the weight of the virtual mouse.
    //I do this by computing a random number between 0 and 490 and shifting the result by 10,
so I will get a number between 10 and 500 and not between 0 and 490.
    this.temperatureInC = Math.round((1 + (random.nextFloat()*(45-1)))*10.0f)/10.0f; //I create a
random float number between 1 and 45 for the temperature of the mouse.
    /*I do it by multiplying the random float by the range of possible temperatures (which is 44)
and shift the result by one in order to obtain a number between 1 and 45 instead of 0 and 44.
    I also round the obtained value multiplied by 10.0f, which is then divided by the same value
in order to obtain only 1 digit after the comma.
    */
    this.description = "Mouse " + this.refCode + " weights: " + this.weightGr + " gr.";
}
```

/**

* Getter for the reference code which makes the attribute accessible from external classes.

* @return the reference code of the mouse, stored in an integer variable.

*/

```
public int getRefCode() {
    return refCode;
}
```

/**

Abril Maitena Betti

```

    * Getter for the birthdate of the mouse, in order for it to be accessible from external classes.
    * @return the birthdate of the mouse, which is stored in a variable of type 'LocalDate'.
    */
    public LocalDate getBirthDate() {
        return birthDate;
    }

    /**
     * Getter for the weight of the mouse, in order for it to be accessible from external classes.
     * @return the weight of the mouse, stored in an integer variable.
     */
    public int getWeightGr() {
        return weightGr;
    }

    /**
     * Getter for the gender of the mouse, in order for it to be accessible from external classes.
     * @return a variable of type 'Gender' that stores the gender of the mouse (MALE or FEMALE)
     */
    public Gender getGender() {
        return gender;
    }

    /**
     * Getter for the temperature of the mouse, in order for it to be accessible from external classes.
     * @return a variable of type float that stores the temperature in Celsius degrees of the mouse.
     */
    public float getTemperatureInC() {
        return temperatureInC;
    }

    /**
     * Getter for the description of the mouse, in order for it to be accessible from external classes.
     * @return a string variable that stores the description of the mouse.
     */
    public String getDescription() {
        return description;
    }

    /**
     * Getter for the first chromosome of the mouse, in order for it to be accessible from external
     classes.
     * @return a variable of type 'Chromosome' that stores the first chromosome of the mouse (X or
     Xmut)
     */
    public Chromosome getChromosome1() {
        return chromosome1;
    }

    /**

```

* Getter for the second chromosome of the mouse, in order for it to be accessible from external classes.

* @return a variable of type 'Chromosome' that stores the second chromosome of the mouse (X, Xmut, Y or Ymut)

*/

```
public Chromosome getChromosome2() {  
    return chromosome2;  
}
```

/**

* Setter in order to set a value for the weight of the mouse. In order for the value to be set, it needs to be greater

* than 10 but lower than 500. If it does not fulfill this requirement, an exception is thrown preventing the value to be set.

* @param weightGr variable of integer type that stores the new value for the weight attribute.

* @throws MouseExceptions an exception of type 'invalid weight' is thrown if we try to set the attribute to an invalid value.

*/

```
public void setWeightGr(int weightGr) throws MouseExceptions{  
    if(weightGr < 10 || weightGr > 500){  
        throw new MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_WEIGHT);  
    }  
    this.weightGr = weightGr;  
}
```

/**

* Setter for the temperature of the mouse. In order for the value of the parameter to be set, it needs to be bigger than 1 and

* smaller than 45. If the value does not fulfill this requirement, an exception is thrown, preventing the value to be set.

* @param temperatureInC variable of floating type that stores the new value for the temperature in Celsius degrees.

* @throws MouseExceptions an exceptions of type 'invalid temperature' is thrown if the value received as parameter is invalid.

*/

```
public void setTemperatureInC(float temperatureInC) throws MouseExceptions{  
    if(temperatureInC < 1 || temperatureInC > 45){  
        throw new  
MouseExceptions(MouseExceptions.ErrorTypeMouse.INVALID_TEMPERATURE);  
    }  
    //The temperature of the mouse must be between [1,45] in order to be valid  
    this.temperatureInC = temperatureInC;  
}
```

/**

* Setter for the description of the mouse.

* @param description variable of string type that stores the new description of the mouse.

*/

```
public void setDescription(String description) {  
    this.description = description;  
}
```

Abril Maitena Betti

```

/**
 * Method that generates a unique reference code for each mouse. It does it increasingly.
 * @return the generated reference code.
 */
public static int generateRefCode(){
    numMouse++;
    return numMouse;
}

/**
 * Method that checks whether the birthdate introduced by the user is valid (a past or the
current date)
 * @param birthDate date introduced by the user
 * @return true if the date is valid and false if it is not
 */
private boolean isDateValid(LocalDate birthDate){
    LocalDate now = LocalDate.now();
    //If the birthday is in the future it is not valid and a false will be returned.
    return !birthDate.isAfter(now);
}

/**
 * Method tht receives as parameters the new values of the mouse and calls the setter in order
to change the data of the mice.
 * @param weight new value of weight.
 * @param temp new value of the temperature of the mouse.
 * @param description new description of the mouse.
 * @throws MouseExceptions rethrows the exception thrown in the called setters of Mouse (if
they were to throw one because the values are invalid).
 */
public void changeMouseData(int weight, float temp, String description)throws
MouseExceptions {
    //I call the setters and pass them the new values for their attributes
    setWeightGr(weight);
    setTemperatureInC(temp);
    setDescription(description);
    /*The user can only modify the weight, temperature and description of a mouse because the
other attributes (chromosomes, gender, birthday
and reference code) are final and, therefore, cannot be modified once initialised */
}

/**
 * Method that checks whether a mouse is sterile or not based on the gender of the mouse and
its chromosomes
 * @return true if the mouse is sterile and false otherwise
 */
public boolean isSterile(){
    if(this.gender == Gender.FEMALE){
        if(chromosome1 == Chromosome.Xmut && chromosome2 == Chromosome.Xmut){
            return true; //the female mouse is sterile
        }
    }
}

```

```

    }
}
else if(this.gender == Gender.MALE){
    if(chromosome1 == Chromosome.Xmut){
        return true; //the male mouse is sterile
    }
}
return false;
}

/**
 * Method that checks whether a mouse is polygamous or not.
 * @return true if the mouse is polygamous and false if the mouse is not.
 */
public boolean isPolygamous(){
    if(this. gender == Gender.MALE && chromosome2 == Chromosome.Ymut){
        return true; //The male mouse is polygamous
    }
    return false;
}

/**
 * Method that sets the birthday of the instance to a random date that ranges from the present
to 2.5 years in the past.
 */
private LocalDate generateRandomBirthday(){
    //I declare 2 local dates that will store the maximum and minimum dates that a virtual mouse
can be born in.
    LocalDate youngerDate = LocalDate.now();
    LocalDate eldestDate = youngerDate.minusDays((long) (2.5 * 365));
    //I compute the total amount of days between the eldest and younger dates in which the
mice can be born.
    long daysBetween = (youngerDate.getYear()-eldestDate.getYear())*365 +
(youngerDate.getDayOfYear()-eldestDate.getDayOfYear());
    Random rand = new Random();
    //I generate the random number between 0 and the amount of days between the two dates
    long randomNumberOfDays = rand.nextInt((int) daysBetween + 1);

    return eldestDate.plusDays(randomNumberOfDays);
}

/**
 * Method that checks if a mouse is sexually mature or not by analysing whether 70 days have
or not passed since the birth of the mouse and the "present" date according to the simulation.
The
 * number 70 comes from the condition that states that mice can reproduce after 2 cycles; i.e.
after 45 days and the 25 days it takes the mice to
 * get independent (days that belong to the cycle in which the mice are born)
 * @param daysPassedSimulation integer variable that stores the amount of days that have
passed since in the simulation, which indicates how many cycles of reproduction have been
carried on.

```

Abril Maitena Betti

```

    * @return true if the mouse is sexually mature or false otherwise.
    */
    protected boolean isSexuallyMature(int daysPassedSimulation){
        //I am checking that the mouse was born 2 cycles ago (actually, 70 days before the present
date). If this is
        // (45 + 25 days ago (1 cycle and the 25 days it took them to get independent during the
cycle in which they were born)
        LocalDate dateOfSexualMaturity = this.getBirthDate().plusDays(70);
        LocalDate simulatedPresentDate = LocalDate.now().plusDays(daysPassedSimulation);
        if(simulatedPresentDate.isAfter(dateOfSexualMaturity)){
            return true;
        }
        return false;
    }

    /**
    * Overridden equals method that defines the conditions under which two mouse objects will be
considered as equals. This is, only when their reference codes are the same.
    * @param o Mouse instance to be compared.
    * @return true if the reference codes of the 2 mice are equal, or false otherwise.
    */
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Mouse mouse = (Mouse) o;
        return refCode == mouse.refCode;
    }

    /**
    * Overridden hashCode method that generates a hash code for every mouse depending on
their reference codes.
    * @return the hash code of the mouse with reference code "refCode".
    */
    @Override
    public int hashCode() {
        return Objects.hashCode(refCode);
    }

    /** Overridden compareTo method that
    * @param mouse1 instance of Mouse class that we wish to compare.
    * @return 0 if the objects, according to the equals method, are equal or the difference of the
two reference codes of the mice if they are not equal.
    */
    @Override
    public int compareTo(Mouse mouse1) {
        if(this.equals(mouse1)){
            return 0;
        }
        else{
            return this.refCode - mouse1.refCode;
        }
    }

```

```

    }
}

/**
 * Overridden toString method used to state how mouse objects are outputted.
 * @return the output for the attributes of the mouse class
 */
@Override
public String toString() {
    return "Mouse {" +
        "Reference Code = " + refCode +
        "; Birthdate = " + birthDate +
        "; Weight in gr = " + weightGr +
        "; Gender = " + gender +
        "; Temperature in C = " + temperatureInC +
        "; Description = '" + description + "'" +
        "; Chromosome 1 = " + chromosome1 +
        "; Chromosome 2 = " + chromosome2 +
        '}';
}
}

```

III. Clase Gender (Enumerado)

```

package aed.Laboratory;

/**
 * Enum that contains the different values that the gender field of instances of the mouse class
 * can take.
 * @author abrilbetti
 */
public enum Gender {
    MALE, FEMALE;

    /**
     * Method that checks whether the introduced gender and chromosomes are concordant
     * between one another
     * @param chromosome2 chromosome of the mouse that we want to analyse in order to
     * determine what is the appropriate gender for the mouse.
     * @param gender introduced gender, which we will see if it is concordant with the expected
     * gender.
     * @return true if the gender is concordant
     */
    public static boolean isGenderConcordant(Chromosome chromosome2, Gender gender){
        if(chromosome2 == Chromosome.X || chromosome2 == Chromosome.Xmut){
            return gender == Gender.FEMALE;
        }
        else if(chromosome2 == Chromosome.Y || chromosome2 == Chromosome.Ymut){
            return gender == Gender.MALE;
        }
    }
}

```

Abril Maitena Betti

```

    }
    return false;
}
/**
 * Method that determines the gender of the virtual mouse by analysing its second
chromosome.
 * @param chromosome2 chromosome of the mouse that we want to analyse in order to assign
the appropriate gender to the virtual mouse.
 * @return the gender.
 */
public static Gender assignGenderToVirtualMouse(Chromosome chromosome2){
    if(chromosome2 == Chromosome.X || chromosome2 == Chromosome.Xmut){
        return Gender.FEMALE;
    } else{
        return Gender.MALE;
    }
}
}

```

IV. Clase **Chromosome (Enumerado)**

```

package aed.Laboratory;
/**
 * Enum that contains the different values that the chromosomes fields of mouse objects can take
 * @author abrilbetti
 */
public enum Chromosome {
    X, Xmut, Y, Ymut //possible values for the chromosomes
}

```

C. Paquete **InputOutput**

I. Clase **Input**

```

package aed.InputOutput;

import aed.Comparators.ComparatorByBirthdate;
import aed.Comparators.ComparatorByWeight;
import aed.Exceptions.MouseExceptions;
import aed.Exceptions.PopulationExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Population;
import aed.Laboratory.Mouse;
import aed.Main.MainAnimalLab;

import java.io.BufferedReader;

Abril Maitena Betti

```

```

import java.io.IOException;
import java.io.InputStreamReader;
import java.time.LocalDate;
import java.time.DateTimeException;
import java.util.Collections;
import java.util.List;

/**
 * Input is the class that deals with reading the information the user introduces as inputs and
 * making sure it is the data type the program expects. Also,
 * some of its functions output lines of codes in order to ask the user to introduce an input.
 * This class has functions that represent options number 2, 3, 6 and 7 from the menu shown to
 * the user.
 * @author abrilbetti
 */
public class Input {
    /**
     * Function that creates a population with the information the user introduces as inputs (the
     * attributes of the class population)
     * @return the population created
     */
    public static Population createPopulation(){
        //Ask the user to introduce the information needed in order to create the population
        System.out.println("Introduce the population's name (string): ");
        String namePopulation = Input.readString();
        System.out.println("Introduce the name of the responsible of the population (string): ");
        String nameResponsible = Input.readString();
        System.out.println("Introduce the amount of days of procreation of the population (integer
smaller or equal to 630 and multiple of 45): ");
        int daysProcreation = Input.readInteger();
        Population usersPopulation = null;
        try {
            usersPopulation = new Population(namePopulation, nameResponsible, daysProcreation);
            MainAnimalLab.filePath = null; //If the population was properly created, I also set the
current path to null, which is
            // useful in the 'save' function in order to check whether this new population has already
been 'saved as' or not
        } catch (PopulationExceptions pe){
            System.out.println("ERROR: " + pe);
        }
        return usersPopulation;
    }
    /**
     * Function that creates a virtual population with the information the user introduces as inputs
     * (the attributes of the class population) and the percentages of the virtual mice.
     * @return the virtual population created.
     */
    public static Population createVirtualPopulation(){
        //Ask the user to introduce the information needed in order to create the virtual population
        System.out.println("Introduce the population's name (string): ");
        String namePopulation = Input.readString();

```

Abril Maitena Betti

```

        System.out.println("Introduce the name of the responsible of the population (string): ");
        String nameResponsible = Input.readString();
        System.out.println("Introduce the amount of days of procreation of the population (integer
smaller or equal to 630 and multiple of 45): ");
        int daysProcreation = Input.readInteger();
        System.out.println("Introduce the total number of mice you want the virtual population to
have: ");
        int totalNumberMice = Input.readInteger();
        System.out.println("Introduce the percentage of male mice in the population: ");
        int percentageMaleMice = Input.readInteger();
        System.out.println("Introduce the percentage of female mice in the population: ");
        int percentageFemaleMice = Input.readInteger();
        System.out.println("Introduce the percentage of sterile male mice in the population: ");
        int percentageSterileMaleMice = Input.readInteger();
        System.out.println("Introduce the percentage of polygamous male mice in the population: ");
        int percentagePolygamousMaleMice = Input.readInteger();
        System.out.println("Introduce the percentage of mutated X chromosomes among the female
mice of the population: ");
        int percentageXmutInFemales = Input.readInteger();
        Population virtualPopulation = null;
        try {
            //I call the constructor of virtual populations
            virtualPopulation = new Population(namePopulation, nameResponsible, daysProcreation,
totalNumberMice, percentageMaleMice, percentageSterileMaleMice,
percentagePolygamousMaleMice, percentageFemaleMice, percentageXmutInFemales);
            MainAnimalLab.filePath = null; //If the population was properly created, I also set the
current path to null, which is
            // useful in the 'save' function in order to check whether this new population has already
been 'saved as' or not
            //I call the method that fills the virtual population with virtual mice
            virtualPopulation.fillVirtualPopulation(totalNumberMice, percentageFemaleMice,
percentageMaleMice, percentageSterileMaleMice, percentagePolygamousMaleMice,
percentageXmutInFemales);
        } catch (PopulationExceptions | MouseExceptions pe){
            System.out.println("ERROR: " + pe);
        }
        //I return the virtual population
        return virtualPopulation;
    }

/**
 * Function that calls the constructor of the mouse class and creates a mouse with the data
introduced by the user.
 * @return the mouse that was created
 */
    public static Mouse createMouse(){
        Mouse mouse1 = null;
        //I ask the user to introduce the information needed to create a new mouse
        System.out.println("Introduce the mouse's birthday (integer): ");
        LocalDate date = Input.readLocalDate();

```

```

        System.out.println("Introduce the weight of the mouse (integer between 10 (inclusive) and
500 (inclusive)): ");
        int weightMouse1 = Input.readInteger();
        System.out.println("Introduce the temperature in C of the mouse (floating-point number
between 1 (inclusive) and 45 (inclusive)): ");
        float temperatureInC = Input.readFloat();
        System.out.println("Introduce the gender of the mouse (FEMALE or MALE): ");
        Gender gender;
        //I check that the information introduced by the user is valid (in the cases of gender and
chromosomes)
        while(true) {
            try {
                gender = Input.readGender();
                break;
            } catch (IllegalArgumentException iae) {
                System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
introducing an appropriate value for the gender (male or female)!");
            }
        }
        System.out.println("Introduce any description of the mouse (String): ");
        String descriptionMouse = Input.readString();
        System.out.println("Introduce X or Xmut for the first chromosome of the mouse: ");
        Chromosome chromosome1Mouse = null;
        try{
            chromosome1Mouse = Input.readChromosome();
        } catch (IllegalArgumentException iae){
            System.out.println("ERROR: the type of data you introduced is incorrect. Try again, now
introducing an appropriate value for the first chromosome (X or Xmut)!");
        }
        System.out.println("Introduce X, Xmut, Y or Ymut for the gender-determining chromosome of
the mouse: ");
        Chromosome chromosome2Mouse = null;
        try {
            chromosome2Mouse = Input.readChromosome();
        } catch (IllegalArgumentException iae){
            System.out.println("ERROR: the type of data you introduced is incorrect. Try again, now
introducing an appropriate value for the second chromosome!");
        }
        try{
            //I create the mouse with the data provided by the user
            mouse1 = new Mouse(date, weightMouse1, chromosome1Mouse, chromosome2Mouse,
gender, temperatureInC, descriptionMouse);
        } catch (MouseExceptions me){
            System.out.println("ERROR:" + me);
        }
        return mouse1;
    }
}

```

/**

* Function that receives 2 parameters and, depending on the value of the "ordinationCriteria" parameter, calls the appropriate ordination method of population class.

Abril Maitena Betti

* @param ordinationCriteria integer that stores a number between 1 and 3 that the user introduced, which states the type of ordination the user wants to perform.

* @param populationCreated instance of population that stores the previously created population.

*/

```
public static List<Mouse> orderMiceOfPopulation(int ordinationCriteria, Population
populationCreated){
    //I create a switch and a case for each possible value that "ordinationCriteria" can store
    List<Mouse> arrayListMice = populationCreated.getCopyOfMiceInPopulation();
    switch(ordinationCriteria){
        case 1: //order the mice in increasing order of their reference codes
            Collections.sort(arrayListMice);
            break;
        case 2: //order the mice in chronological order
            //populationCreated.orderMiceChronologically();
            Collections.sort(arrayListMice, new ComparatorByBirthdate());
            break;
        case 3: //order the mice in increasing order of their weights
            Collections.sort(arrayListMice, new ComparatorByWeight());
            break;
        default:
            //If the number the user introduced is not 1, 2 nor 3, I print an error message.
            System.out.println("The option you introduced is invalid. ");
    }
    return arrayListMice;
}
```

/**

* Function that modifies the data of a mouse from the created population by calling the method "changeMouseData" of the class

* population. It catches exceptions in the case the mouse is not found within the array of mice of the population or if any

* of the introduced values (the data to modify of the mouse) are invalid (temperature>45, temperature<=0, weight<=0 or weight>501

* @param usersPopulation which stores the population the user created

*/

```
public static void modifyMouseData(Population usersPopulation){
    //I ask the user to introduce the data I need in order to modify the mouse's data
    System.out.println("Introduce the reference code of the mouse whose information you wish
to modify (integer): ");
    int refCodeOfMouseToModify = Input.readInteger();
    System.out.println("Introduce the new weight of the mouse (integer between 10 (inclusive)
and 500 (inclusive)): ");
    int newWeightMouse = Input.readInteger();
    System.out.println("Introduce the new temperature in C of the mouse (floating-point number
between 1 (inclusive) and 45 (inclusive)): ");
    float newTemperatureInC = Input.readFloat();
    System.out.println("Introduce the new description of the mouse (String): ");
    String newDescriptionMouse = Input.readString();
    try {
        //I call the method of population that modifies a mouse's data

```

Abril Maitena Betti

```

        Mouse mouseToModify = usersPopulation.identifyMouse(refCodeOfMouseToModify);
        mouseToModify.changeMouseData(newWeightMouse, newTemperatureInC,
newDescriptionMouse);
        //The following line will only be printed if the mouse's information is properly modified
        System.out.println("The mouse's data has been successfully modified.");
    } catch (PopulationExceptions | MouseExceptions pe) {
        System.out.println("ERROR: " + pe);
    }
}
/**
 * Function that shows the information of the desired mouse by calling the method
"seeMouseInformation" of the class population.
 * It catches an exception that is thrown if the mouse is not found within the array of mice
the population has.
 * @param usersPopulation which stores the population the user created
 */
public static void seeInformationOfMice(Population usersPopulation){
    //I ask for the reference code of the mouse whose information the user wants to see
    System.out.println("Introduce the reference code of the mouse whose information you wish
to see (integer): ");
    int refCodeOfMouse = Input.readInteger();
    try {
        String mouseInformation = usersPopulation.seeMouseInformation(refCodeOfMouse);
        //I print the information of the mouse
        System.out.println(mouseInformation);
    } catch (PopulationExceptions pe) {
        //An exception is thrown if the mouse with the introduced reference code does not exist.
        System.out.println("ERROR: " + pe);
    }
}
/**
 * Function that reads the integer input the user introduced
 * @return an int variable that stores the input introduced by the user
 */
public static int readInteger(){
    /*The while is a controlled loop that will run until a "return" is found within the code. This will
only happen when the
    input introduced by the user is an int (when the "parse" conversion is accomplished) and this
value is returned. On the
    other hand, if the input introduced is not an integer type, it won't be possible to convert it to
an int value, which will
    throw an exception, causing the while loop to repeat itself again.*/
    while(true){
        try{
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String readString = br.readLine(); //Can throw an IOException
            if(readString.trim().isEmpty()){
                //Trim function removes whitespaces, tabs, etc. Once removed, I check whether it is
empty or not.
                throw new IllegalArgumentException("ERROR: input cannot be empty or
whitespace.");
            }
        }
    }
}

```

```

    }
    int integer = Integer.parseInt(readString);
    return integer;
} catch(IOException e){
    System.out.println("ERROR: there was an error when reading you input, try again!");
} catch(IllegalArgumentException iae){
    System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
introducing an integer!");
}
}
}
/**
 * Function that reads the string input the user introduced
 * @return a string variable that stores the input introduced by the user
 */
public static String readString(){
    while(true){
        try{
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String readString = br.readLine();
            if(readString.trim().isEmpty()){
                throw new IllegalArgumentException("ERROR: input cannot be empty or
whitespace.");
            }
            return readString;
        } catch(IOException e){
            System.out.println("ERROR: there was an error when reading you input, try again!");
        } catch(IllegalArgumentException iae){
            System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
introducing a string!");
        }
    }
}
/**
 * Function that reads the float input the user introduced
 * @return a float variable that stores the input introduced by the user
 */
public static float readFloat(){
    while(true){
        try{
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String readString = br.readLine();
            if(readString.trim().isEmpty()){
                throw new IllegalArgumentException("ERROR: input cannot be empty or
whitespace.");
            }
            float floatVariable = Float.parseFloat(readString);
            return floatVariable;
        } catch(IOException e){
            System.out.println("ERROR: there was an error when reading you input, try again!");
        } catch(IllegalArgumentException iae){

```

```

        System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
        introducing a floating-point number!");
    }
}

/**
 * Function that reads an integer introduced by the user, checks if it is between the valid range
 (end and begin integer values it receives as parameters)
 * and returns the valid integer or throws an exception if it is not valid.
 * @param begin states the minimum value the input integer can take
 * @param end states the maximum value the input integer can take
 * @param message string that we want to output to the user
 * @return the read integer (only if it is a valid one)
 */
public static int readIntegerMessage(int begin, int end, String message){
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    String readString = "";
    /*The buffered reader is declared outside the loop because if not, each readLine() call re-
    initializes BufferedReader,
    which can lead to input lag or double prompts (such as having to introduce "return" twice
    after introducing an invalid value
    for day or month or year of a date).
    */
    System.out.println(message);
    while(true){
        try{
            readString = br.readLine();
            if(readString.trim().isEmpty()){
                //Trim function removes whitespaces, tabs, etc. Once removed, I check whether it is
                empty or not.
                throw new IllegalArgumentException("ERROR: input cannot be empty or
                whitespace.");
            }
            int integer = Integer.parseInt(readString);
            if(integer >= begin && integer <= end){
                return integer;
            }
        } catch(IOException e){
            System.out.println("ERROR: there was an error when reading you input, try again!");
        } catch(IllegalArgumentException iae){
            System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
            introducing an integer between " + begin + " and " + end + "!");
        }
    }
}

/**
 * Function that reads the dates the user introduces as inputs. It calls the function
 "readInputMessage" 3 times in order to read each component of the date (day, month and year).
 * @return the date if it is a valid one

```

```

*/
public static LocalDate readLocalDate(){
    while(true) {
        int dia = readIntegerMessage(1, 31, "Introduce the day: ");
        int mes = readIntegerMessage(1, 12, "Introduce the month: ");
        int year = readIntegerMessage(0, 2024, "Introduce the year: ");
        try {
            //If the date is invalid (for example February 29th or April the 31st) an exception is
thrown
            LocalDate date = LocalDate.of(year, mes, dia);
            return date;
        } catch (DateTimeException dateTimeException) {
            System.out.println("ERROR: The date you introduced is incorrect. Try again introducing
a valid one.");
        }
    }
}

/**
 * Function that reads the input the user introduces for stating the chromosome of the mouse.
 * @return the chromosome if th introduced input is a valid one
 */
public static Chromosome readChromosome(){
    while(true){
        try{
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String readString = br.readLine();
            if(readString.trim().isEmpty()){
                throw new IllegalArgumentException();
            }
            Chromosome chromosome = Chromosome.valueOf(readString.trim());
            return chromosome;
        } catch(IOException e){
            System.out.println("ERROR: there was an error when reading you input, try again!");
        } catch(IllegalArgumentException iae){
            System.out.println("ERROR: The type of data you introduced is incorrect. Try again, now
introducing an appropriate chromosome!");
        }
    }
}

/**
 * Function that reads the input the user introduces for stating the chromosome of the mouse.
 * @return the chromosome if the introduced input is a valid one
 */
public static Gender readGender(){
    while(true){
        try{
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            String readString = br.readLine();
            if(readString.trim().isEmpty()){
                throw new IllegalArgumentException();
            }

```

```

    }
    //The function 'value of' automatically throws an illegal argument exception if it is not
able to perform the parse operation
    Gender gender = Gender.valueOf(readString.trim().toUpperCase());
    return gender;
} catch(IOException e){
    System.out.println("ERROR: there was an error when reading you input, try again!");
}
}
}
}
}

```

II. Clase Output

```

package aed.InputOutput;
import aed.Laboratory.Mouse;
import aed.Laboratory.Population;
import java.util.List;

```

```

/**
 * Output is a class that is composed by 2 functions that describe its behaviour. One of them
outputs the menu of options, which is printed in the console. The other
 * function prints a population that was read from a file.
 * @author abrilbetti
 */
public class Output {
    /**
     * Function that prints the menu which is offered to the user in the main.
     */
    public static void showMenu() {
        System.out.println("\nPlease, introduce the number of the operation you wish the program to
perform from the following menu. \n----- MENU ----- \n" +
            "1. Open a population from a text file. \n2. Create a new population of mice. \n3. Create
a virtual population of mice from the percentages of male, female and mutated mice." +
            "\n4. Add a new mouse to an existing population." + "\n5. List the mice of the
population. \n6. Delete a mouse of the population. \n" +
            "7. Modify a mouse's data. \n8. Detailed information about a mouse. \n9. Simulate the
population's evolution over time.\n10. Save \n11. Save as. \n0. EXIT");
    }

    /**
     * Function that prints the 3 different sorting criteria the user can choose when selects option 5
(list the mice of the population) in the main.
     */
    public static void showOrdinationCriteria() {
        System.out.println("What type of ordination do you wish to perform? (introduce 1, 2 or 3) \n1.
Depending on the reference codes of the mice (from lower reference codes to higher ones). " +
            "\n2. Depending on the birthdays of the mice. \n3. Depending on the weight of the
mice, from lighter-weighted mice to the heaviest ones.");
    }
}

```

Abril Maitena Betti

```

}

/**
 * Function that calls the method of population that prints the mouse that are in the population.
 *
 * @param listOfMice array list that contains the mice of the population.
 */
public static void showOrderedMiceOfPopulation(List<Mouse> listOfMice) {
    //population1.printMiceInPopulation();
    for(int i=0; i<listOfMice.size(); i++) {
        System.out.println(listOfMice.get(i));
    }
}

/**
 * Function that prints the data of the population that was read from a CSV file. Both the
 * population data and their data of their mice (if it has any) are printed
 *
 * @param populationOpened stores the population that was read from the file that was opened
 * (the CSV file)
 */
public static void printPopulationOpened(Population populationOpened) {
    //I call the toString method of the class Population
    System.out.println(populationOpened);
    //If the population is not empty, we print its mice
    if (!populationOpened.isListOfMiceEmpty()) {
        System.out.println("\nMice in the population: ");
        //I call the method that returns a copy of the array list that contains the mice present in the
        population
        List<Mouse> arrayListMice = populationOpened.getCopyOfMiceInPopulation();
        for (int i = 0; i < populationOpened.amountOfMiceInPopulation(); i++) {
            //I store the instance of mouse stored in position i of the array list in mouse in order to
            print its information by using dot operators
            Mouse mouse = arrayListMice.get(i);
            //I call the method toString of Mouse
            System.out.println(mouse);
        }
    } else {
        System.out.println("The population does not have any mice.");
    }
}

/**
 * Function that simulates how a population evolves through the amount of days it spends in
 * the laboratory. At the end of each cycle of reproduction,
 *
 * the percentages that illustrate the amount of female, male, sterile, polygamous and normal
 * mice that there are in the population.
 *
 * @param populationCreated object of Population that contains the population that was
 * created in the main (either a virtual one or one created by the user's inputs)
 *
 * @param numberOfCycle integer variable that stores the number of the cycle whose
 * information we are about to print.

```

Abril Maitena Betti

```

*/
public static void simulatePopulationsEvolution(Population populationCreated, int
numberOfCycle) {
    int numberCycle = numberOfCycle+1;
    int amountMicelnPopulation = populationCreated.amountOfMicelnPopulation();
    int amountFemaleMicelnPopulation = populationCreated.getAmountFemaleMice();
    int amountMaleMicelnPopulation = populationCreated.getAmountMaleMice();
    int amountSterileMaleMicelnPopulation = populationCreated.getAmountSterileMaleMice();
    int amountPolygamousMaleMicelnPopulation =
populationCreated.getAmountPolygamousMaleMice();
    int amountNormalMaleMicelnPopulation = populationCreated.getAmountNormalMaleMice();
    int amountSterileFemaleMicelnPopulation =
populationCreated.getAmountSterileFemaleMice();
    int amountFertileWithMutationFemaleMicelnPopulation =
populationCreated.getAmountFemaleMiceWithOneMutation();
    int amountNormalFemaleMicelnPopulation = amountFemaleMicelnPopulation -
(amountFertileWithMutationFemaleMicelnPopulation + amountSterileFemaleMicelnPopulation);

    float percentageMales = Math.round(((float)amountMaleMicelnPopulation * 100) * 10 /
amountMicelnPopulation)/10.0f;
    float percentageFemales = Math.round(((float)amountFemaleMicelnPopulation * 100) * 10 /
amountMicelnPopulation)/10.0f;
    System.out.println("\nThe total number of mice of cycle number " + numberCycle + " is: " +
amountMicelnPopulation);
    if(amountFemaleMicelnPopulation > 0) {
        float percentageSterileFemales = Math.round(((float)amountSterileFemaleMicelnPopulation
* 100) * 10 / amountFemaleMicelnPopulation)/10.0f;
        float percentageFertileFemalesWithMutation =
Math.round(((float)amountFertileWithMutationFemaleMicelnPopulation * 100) * 10 /
amountFemaleMicelnPopulation)/10.0f;
        float percentageNormalFemales =
Math.round(((float)amountNormalFemaleMicelnPopulation * 100) * 10 /
amountFemaleMicelnPopulation) / 10.0f;
        System.out.println("The percentage of female mice is " + percentageFemales + "%, of
which: \n- " + percentageSterileFemales + "% are sterile. \n- " +
percentageFertileFemalesWithMutation +
"% present the mutation but are fertile. \n- " + percentageNormalFemales + "% don't
present any mutation.");
    }
    if (amountMaleMicelnPopulation > 0) {
        float percentageSterileMales = Math.round(((float)amountSterileMaleMicelnPopulation *
100) * 10 / amountMaleMicelnPopulation)/10.0f;
        float percentagePolygamousMales =
Math.round(((float)amountPolygamousMaleMicelnPopulation * 100) * 10 /
amountMaleMicelnPopulation)/10.0f;
        float percentageNormalMales = Math.round(((float)amountNormalMaleMicelnPopulation *
100) * 10 / amountMaleMicelnPopulation)/10.0f;
        System.out.println("The percentage of male mice is " + percentageMales + "%, of which:
\n- " + percentageSterileMales + "% are sterile. \n- " + percentagePolygamousMales +
"% have the mutation of polygamy. \n- " + percentageNormalMales + "% don't
present any mutation.");
    }
}
Abril Maitena Betti

```

```

    }
}
}

```

D. Paquete **CSV Administration**

I. Clase **CSV**

```
package aed.CSVAdministration;
```

```
import aed.Exceptions.CSVExceptions;
import aed.Exceptions.InputExceptions;
import aed.Exceptions.MouseExceptions;
import aed.Exceptions.PopulationExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Mouse;
import aed.Laboratory.Population;
```

```
import java.io.*;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;
```

```
/**
 * CSV is the class that is in charge of opening a file and reading its information, saving a file and
 * 'saving as' a file.
 * In summary, this class has several functions that represent options number 1, 8 and 9 from the
 * menu shown to the user.
 * @author abrilbetti
 */
```

```
public class CSV {
```

```
    /**
     * Function that opens a file using the path stored in the variable current file path and prints a
     * message showing that the operation was
     * successful or throws an error if it was not.
     * @param filePath stores the file path the user introduced in the main
     * @return the read population which is created and loaded with the mice if there were any in
     * the CSV file. If not, the function returns null
     */
```

```
    public static Population openPopulationFromFile(String filePath) {
        Population populationRead = null;
        try(BufferedReader bfR = new BufferedReader(new FileReader(filePath))){
            String line = bfR.readLine();
            //If there is something written in the file, I try to read it
            if(line != null){
                try {
```

Abril Maitena Betti

```

        populationRead = isLineAPopulation(line);
    } catch (CSVExceptions csve) {
        System.out.println("ERROR: " + csve);
    }
    if (populationRead != null) {
        //I keep reading until the read line is null (when there are no more lines with content to
read)
        while ((line = bfR.readLine()) != null) {
            try {
                //I call the method that verifies if the read line is a mouse and, if it is, creates it
and returns it
                Mouse mouseRead = isLineAMouse(line);
                //Then, I add the mouse to the population we read and created previously.
                populationRead.addMiceToList(mouseRead);
            } catch (CSVExceptions | PopulationExceptions pe) {
                System.out.println("ERROR: " + pe);
            }
        }
    }
} catch (IOException | NumberFormatException | DateTimeParseException e) {
    //DateTimeParseException can be thrown if the parse function fails when it converts the
read information into a 'LocalDate' variable
    System.out.println("ERROR: there was an error reading the file.");
}
return populationRead;
}

/**
 * Method that receives a string (the line read from the introduced file) and checks that its length
is the appropriate one (in order for it to
 * be a population). If it is, then it creates a population with the information in the string received
as parameter, which is returned.
 * @param readLine string that contains a line that was read from the file whose file path the
user introduced.
 * @return the population created with the line read from the file.
 * @throws CSVExceptions when the length of the read line does not coincide with the length a
population should have.
 */
private static Population isLineAPopulation(String readLine) throws CSVExceptions {
    //I create an array which is filled with the content of the line read of the file. Positions store
whatever is written between the commas.
    String[] populationData = readLine.split(",");
    //If the array contains more or less than 3 positions (the amount of attributes the population
class has
    and therefore the amount of data that should be introduced in the file in order for it to be a
population),
    then I throw an exception.
    */
    //If the read line is a population, then we create a population with its data.
    Population populationCreated = null;

```

```

        if(populationData.length == 9) {
            //I assign each position of the array I created to an attribute of population
            String namePopulation = populationData[0].trim();
            String nameResponsible = populationData[1].trim();
            int daysInLab = Integer.parseInt(populationData[2].trim());
            int amountMice = Integer.parseInt(populationData[3].trim());
            int percentageMales = Integer.parseInt(populationData[4].trim());
            int percentageSterileMales = Integer.parseInt(populationData[5].trim());
            int percentagePolygamousMales = Integer.parseInt(populationData[6].trim());
            int percentageFemales = Integer.parseInt(populationData[7].trim());
            int percentageXmutFemales = Integer.parseInt(populationData[8].trim());
            //I call the method that creates populations and create one with the data I read from the
file
            try {
                populationCreated = new Population(namePopulation, nameResponsible, daysInLab,
amountMice, percentageMales, percentageSterileMales, percentagePolygamousMales,
percentageFemales, percentageXmutFemales);
            } catch (PopulationExceptions pe) {
                System.out.println("ERROR: " + pe);
            }
        } else if(populationData.length == 3) {
            //I assign each position of the array I created to an attribute of population
            String namePopulation = populationData[0].trim();
            String nameResponsible = populationData[1].trim();
            int daysInLab = Integer.parseInt(populationData[2].trim());
            //I call the method that creates populations and create one with the data I read from the
file
            try {
                populationCreated = new Population(namePopulation, nameResponsible, daysInLab);
            } catch (PopulationExceptions pe) {
                System.out.println("ERROR: " + pe);
            }
        } else{
            throw new
CSVExceptions(CSVExceptions.ErrorTypeCSV.INVALID_FORMAT_OF_LINE_POPULATION);
        }
        return populationCreated;
    }

    /**
     * Method that receives a string (the line read from the introduced file) and checks that its length
is the appropriate one (in order for it to
     * be a mouse). If it is, then it creates a mouse with the information in the string received as
parameter, which is returned.
     * @param readLine string that contains a line that was read from the file whose file path the
user introduced.
     * @return the created mouse.
     * @throws CSVExceptions when the length of the read line does not coincide with the length a
mouse should have.
     */
    private static Mouse isLineAMouse(String readLine) throws CSVExceptions {

```

Abril Maitena Betti

```

        //I create an array which is filled with the content of the line read of the file. Positions store
whatever is written between the commas.
        String[] mouseData = readLine.split(",");
        // If the array contains an amount of positions different to the one that we should read if we
are reading a mouse, I throw an exception.
        if(mouseData.length != 7) {
            throw new
CSVExceptions(CSVExceptions.ErrorTypeCSV.INVALID_FORMAT_OF_LINE_MOUSE);
        }
        LocalDate birthday = LocalDate.parse(mouseData[0].trim());
        int weight = Integer.parseInt(mouseData[1].trim());
        Chromosome chromosome1 = Chromosome.valueOf(mouseData[2].trim());
        Chromosome chromosome2 = Chromosome.valueOf(mouseData[3].trim());
        Gender gender = Gender.valueOf(mouseData[4].toUpperCase().trim());
        float temperature = Float.parseFloat(mouseData[5].trim());
        String description = mouseData[6];
        Mouse mouseRead = null;
        //I create the mouse with the information read from the CSV file and add it to the population
that was also read from the CSV file
        try {
            mouseRead = new Mouse(birthday, weight, chromosome1, chromosome2, gender,
temperature, description);
        } catch(MouseExceptions me){
            System.out.println("ERROR:" + me);
        }
        if(mouseRead == null){
            throw new IllegalArgumentException("ERROR: The mouse could not be created.");
        }
        return mouseRead;
    }

    /**
     * Function that saves a population only if it has already been saved as in the past. First, it
checks if the population
     * was already saved as in the past and, if it wasn't it calls the function 'save as'. If it was, it
writes the data to save
     * in the file, just as it is done in the 'save as' method.
     * @param filePath stores the path where the file has already been saved as, or null if it hasn't
been 'saved as' yet
     * @param populationCreated stores the population the user has created (if any) or null
otherwise
     * @throws CSVExceptions if the population was not saved as or if the user has not opened a
file.
     * @throws InputExceptions if the user has not created a population.
     */
    public static void savePopulation(Population populationCreated, String filePath, boolean
openedFile, boolean savedAs) throws CSVExceptions {
        /*If that checks whether the user has already created a population and if the file path is null
(if it is null it is either because the file
        has not been saved yet or the program has just started). In any of these scenarios, a
message is printed stating what the problem is */

```

Abril Maitena Betti

```

        if (populationCreated == null) {
            throw new InputExceptions(InputExceptions.ErrorTypeInput.NULL_POPULATION);
        } else if (!openedFile) { //If the boolean variable is false, it is because we have not opened a
population yet
            throw new CSVExceptions(CSVExceptions.ErrorTypeCSV.NO_FILE_OPENED);
        } else if (!savedAs) {
            throw new
CSVExceptions(CSVExceptions.ErrorTypeCSV.POPULATION_WAS_NEVER_SAVED);
        }
        else { //The file is saved
            try (BufferedWriter bfW = new BufferedWriter(new FileWriter(filePath))) {
                //I write the data of the population in the file, separating each field by a comma
                bfW.write(populationCreated.getNamePopulation() + ", " +
populationCreated.getNameResponsible() + ", " + populationCreated.getDaysProcreation());
                bfW.newLine();
                //I create a copy of the array of mice of the population I am saving which is as long as
the amount of mice the population has
                List<Mouse> arrayListMice = new
ArrayList<>(populationCreated.getCopyOfMiceInPopulation());

                //For each that runs as many times as mice were added to the population
                for (Mouse mouse : arrayListMice) {
                    //I write the data of the mouse stored in the position i of the copied array in the file,
separating each field with a comma
                    bfW.write(mouse.getBirthDate() + ", " + mouse.getWeightGr() + ", " +
mouse.getChromosome1() + ", " + mouse.getChromosome2()
                        + ", " + mouse.getGender() + ", " + mouse.getTemperatureInC() + ", " +
mouse.getDescription() + ", " + mouse.getRefCode());
                    bfW.newLine();
                }
            } catch (IOException | NullPointerException e) {
                System.out.println("ERROR: there was an error when saving the file.");
            }
        }
    }
}

/**
 * Function that saves the created population in a text file.
 * @param newFilePath path introduced by the user
 * @param populationCreated stores the population the user wants to 'save as' if he/she has
created one or null otherwise
 * @throws InputExceptions if the user has not created a population yet.
 * @throws CSVExceptions if the user has not opened a file.
 */
public static void savePopulationAs(Population populationCreated, String newFilePath, boolean
fileOpened, boolean savedAs) throws InputExceptions, CSVExceptions {
    if (populationCreated == null) {
        throw new InputExceptions(InputExceptions.ErrorTypeInput.NULL_POPULATION);
    }
    else if (!fileOpened) { //If the boolean variable is false, it is because we have not opened a
population yet

```

Abril Maitena Betti

```

        throw new CSVExceptions(CSVExceptions.ErrorTypeCSV.NO_FILE_OPENED);
    }
    else if(savedAs){
        throw new
CSVExceptions(CSVExceptions.ErrorTypeCSV.POPULATION_WAS_ALREADY_SAVED_AS);
    }
    else {
        try (BufferedWriter bfW = new BufferedWriter(new FileWriter(newFilePath))) {
            bfW.write(populationCreated.getNamePopulation() + ", " +
populationCreated.getNameResponsible() + ", " + populationCreated.getDaysProcreation());
            bfW.newLine();
            //I create a copy of the list of mice in the population by calling the method of population
that does so.
            List<Mouse> arrayListMice = new
ArrayList<>(populationCreated.getCopyOfMicelnPopulation());
            //For each that runs the list of mice
            for(Mouse mouse : arrayListMice) {
                bfW.write(mouse.getBirthDate() + ", " + mouse.getWeightGr() + ", " +
mouse.getChromosome1() + ", " + mouse.getChromosome2()
                    + ", " + mouse.getGender() + ", " + mouse.getTemperatureInC() + ", " +
mouse.getDescription() + ", " + mouse.getRefCode());
                bfW.newLine();
            }
        } catch (IOException | NullPointerException e) {
            System.out.println("ERROR: there was an error saving the file.");
        }
    }
}
}
}

```

E. Paquete Exceptions

I. Clase MouseExceptions

```
package aed.Exceptions;
```

```

/**
 * Class that contains personalised exceptions in order to handle with unexpected scenarios
regarding the use of "Mouse" instances.
 * @author abrilbetti
 */
public class MouseExceptions extends RuntimeException {
    /**
     * Enumerate that contains the different types of exceptions we can find when dealing with
instances of the class mouse.
     */
    public enum ErrorTypeMouse{

```

Abril Maitena Betti

```

        INVALID_WEIGHT, INVALID_TEMPERATURE, INVALID_CHROMOSOME_1,
        INVALID_CHROMOSOME_2, NON_CONCORDANT_GENDER, INVALID_BIRTHDAY
    }

```

```

    public ErrorTypeMouse errorTypeMouse;

```

```

    /**

```

* Constructor which initialises the only attribute of this class, "errorTypeMouse" with a value which is received as parameter.

* @param errorTypeMouse contains the type of exception, which is determined when creating an instance of 'Mouse' and encountering diverse unexpected situations.

```

    */

```

```

    public MouseExceptions(ErrorTypeMouse errorTypeMouse){
        this.errorTypeMouse = errorTypeMouse;
    }

```

```

    /**

```

* Getter for the attribute of the "MouseExceptions" class which makes the attribute of this class accessible

* @return the attribute of the class

```

    */

```

```

    public ErrorTypeMouse getErrorTypeMouse(){
        return errorTypeMouse;
    }

```

```

    /**

```

* Method to string which provides the string representation of the instances of this class. It contains a switch case which

* determines a specific string for each type of exception, which is to be returned in order to let the user know what was the error

* @return a string which explains what was the exception

```

    */

```

```

    public String toString(){
        switch(getErrorTypeMouse()){
            case INVALID_WEIGHT:
                return "The introduced value of the mouse's weight is invalid.";
            case INVALID_TEMPERATURE:
                return "The introduced value of the mouse's temperature is invalid.";
            case INVALID_CHROMOSOME_1:
                return "The first chromosome you introduced is not valid.";
            case INVALID_CHROMOSOME_2:
                return "The second chromosome you introduced is not valid.";
            case INVALID_BIRTHDAY:
                return "The introduced date of the mouse's birthday is invalid.";
            case NON_CONCORDANT_GENDER:
                return "The introduced gender does not match the introduced chromosomes.";
            default:
                return "An error has occurred.";
        }
    }
}

```

II. Clase **PopulationExceptions**

```
package aed.Exceptions;
/**
 * Class that contains personalised exceptions in order to handle with unexpected scenarios
 regarding the use of "Population" instances.
 * @author abrilbetti
 */
public class PopulationExceptions extends RuntimeException {
    /**
     * Enumerate that contains the different types of exceptions we can find when dealing with
 instances of the class population
     */
    public enum ErrorType{
        MOUSE_NOT_FOUND, INVALID_PROCREATION_DAYS, INVALID_NAME,
        NON_MULTIPLE_45_PROCREATION_DAYS, INVALID_TYPE_PROCREATION_DAYS,
        FAILED_TO_REMOVE_MOUSE, FAILED_TO_ADD_MOUSE,
        INVALID_NUMBER_OF_MICE, INVALID_PERCENTAGE_MALES,
        INVALID_PERCENTAGE_FEMALES, INVALID_PERCENTAGE_STERILE_MALES,
        INVALID_PERCENTAGE_POLYGAMOUS_MALES, INVALID_PERCENTAGE_XMUT,
        INVALID_MALE_FEMALE_PERCENTAGES
    }

    public ErrorType errorType;

    /**
     * Constructor that initialises the only attribute of this class, "errorType" with a value which is
 received as parameter.
     * @param errorType contains the type of exception, which is determined when creating an
 instance of 'Population' and encountering diverse unexpected situations.
     */
    public PopulationExceptions(ErrorType errorType){
        this.errorType = errorType;
    }
    /**
     * Getter for the attribute of the "PopulationExceptions" class which makes the attribute of this
 class accessible
     * @return the attribute of the class
     */
    public ErrorType getErrorType() {
        return errorType;
    }
    /**
     * Method to string which provides the string representation of the instances of this class. It
 contains a switch case which
     * determines a specific string for each type of exception, which is to be returned in order to let
 the user know what was the error
     * @return a string which explains what was the exception
     */
    public String toString(){
        switch(getErrorType()){
            case MOUSE_NOT_FOUND:
                return "There is no mouse with that reference code.";
            case INVALID_PROCREATION_DAYS:
                return "The days of procreation of the population is invalid (it must be a number
 between 1 and 630).";
            case NON_MULTIPLE_45_PROCREATION_DAYS:
```

Abril Maitena Betti

```

        return "The introduced days of procreation of the population is invalid because it is not
a multiple of 45.";
    case INVALID_TYPE_PROCREATION_DAYS:
        return "The input you introduced is invalid as it is not an integer.";
    case INVALID_NAME:
        return "The name of the population's responsible introduced is invalid.";
    case FAILED_TO_REMOVE_MOUSE:
        return "There was an error when trying to remove the mouse from the population.";
    case FAILED_TO_ADD_MOUSE:
        return "The mouse you are trying to add to the population is invalid.";
    case INVALID_NUMBER_OF_MICE:
        return "The introduced value of the total number of mice is invalid.";
    case INVALID_PERCENTAGE_MALES:
        return "The introduced value of the percentage of male mice is invalid.";
    case INVALID_PERCENTAGE_FEMALES:
        return "The introduced value of the percentage of female mice is invalid.";
    case INVALID_PERCENTAGE_STERILE_MALES:
        return "The introduced value of the percentage of sterile mice is invalid.";
    case INVALID_PERCENTAGE_POLYGAMOUS_MALES:
        return "The introduced value of the percentage of polygamous mice is invalid.";
    case INVALID_PERCENTAGE_XMUT:
        return "The introduced value of the percentage of muted 'x' chromosomes is invalid.";
    case INVALID_MALE_FEMALE_PERCENTAGES:
        return "The introduced percentages of male and female mice do not add up to 100 and,
therefore, are invalid.";
    default:
        return "An error has occurred.";
    }
}
}
}

```

III. Clase **CSVExceptions**

```
package aed.Exceptions;
```

```

/**
 * Class that contains personalised exceptions in order to handle with unexpected scenarios
 regarding the use of files in the
 * "CSV" class, which may arise when trying to either read or save files.
 * @author abrilbetti
 */
public class CSVExceptions extends RuntimeException {
    /**
     * Enumerate that contains the different types of exceptions we can find when dealing with files.
     */
    public enum ErrorTypeCSV{
        INVALID_FORMAT_OF_LINE_POPULATION, INVALID_FORMAT_OF_LINE_MOUSE,
        POPULATION_WAS_NEVER_SAVED, NO_FILE_OPENED,
        POPULATION_WAS_ALREADY_SAVED_AS
    }
    public ErrorTypeCSV errorTypeCSV;

    /**
     * Constructor that initialises the only attribute of this class, "errorTypeCSV" with a value which is
     received as parameter.
     * @param errorTypeCSV contains the type of exception, which is determined when working with
     the class "CSV" and encountering diverse unexpected situations.
     */
    public CSVExceptions(ErrorTypeCSV errorTypeCSV){
Abril Maitena Betti

```

```

    this.errorTypeCSV = errorTypeCSV;
}
/**
 * Getter for the attribute of the "CSVExceptions" class which makes the attribute of this class
accessible.
 * @return the attribute of the class.
 */
public ErrorTypeCSV getErrorTypeCSV() {
    return errorTypeCSV;
}
/**
 * Method to string which provides the string representation of the instances of these classes. It
contains a switch case which
 * determines a specific string for each type of exception, which is to be returned in order to let
the user know why was the exception thrown.
 * @return a string which explains what was the exception.
 */
public String toString() {
    switch (getErrorTypeCSV()) {
        case INVALID_FORMAT_OF_LINE_POPULATION:
            return "The format of the line read from the file does not coincide with the format of a
population.";
        case INVALID_FORMAT_OF_LINE_MOUSE:
            return "The format of the line read from the file does not coincide with the format of a
mouse.";
        case POPULATION_WAS_NEVER_SAVED:
            return "The population was never saved. Redirecting to 'Save as'...";
        case NO_FILE_OPENED:
            return "You have not opened a file yet. Please do so before you try to save it.";
        case POPULATION_WAS_ALREADY_SAVED_AS:
            return "The population has already been saved as. Redirecting to 'Save'...";
        default:
            return "An error has occurred.";
    }
}
}
}

```

IV. Clase **InputExceptions**

```

package aed.Exceptions;
/**
 * Class that contains personalised exceptions in order to handle with unexpected scenarios
regarding the use of the
 * methods of "Input" class.
 * @author abrilbetti
 */
public class InputExceptions extends RuntimeException {
    /**
     * Enumerate that contains the different types of exceptions we can find when using the
methods of Input.
     */
    public enum ErrorTypeInput{
        NULL_POPULATION, EMPTY_LIST_OF_MICE
    }
    public ErrorTypeInput errorTypeInput;
}
/**

```

```

    * Constructor that initialises the only attribute of this class, "errorTypeInput" with a value which
    is received as parameter.
    * @param errorTypeInput contains the type of exception, which is determined when working
    with the class "Input" and encountering diverse unexpected situations.
    */
    public InputExceptions(ErrorTypeInput errorTypeInput){
        this.errorTypeInput = errorTypeInput;
    }
    /**
    * Getter for the attribute of the "InputExceptions" class which makes the attribute of this class
    accessible.
    * @return the attribute of the class.
    */
    public ErrorTypeInput getErrorTypeInput() {
        return errorTypeInput;
    }
    /**
    * Method to string which provides the string representation of the instances of these classes. It
    contains a switch case which
    * determines a specific string for each type of exception, which is to be returned in order to let
    the user know why was the exception thrown.
    * @return a string which explains what was the exception.
    */
    public String toString() {
        switch (getErrorTypeInput()) {
            case NULL_POPULATION:
                return "You have not created a population yet. Please make sure you do before you try
                to use this operation.";
            case EMPTY_LIST_OF_MICE:
                return "You have not added mice to your population yet. Please do so prior to using this
                operation.";
            default:
                return "An error has occurred.";
        }
    }
}

```

V. Clase **FamilyExceptions**

```
package aed.Exceptions;
```

```

/**
 * Class that contains personalised exceptions in order to handle with unexpected scenarios
 regarding the use of the instances
 * of the "Family" class and its daughter classes.
 * @author abrilbetti
 */
public class FamilyExceptions extends RuntimeException {
    /**
    * Enumerate that contains the different types of exceptions we can find when dealing with
    instances of the class Family or its daughter classes.
    */
    public enum ErrorTypeFamily{
        INVALID_GENDER_FATHER, INVALID_GENDER_MOTHER, NO_FERTILE_MALE_ASSIGNED,
        MUTATION_MALE_LACKS_POLYGAMY, MUTATION_MALE_LACKS_STERILITY,
        MUTATION_MALE_HAS_POLYGAMY, MUTATION_MALE_HAS_STERILITY,
        NO_MOTHER_ASSIGNED
    }
}

```

Abril Maitena Betti

```

public ErrorTypeFamily errorTypeFamily;

/**
 * Constructor that initialises the only attribute of this class, "errorTypeFamily" with a value
 which is received as parameter.
 * @param errorTypeFamily contains the type of exception, which is determined when creating
 an instance of 'Family' and encountering diverse unexpected situations.
 */
public FamilyExceptions(ErrorTypeFamily errorTypeFamily) {
    this.errorTypeFamily = errorTypeFamily;
}

/**
 * Getter for the attribute of the "FamilyExceptions" class which makes the attribute of this class
 accessible.
 * @return the attribute of the class.
 */
public ErrorTypeFamily getErrorTypeFamily() {
    return errorTypeFamily;
}

/**
 * Method to string which provides the string representation of the instances of these classes. It
 contains a switch case which
 * determines a specific string for each type of exception, which is to be returned in order to let
 the user know why was the exception thrown.
 * @return a string which explains what was the exception.
 */
public String toString() {
    switch (getErrorTypeFamily()) {
        case INVALID_GENDER_FATHER:
            return "The gender of the father of the family is not male, and therefore it is not valid.";
        case INVALID_GENDER_MOTHER:
            return "The gender of the mother of the family is not male, and therefore it is not valid.";
        case NO_FERTILE_MALE_ASSIGNED:
            return "This family will produce 0 offspring as it lacks a fertile male.";
        case MUTATION_MALE_LACKS_POLYGAMY:
            return "The male mouse of the family does not present the mutation of polygamy and he
should have it.";
        case MUTATION_MALE_LACKS_STERILITY:
            return "The male mouse of the family does not present the mutation of sterility and he
should have it.";
        case MUTATION_MALE_HAS_POLYGAMY:
            return "The male mouse of the family has the gen of polygamy and he should not have
it.";
        case MUTATION_MALE_HAS_STERILITY:
            return "The male mouse of the family has the gen of sterility and he should not have it.";
        case NO_MOTHER_ASSIGNED:
            return "This family was not assigned a female mouse.";
        default:
            return "An error has occurred.";
    }
}
}

```

F. Paquete **Families**

I. Clase **Family**

Abril Maitena Betti

```

package aed.Families;

import aed.Exceptions.FamilyExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Mouse;

import java.time.LocalDate;
import java.util.Iterator;
import java.util.Objects;
import java.util.Random;
import java.util.TreeSet;

public abstract class Family implements Comparable<Family> {
    protected Mouse fatherMouse;
    protected TreeSet<Mouse> mothersMice;
    protected Mouse biologicalFather;
    private final int familyID;
    private static int numFamily = 0;

    /**
     * Constructor of the class Family, in which an instance of Mouse and a set of Mouse objects
     are set as the father and the mothers
     * of the family.
     * @param _fatherMouse object of Mouse whose gender must be 'MALE' in order to be
     established as the father of the family.
     * @param _mothersMice set of Mouse objects whose genders must be 'FEMALE' in order to
     be established as the mother/mothers of the family.
     * @throws FamilyExceptions any of the parameters have a gender different to the one they
     should have or if the tree set of mothers is empty.
     */
    public Family(Mouse _fatherMouse, TreeSet<Mouse> _mothersMice) throws FamilyExceptions{
        if(_mothersMice.isEmpty()){
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.NO_MOTHER_ASSIGNED);
        }
        if(!checkMothersGender(_mothersMice)){
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.INVALID_GENDER_MOTHER);
        }
        if(_fatherMouse.getGender() != Gender.MALE){
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.INVALID_GENDER_FATHER);
        }
        familyID = generateFamilyID();
        this.fatherMouse = _fatherMouse;;
        this.mothersMice = _mothersMice;
    }

    /**
     * Getter for the attribute that contains the father of the family (a male mouse).
     * @return the male mouse.
     */
    public Mouse getFatherMouse() {
        return fatherMouse;
    }

    /**
     * Getter for the attribute that contains the biological father of the family (a male mouse).
     * @return the male mouse.

```

Abril Maitena Betti

```

*/
public Mouse getBiologicalFather() {
    return biologicalFather;
}

/**
 * Getter for the tree set that contains the mothers of the family.
 * @return the tree set of mothers.
 */
public TreeSet<Mouse> getMothersMice() {
    return mothersMice;
}

/**
 * Setter for the biological father that receives a mouse as parameter and sets it as the
biological father of the family.
 * @param biologicalFather mouse we want to set as the biological father.
 */
public void setBiologicalFather(Mouse biologicalFather) {
    this.biologicalFather = biologicalFather;
}

/**
 * Method that simulates the reproduction of the family.
 * @return a tree set that contains the offspring of the family.
 */
public abstract TreeSet<Mouse> reproduce(int daysPassed);

/**
 * Method that computes the amount of offspring a family will produce.
 * @return an integer that represents how many baby mice will be born in the family.
 */
protected abstract int amountOfBabyMice();

/**
 * Method that creates a random number and, based on the value of this number, establishes
what chromosome will the baby mouse inherit from the parent "mouse".
 * @return the inherited chromosome.
 */
protected Chromosome inheritChromosome(Mouse mouse){
    Random rand = new Random();
    int randNum = rand.nextInt(100);

    if(randNum < 50){
        //The baby mouse inherits the parent's first chromosome.
        return mouse.getChromosome1();
    } else{
        //The baby mouse inherits the parent's second chromosome.
        return mouse.getChromosome2();
    }
}

/**
 * Method that receives an integer by parameter and converts the tree set into an array in order
to have direct access to the elements that are contained in the tree set
 * (we are not accessing them through the tree set as it is not possible, which is why we need to
convert the tree set into an array)
 * @param position integer that stores the position in which the needed mouse is stored in the
tree set of mothers.
 * @return the mouse stored in the "position" position of the array.

```

```

    */
    protected Mouse retrieveMouseFromMothersSet(int position){
        if(mothersMice.size() > position) {
            Mouse[] mothers = mothersMice.toArray(new Mouse[mothersMice.size()]);
            return mothers[position];
        }
        throw new IndexOutOfBoundsException();
    }

    /**
     * Method that checks that the gender of all the mice in the tree set is female. If not, it returns
     false.
     * @return true if all the mice in the tree set are females; return false if any of the mice in the tree
     set is a male.
     */
    private boolean checkMothersGender(TreeSet<Mouse> mothers){
        /*I create an iterator instance in order to access the elements of the TreeSet (because
        TreeSets implement the Set interface,
        which does not provide a direct method to access elements)
        */
        Iterator<Mouse> iterator = mothers.iterator();
        //While the iterator has an element in the next position, I keep entering the 'for' and checking
        the mouse's gender.
        while(iterator.hasNext()) {
            for (int i = 0; i < mothers.size(); i++) {
                Mouse mother = iterator.next();
                if(mother.getGender() == Gender.MALE){
                    return false;
                }
            }
        }
        return true;
    }

    /**
     * Static method that increments by one the variable that stores the amount of families that
     have been created, in order to initialise the ID of the family in
     * the constructor of Family.
     * @return an integer incremented by one, which represents how many families have been
     created.
     */
    private static int generateFamilyID(){
        numFamily++;
        return numFamily;
    }

    /**
     * Method that receives a mouse and checks whether the mouse received as parameter is more
     than 2.5 years old. If it is, it returns false (the mouse should die)
     * or returns true otherwise (the mouse can continue living)
     * @param mouse Mouse object whose birthday we want to check in order to determine
     whether it has to die or not.
     * @return true if the mouse can continue living or false if it is too old and needs to die.
     */
    public boolean shouldBeAlive(Mouse mouse, int daysPassed){
        if(mouse != null) {
            LocalDate birthday = mouse.getBirthDate();
            //LocalDate eldestDate = birthday.minusDays((long) (2.5 * 365));
            LocalDate simulatedDate = LocalDate.now().plusDays(daysPassed);
            LocalDate eldestDate = simulatedDate.minusDays((long) (2.5 * 365));

```

```

        if (birthday.isBefore(eldestDate)) {
            return false;
        }
        return true;
    }
    return false;
}

/**
 * Overridden equals method that establishes the criteria under which two families are
 considered to be equals (when their ID of family coincide).
 * @param o object we wish to check for equality with the object from which the method is
 called.
 * @return true if they are equal or false otherwise.
 */
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Family family)) return false;
    return familyID == family.familyID;
}

/**
 * Overridden hashCode method that generates a hash code for every family depending on their
 family IDs.
 * @return the hash code of the family with family ID "familyID"
 */
@Override
public int hashCode() {
    return Objects.hashCode(familyID);
}

/** Overridden compareTo method that compares two objects of the class Family and returns a
 0 if they are equal or a positive or negative number when they are different.
 * The method is final in order to prevent the child classes of Family to override it.
 * @param family instance of Family class that we wish to compare
 * @return 0 if the objects, according to the equals method, are equal or the difference of the
 two reference codes of the mice if they are not equal.
 */
@Override
public final int compareTo(Family family) {
    if (this.equals(family)) {
        return 0;
    }
    else {
        return this.familyID - family.familyID;
    }
}
}

```

II. Clase **NormalFamily**

```
package aed.Families;
```

```
import aed.Exceptions.FamilyExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Mouse;
```

```
import java.time.LocalDate;
import java.util.Random;
```

Abril Maitena Betti

```

import java.util.TreeSet;

public class NormalFamily extends Family {
    /**
     * Constructor of the class Family, in which an instance of Mouse and a set of Mouse objects
     * are set as the father and the mother/mothers
     * of the family.
     *
     * @param fatherMouse object of Mouse whose gender must be 'MALE' in order to be
     * established as the father of the family.
     * @param mothersMice set of Mouse objects whose genders must be 'FEMALE' in order to be
     * established as the mother/mothers of the family.
     * @throws FamilyExceptions if the gender of the father or any of the mothers is not the
     * appropriate one.
     */
    public NormalFamily(Mouse fatherMouse, TreeSet<Mouse> mothersMice) throws
    FamilyExceptions {
        super(fatherMouse, mothersMice);
        if(fatherMouse.isSterile()){
            throw new
    FamilyExceptions(FamilyExceptions.ErrorTypeFamily.NO_FERTILE_MALE_ASSIGNED);
        }
        if(fatherMouse.isPolygamous()){
            throw new
    FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_HAS_POLYGAMY);
        }
    }

    /**
     * Overridden method reproduce, that generates the virtual baby mice and adds them to the list
     * that contains the offspring of the family.
     * @return the tree set that stores the virtual offspring of the family.
     */
    @Override
    public TreeSet<Mouse> reproduce(int daysPassed) {
        TreeSet<Mouse> breedingMice = new TreeSet<>();
        Chromosome babyMouseChromosome1;
        Chromosome babyMouseChromosome2;
        //I get the female mouse stored in the position 'j' of the set of mothers.
        Mouse biologicalMother = retrieveMouseFromMothersSet(0);
        //I create a variable tha will store the amount of baby mice that the couple produces and set
        it to 0.
        int babies = 0;
        //I will change the value of the 'babies' variable by calling the appropriate method only if the
        female mouse is non-sterile.
        if(!biologicalMother.isSterile()) {
            babies = amountOfBabyMice();
        }
        for(int i = 0; i < babies; i++) {
            //The baby mouse inherits the first chromosome from its mother and its second one from
            the father.
            babyMouseChromosome1 = inheritChromosome(biologicalMother);
            babyMouseChromosome2 = inheritChromosome(fatherMouse);
            //The birthdate of the virtual offspring will be the present date plus the amount of days that
            have passed (according to our simulation) minus 25 days because the
            //cycle's duration is 45 days but during its last 25 days the mice get independent (i.e.
            when a cycle finishes, its offspring are 25 days old)
            Mouse babyMouse = new Mouse(babyMouseChromosome1, babyMouseChromosome2,
            LocalDate.now().plusDays(daysPassed - 25));
            //I add the created baby mouse to the set of the offspring.

```

```

        breedingMice.add(babyMouse);
    }
    return breedingMice;
}

/**
 * Overridden method that calculates the amount of offspring the family will produce by
generating a random number.
 * @return an integer which represents the amount of baby mice to be born.
 */
@Override
public int amountOfBabyMice() {
    Random rand = new Random();
    //I generate a random number from 0 to 99 and use it in order to determine the amount of
offspring the family will have.
    int randNum = rand.nextInt(100);
    if(randNum < 5){
        return 2;
    } else if(randNum >= 5 && randNum < 15){
        return 3;
    } else if(randNum >= 15 && randNum < 30){
        return 4;
    } else if(randNum >= 30 && randNum < 50){
        return 5;
    } else if(randNum >= 50 && randNum < 70){
        return 6;
    } else if(randNum >= 70 && randNum < 85){
        return 7;
    } else if(randNum >= 85 && randNum < 95){
        return 8;
    } else { //randNum >= 95 && randNum <= 99
        return 9;
    }
}
}
}

```

III. Clase **PolygamousFamily**

```
package aed.Families;
```

```
import aed.Exceptions.FamilyExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Mouse;
```

```
import java.time.LocalDate;
import java.util.Random;
import java.util.TreeSet;
```

```
public class PolygamousFamily extends Family {
```

```

    /**
     * Constructor of the class Family, in which an instance of Mouse and a set of Mouse objects
are set as the father and the mother/mothers
     * of the family.
     *
     * @param fatherMouse object of Mouse whose gender must be 'MALE' in order to be
established as the father of the family.
     * @param mothersMice set of Mouse objects whose genders must be 'FEMALE' in order to be
established as the mother/mothers of the family.

```

Abril Maitena Betti

* @throws FamilyExceptions is the gender of the father or any of the mothers is not the appropriate one.

```
*/
public PolygamousFamily(Mouse fatherMouse, TreeSet<Mouse> mothersMice) throws
FamilyExceptions {
    super(fatherMouse, mothersMice);
    if(fatherMouse.isSterile()){
        throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_HAS_STERILITY);
    }
    if(!fatherMouse.isPolygamous()){
        throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_LACKS_POLYGAMY);
    }
}
```

/**
* Overridden method reproduce, that generates the virtual baby mice and adds them to the list that contains the offspring of the family.

* @return the tree set that stores the virtual offspring of the family.

```
*/
@Override
public TreeSet<Mouse> reproduce(int daysPassed) {
    TreeSet<Mouse> breedingMice = new TreeSet<>();
    Chromosome babyMouseChromosome1;
    Chromosome babyMouseChromosome2;
    int babies = 0;
    //Nested 'for' loops that generate the offspring each female produces.
    for(int j=0; j<mothersMice.size(); j++) {
        //I get the female mouse stored in the position 'j' of the set of mothers.
        Mouse biologicalMother = retrieveMouseFromMothersSet(j);
        //I call the method to compute the amount of babies this female will produce only if it is
non-sterile. If it is sterile, the amount of babies will be 0 and the following for will be skipped
        if(!biologicalMother.isSterile()) {
            //Each female mouse can have a different amount of offspring, which is why the value of
the variable 'baby' is changed every time.
            babies = amountOfBabyMice();
        }
        for (int i = 0; i < babies; i++) {
            //The baby mouse inherits its first chromosome from the mother and the second one
from its father.
            babyMouseChromosome1 = inheritChromosome(biologicalMother);
            babyMouseChromosome2 = inheritChromosome(fatherMouse);
            //I create the mice, passing as parameter the inherited chromosomes and the
appropriate birthdate.
            //The birthdate of the virtual offspring will be the present date plus the amount of days
that have passed (according to our simulation) minus 25 days because the
            //cycle's duration is 45 days but during its last 25 days the mice get independent (i.e.
when a cycle finishes, its offspring are 25 days old)
            Mouse babyMouse = new Mouse(babyMouseChromosome1,
babyMouseChromosome2, LocalDate.now().plusDays(daysPassed - 25));
            breedingMice.add(babyMouse);
        }
    }
    return breedingMice;
}
```

/**
* Overridden method that calculates the amount of offspring the family will produce by generating a random number. In addition, it is

```

    * checked that the biological father is not null (i.e. that a fertile mouse was assigned to this
    family)
    * @return an integer which represents the amount of baby mice to be born.
    */
    @Override
    public int amountOfBabyMice() {
        Random rand = new Random();
        //I generate a random number from 0 to 99 and use it in order to determine the amount of
        offspring the family will have.
        int randNum = rand.nextInt(100);
        if(randNum < 10){
            return 2;
        } else if(randNum >= 10 && randNum < 25){
            return 3;
        } else if(randNum >= 25 && randNum < 45){
            return 4;
        } else if(randNum >= 45 && randNum < 60){
            return 5;
        } else if(randNum >= 60 && randNum < 75){
            return 6;
        } else if(randNum >= 75 && randNum < 95){
            return 7;
        } else {
            return 8;
        }
    }
}

```

IV. Clase **SterileMaleFamily**

```
package aed.Families;
```

```
import aed.Exceptions.FamilyExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Mouse;
```

```
import java.time.LocalDate;
import java.util.Random;
import java.util.TreeSet;
```

```

public class SterileMaleFamily extends Family {
    /**
     * Constructor of the class Family, in which an instance of Mouse and a set of Mouse objects
     are set as the father and the mothers
     * of the family.
     *
     * @param fatherMouse object of Mouse whose gender must be 'MALE' in order to be
     established as the father of the family. It must lack both polygamy and sterility,
     * @param mothersMice set of Mouse objects whose genders must be 'FEMALE' in order to be
     established as the mother/mothers of the family.
     * @param biologicalFatherMouse object of Mouse whose gender must be 'MALE' and must
     lack both mutations *sterility and polygamy)
     * @throws FamilyExceptions if the gender of the father or any of the mothers is not the
     appropriate one.
     */
}

```

```

    public SterileMaleFamily(Mouse fatherMouse, Mouse biologicalFatherMouse, TreeSet<Mouse>
mothersMice) throws FamilyExceptions {
        super(fatherMouse, mothersMice);
        /*If the biological father is null, it means that there were no more sterile and non-polygamous
male mice in the list of male mice and,
        it was not possible to assign one to this family. Therefore, as the assigned male of the family
is sterile and there is no second male
        mouse, the offspring of the family will be 0.
        */
        if(biologicalFatherMouse != null) {
            if (biologicalFatherMouse.isSterile()) {
                throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.NO_FERTILE_MALE_ASSIGNED);
            }
            if (biologicalFatherMouse.getGender() != Gender.MALE) {
                throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.INVALID_GENDER_FATHER);
            }
            if (biologicalFatherMouse.isPolygamous()) {
                throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_HAS_POLYGAMY);
            }
        }
        if (!fatherMouse.isSterile()) {
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_LACKS_STERILITY);
        }
        if (fatherMouse.isPolygamous()) {
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_HAS_POLYGAMY);
        }
        setBiologicalFather(biologicalFatherMouse);
    }

    /**
     * Overridden method reproduce, that generates the virtual baby mice and adds them to the list
that contains the offspring of the family.
     * @return the tree set that stores the virtual offspring of the family.
     */
    @Override
    public TreeSet<Mouse> reproduce(int daysPassed) {
        TreeSet<Mouse> breedingMice = new TreeSet<>();
        Chromosome babyMouseChromosome1;
        Chromosome babyMouseChromosome2;
        //I call the method that returns the mouse stored in the set (the position is 0 because the
family has only one mother).
        Mouse biologicalMother = retrieveMouseFromMothersSet(0);
        int babies = 0;
        //I call the method that calculates how many babies the couple will produce only if the female
mouse is non-sterile and if we have a fertile male mouse (if it is != null).
        if(!biologicalMother.isSterile() && biologicalFather != null){
            babies = amountOfBabyMice();
        }
        for(int i = 0; i < babies; i++) {
            //The baby mouse inherits its first chromosome from this female mouse.
            babyMouseChromosome1 = inheritChromosome(biologicalMother);
            //The baby mice inherits its second chromosome from its biological father (the sterile male)
            babyMouseChromosome2 = inheritChromosome(biologicalFather);
            //I create the mice, passing as parameter the inherited chromosomes and the appropriate
birthdate.

```

```

        //The birthdate of the virtual offspring will be the present date plus the amount of days that
        have passed (according to our simulation) minus 25 days because the
        //cycle's duration is 45 days but during its last 25 days the mice get independent (i.e.
        when a cycle finishes, its offspring are 25 days old)
        Mouse babyMouse = new Mouse(babyMouseChromosome1, babyMouseChromosome2,
        LocalDate.now().plusDays(daysPassed - 25));
        //I add the created mouse to the set of the offsprings
        breedingMice.add(babyMouse);
    }
    return breedingMice;
}

/**
 * Overridden method that calculates the amount of offspring the family will produce by
 generating a random number. In addition, it is
 * checked that the biological father is not null (i.e. that a fertile mouse was assigned to this
 family)
 * @return an integer which represents the amount of baby mice to be born.
 */
@Override
public int amountOfBabyMice() {
    Random rand = new Random();
    //I generate a random number from 0 to 99 and use it in order to determine the amount of
    offspring the family will have.
    int randNum = rand.nextInt(100);
    if (randNum < 15) {
        return 2;
    } else if (randNum >= 15 && randNum < 35) {
        return 3;
    } else if (randNum >= 35 && randNum < 70) {
        return 4;
    } else if (randNum >= 70 && randNum < 90) {
        return 5;
    } else { //randNum >= 90 && randNum <= 99
        return 6;
    }
}
}
}

```

V. Clase **SterilePolygamousMaleFamily**

```
package aed.Families;
```

```
import aed.Exceptions.FamilyExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Mouse;
```

```
import java.time.LocalDate;
import java.util.Random;
import java.util.TreeSet;
```

```
public class SterilePolygamousMaleFamily extends Family {
    /**
```

```

        * Constructor of the class Family, in which two instances of Mouse and a set of Mouse objects
        are set as the father and biological father and the mothers
        * of the family.
    */
}

```

Abril Maitena Betti

```

*
* @param fatherMouse object of Mouse whose gender must be 'MALE' in order to be
established as the father of the family. It must be polygamous and sterile.
* @param mothersMice set of Mouse objects whose genders must be 'FEMALE' in order to be
established as the mother/mothers of the family.
* @param biologicalFatherMouse object of Mouse whose gender must be 'MALE' and must
possess the mutation of polygamy but lack the gen of sterility.
* @throws FamilyExceptions if the gender of the father or any of the mothers is not the
appropriate one.
*/
public SterilePolygamousMaleFamily(Mouse fatherMouse, Mouse
biologicalFatherMouse, TreeSet<Mouse> mothersMice) throws FamilyExceptions {
    super(fatherMouse, mothersMice);
    /*If the biological father is null, it means that there were no more sterile and non-polygamous
male mice in the list of male mice and,
it was not possible to assign one to this family. Therefore, as the assigned male of the family
is sterile and there is no second male
mouse, the offspring of the family will be 0.
*/
    if(biologicalFatherMouse != null) {
        if (biologicalFatherMouse.isSterile()) {
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.NO_FERTILE_MALE_ASSIGNED);
        }
        if (biologicalFatherMouse.getGender() != Gender.MALE) {
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.INVALID_GENDER_FATHER);
        }
        if (!biologicalFatherMouse.isPolygamous()) {
            throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_LACKS_POLYGAMY);
        }
    }
    if (!fatherMouse.isPolygamous()) {
        throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_LACKS_POLYGAMY);
    }
    if(!fatherMouse.isSterile()){
        throw new
FamilyExceptions(FamilyExceptions.ErrorTypeFamily.MUTATION_MALE_LACKS_STERILITY);
    }
    setBiologicalFather(biologicalFatherMouse);
}

/**
* Overridden method reproduce, that generates the virtual baby mice and adds them to the list
that contains the offspring of the family.
* @return the tree set that stores the virtual offspring of the family.
*/
@Override
public TreeSet<Mouse> reproduce(int daysPassed) {
    TreeSet<Mouse> breedingMice = new TreeSet<>();
    Chromosome babyMouseChromosome1;
    Chromosome babyMouseChromosome2;
    int babies = 0;
    //Nested 'for' loops that generate the offspring each female produces.
    for(int j=0; j<mothersMice.size(); j++) {
        //I get the female mouse stored in the position 'j' of the set of mothers.
        Mouse biologicalMother = retrieveMouseFromMothersSet(j);

```

```

        //I call the method to compute the amount of babies this female will produce only if it is
        non-sterile. If it is sterile, the amount of babies will be 0 and the following for will be skipped
        if(!biologicalMother.isSterile() && biologicalFather != null) {
            //Each female mouse can have a different amount of offspring, which is why the value of
            the variable 'baby' is changed every time.
            babies = amountOfBabyMice();
        }
        for (int i = 0; i < babies; i++) {
            //The baby mouse inherits its first chromosome from the mother and the second one
            from its biological father.
            babyMouseChromosome1 = inheritChromosome(biologicalMother);
            babyMouseChromosome2 = inheritChromosome(biologicalFather);
            //I create the mice, passing as parameter the inherited chromosomes and the
            appropriate birthdate.
            //The birthdate of the virtual offspring will be the present date plus the amount of days
            that have passed (according to our simulation) minus 25 days because the
            //cycle's duration is 45 days but during its last 25 days the mice get independent (i.e.
            when a cycle finishes, its offspring are 25 days old)
            Mouse babyMouse = new Mouse(babyMouseChromosome1,
            babyMouseChromosome2, LocalDate.now().plusDays(daysPassed - 25));
            breedingMice.add(babyMouse);
        }
    }
    return breedingMice;
}

/**
 * Overridden method that calculates the amount of offspring the family will produce by
 * generating a random number. In addition, it is
 * checked that the biological father is not null (i.e. that a fertile mouse was assigned to this
 * family)
 * @return an integer which represents the amount of baby mice to be born.
 */
@Override
public int amountOfBabyMice() {
    Random rand = new Random();
    //I generate a random number from 0 to 99 and use it in order to determine the amount of
    offspring the family will have.
    int randNum = rand.nextInt(100);
    if (randNum < 15) {
        return 2;
    } else if (randNum >= 15 && randNum < 35) {
        return 3;
    } else if (randNum >= 35 && randNum < 70) {
        return 4;
    } else if (randNum >= 70 && randNum < 90) {
        return 5;
    } else { //randNum >= 90 && randNum <= 99
        return 6;
    }
}
}

```

G. Package **Comparators**

I. Clase **ComparatorByWeight**

Abril Maitena Betti

```

package aed.Comparators;

import aed.Laboratory.Mouse;
import java.util.Comparator;

/**
 * Class that implements the interface 'Comparator' in order to compare Mouse objects based on
 their weight.
 * @author abrilbetti
 */
public class ComparatorByWeight implements Comparator<Mouse>{
    /**
     * Overridden method that defines the natural order to sort the mice in increasing order of their
 weights in grams.
     * @param mouse1 the first object to be compared.
     * @param mouse2 the second object to be compared.
     * @return a negative integer, a call to the compareTo method of the Mouse class (which
 evaluates if the objects are equal)
     * or a positive integer as the first argument's weight (mouse1) is less, equal or greater than the
 second argument's weight (mouse2), respectively.
     */
    @Override
    public int compare(Mouse mouse1, Mouse mouse2) {
        int comparison = mouse1.getWeightGr() - mouse2.getWeightGr();
        if(comparison == 0){
            //If the weights of the instances are the same, I call the compare to method of Mouse in
 order to check equality
            return mouse1.compareTo(mouse2);
        }
        else{ //If the weights of the objects are different, then I return the value I got when comparing
 them.
            return comparison;
        }
    }
}

```

II. Clase **ComparatorByBirthdate**

```

package aed.Comparators;

import aed.Laboratory.Mouse;
import java.util.Comparator;

/**
 * Class that defines the comparator that we use to order the mice chronologically.
 * @author abrilbetti
 */
public class ComparatorByBirthdate implements Comparator<Mouse> {
    /**
     * Overridden method that defines the natural order to sort the mice chronologically, depending
 on their birthdays.
     * @param mouse1 the first object to be compared.
     * @param mouse2 the second object to be compared.
     * @return a negative integer, a call to the compareTo method of the Mouse class or a positive
 integer as the first argument's birthday (mouse1)
     * was before, on the same date or after the second argument's birthday (mouse2), respectively.
     */
}

```

Abril Maitena Betti

```

    */
    @Override
    public int compare(Mouse mouse1, Mouse mouse2) {
        int comparison = mouse1.getBirthDate().compareTo(mouse2.getBirthDate());
        //If the birthdays of the instances coincide, I call the method compare to of Mouse class in
order to check for equality.
        if(comparison == 0){
            return mouse1.compareTo(mouse2);
        }
        else{
            //If the birthdays are not the same, I return the value of the comparison of the dates.
            return comparison;
        }
    }
}

```

4. Listado de pruebas unitarias realizadas.

A. Paquete **CSV Administration**

I. **CSVTest**

```

package aed.CSVAdministration;

import aed.Exceptions.CSVExceptions;
import aed.Exceptions.InputExceptions;
import aed.Laboratory.Chromosome;
import aed.Laboratory.Gender;
import aed.Laboratory.Mouse;
import aed.Laboratory.Population;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;

import static org.junit.jupiter.api.Assertions.*;

class CSVTest {

    @Test
    void testOpenPopulationFromFileCorrectly() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/openFileCorrectly.CSV";
        Population expectedPopulationToBeCreated = new Population("Population1", "Abril", 90);
        LocalDate date1 = LocalDate.of(2020, 2, 14);
        LocalDate date2 = LocalDate.of(2024, 8, 30);
        LocalDate date3 = LocalDate.of(2025, 1, 4);
        Mouse mouse1 = new Mouse(date1, 234, Chromosome.X, Chromosome.Y, Gender.MALE,
24, "Old mouse");
        Mouse mouse2 = new Mouse(date2, 321, Chromosome.Xmut, Chromosome.Xmut,
Gender.FEMALE, 27, "sterile female mouse");
        Mouse mouse3 = new Mouse(date3, 245, Chromosome.Xmut, Chromosome.Ymut,
Gender.MALE, 33, "sterile and polygamous male mouse");
        expectedPopulationToBeCreated.addMiceToList(mouse1);
        expectedPopulationToBeCreated.addMiceToList(mouse2);
        expectedPopulationToBeCreated.addMiceToList(mouse3);
    }
}

```

Abril Maitena Betti

```

        Population opnedPopulation = CSV.openPopulationFromFile(filePath);
        assertEquals(expectedPopulationToBeCreated, opnedPopulation);
    }
    @Test
    void testSaveNullPopulation() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        assertThrows(InputExceptions.class, () -> {CSV.savePopulation(null, filePath, true,true)});
    }
    @Test
    void testSavePopulationNoFileOpened() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        Population opnedPopulation = CSV.openPopulationFromFile(filePath);
        assertThrows(CSVExceptions.class, () -> {CSV.savePopulation(opnedPopulation, filePath,
false,true)});
    }
    @Test
    void testSavePopulationNotSavedAs() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        Population opnedPopulation = CSV.openPopulationFromFile(filePath);
        assertThrows(CSVExceptions.class, () -> {CSV.savePopulation(opnedPopulation, filePath,
true,false)});
    }
    @Test
    void testSaveNullPopulationAs() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        assertThrows(InputExceptions.class, () -> {CSV.savePopulationAs(null, filePath, true,false)});
    }
    @Test
    void testSavePopulationAsNoFileOpened() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        Population opnedPopulation = CSV.openPopulationFromFile(filePath);
        assertThrows(CSVExceptions.class, () -> {CSV.savePopulationAs(opnedPopulation, filePath,
false,false)});
    }
    @Test
    void testSavePopulationAsAlreadySavedAs() {
        String filePath = "/Users/abrilbetti/Desktop/Algoritmos/Programas/saveFile.CSV";
        Population opnedPopulation = CSV.openPopulationFromFile(filePath);
        assertThrows(CSVExceptions.class, () -> {CSV.savePopulationAs(opnedPopulation, filePath,
true,true)});
    }
}

```

B. Paquete **Laboratory**

I. **GenderTest**

```

package aed.Laboratory;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class GenderTest {
    @Test
    void genderIsConcordant(){

```

Abril Maitena Betti

```

        boolean expectedValue = true;
        boolean realValue = Gender.isGenderConcordant(Chromosome.Y, Gender.MALE);
        assertEquals(expectedValue, realValue);
    }
    @Test
    void gendersIsNotConcordant(){
        boolean expectedValue = false;
        boolean realValue = Gender.isGenderConcordant(Chromosome.Y, Gender.FEMALE);
        assertEquals(expectedValue, realValue);
    }
    @Test
    void assignGenderToMaleVirtualMouse(){
        Gender realGender = Gender.assignGenderToVirtualMouse(Chromosome.Y);
        Gender expectedGender = Gender.MALE;
        assertEquals(expectedGender, realGender);
    }
    @Test
    void assignGenderToMutatedMaleVirtualMouse(){
        Gender realGender = Gender.assignGenderToVirtualMouse(Chromosome.Ymut);
        Gender expectedGender = Gender.MALE;
        assertEquals(expectedGender, realGender);
    }
    @Test
    void assignGenderToFemaleVirtualMouse(){
        Gender realGender = Gender.assignGenderToVirtualMouse(Chromosome.X);
        Gender expectedGender = Gender.FEMALE;
        assertEquals(expectedGender, realGender);
    }
    @Test
    void assignGenderToMutatedFemaleVirtualMouse(){
        Gender realGender = Gender.assignGenderToVirtualMouse(Chromosome.Xmut);
        Gender expectedGender = Gender.FEMALE;
        assertEquals(expectedGender, realGender);
    }
}

```

II. MouseTest

```

package aed.Laboratory;

import aed.Exceptions.MouseExceptions;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;

import static org.junit.jupiter.api.Assertions.*;
class MouseTest {

    @Test
    void constructorFutureDate(){
        LocalDate date = LocalDate.of(2025, 12, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, Chromosome.X,
        Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
    }
    @Test
    void constructorNegativeWeight(){
        LocalDate date = LocalDate.of(2024, 10, 3);

```

Abril Maitena Betti

```

        assertThrows(MouseExceptions.class, () -> new Mouse(date, -400, Chromosome.X,
Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
    }
    @Test
    void constructorWeight9(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 9, Chromosome.X,
Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
    }
    @Test
    void constructorWeight501(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 501, Chromosome.X,
Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
    }
    @Test
    void constructorWeight10(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        try {
            Mouse mouse1 = new Mouse(date, 10, Chromosome.X, Chromosome.Y, Gender.MALE,
36.8f, "Nice mouse");
            int weightMouse1 = 10;
            assertEquals(weightMouse1, mouse1.getWeightGr());
        } catch (MouseExceptions me) {
            System.out.println("ERROR: " + me);
        }
    }
    @Test
    void constructorWeight500(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        try {
            Mouse mouse1 = new Mouse(date, 500, Chromosome.X, Chromosome.Y, Gender.MALE,
36.8f, "Nice mouse");
            int weightMouse1 = 500;
            assertEquals(weightMouse1, mouse1.getWeightGr());
        } catch (MouseExceptions me) {
            System.out.println("ERROR: " + me);
        }
    }
    @Test
    void constructorNegativeTemperature(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, Chromosome.X,
Chromosome.Y, Gender.MALE, -36.8f, "Nice mouse"));
    }
    @Test
    void constructorTemperature0(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, Chromosome.X,
Chromosome.Y, Gender.MALE, 0f, "Nice mouse"));
    }
    @Test
    void constructorTemperature451(){
        LocalDate date = LocalDate.of(2024, 10, 3);
        assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, Chromosome.X,
Chromosome.Y, Gender.MALE, 45.1f, "Nice mouse"));
    }
    @Test
    void constructorTemperature1(){
        LocalDate date = LocalDate.of(2024, 10, 3);

```

```

    try {
        Mouse mouse1 = new Mouse(date, 10, Chromosome.X, Chromosome.Y, Gender.MALE, 1f,
"Nice mouse");
        float temperatureMouse1 = 1f;
        assertEquals(temperatureMouse1, mouse1.getTemperatureInC());
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void constructorTemperature45(){
    LocalDate date = LocalDate.of(2024, 10, 3);
    try {
        Mouse mouse1 = new Mouse(date, 10, Chromosome.X, Chromosome.Y, Gender.MALE,
45f, "Nice mouse");
        float temperatureMouse1 = 45f;
        assertEquals(temperatureMouse1, mouse1.getTemperatureInC());
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void constructorChromosome1Y(){
    LocalDate date = LocalDate.of(2020, 10, 3);
    assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, Chromosome.Y,
Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
}

@Test
void constructorInvalidChromosome(){
    LocalDate date = LocalDate.of(2020, 10, 3);
    String introducedString = "Ymut"; //either Y, Ymut or anything else
    Chromosome chromosome1 = Chromosome.valueOf(introducedString);
    assertThrows(MouseExceptions.class, () -> new Mouse(date, 400, chromosome1,
Chromosome.Y, Gender.MALE, 36.8f, "Nice mouse"));
}

@Test
void constructor2Chromosome1Y(){
    assertThrows(MouseExceptions.class, () -> new Mouse(Chromosome.Y, Chromosome.Y));
}

@Test
void constructor2InvalidChromosome(){
    String introducedString = "Ymut"; //either Y, Ymut or anything else
    Chromosome chromosome1 = Chromosome.valueOf(introducedString);
    assertThrows(MouseExceptions.class, () -> new Mouse(chromosome1, Chromosome.Y));
}

@Test
void constructor3Chromosome1Y(){
    LocalDate date = LocalDate.now();
    assertThrows(MouseExceptions.class, () -> new Mouse(Chromosome.Y, Chromosome.Y,
date));
}

@Test
void constructor3InvalidChromosome(){
    LocalDate date = LocalDate.now();

    String introducedString = "Ymut"; //either Y, Ymut or anything else
    Chromosome chromosome1 = Chromosome.valueOf(introducedString);
    assertThrows(MouseExceptions.class, () -> new Mouse(chromosome1, Chromosome.Y,
date));
}

```

```

@Test
void changeDataOfExistingMouse(){
    try {
        LocalDate date1 = LocalDate.of(2020, 1, 1);
        Mouse mouse1 = new Mouse(date1, 350, Chromosome.Xmut, Chromosome.Y,
Gender.MALE, 39.1f, "Good mouse");

        mouse1.changeMouseData(456, 34.5f, "Crazy mouse");
        int actualWeight = mouse1.getWeightGr();
        int expectedWeight = 456;
        assertEquals(expectedWeight, actualWeight);
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void isNotSterile(){
    try {
        LocalDate date = LocalDate.of(2020, 10, 3);
        Mouse mouse1 = new Mouse(date, 350, Chromosome.X, Chromosome.Y, Gender.MALE,
36.8f, "Nice mouse");
        boolean expectedValue = false;
        assertEquals(expectedValue, mouse1.isSterile());
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void isSterile(){
    try {
        LocalDate date = LocalDate.of(2020, 10, 3);
        Mouse mouse1 = new Mouse(date, 350, Chromosome.Xmut, Chromosome.Y,
Gender.MALE, 36.8f, "Nice mouse");
        boolean expectedValue = true;
        assertEquals(expectedValue, mouse1.isSterile());
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void isNotPolygamous(){
    try {
        LocalDate date = LocalDate.of(2020, 10, 3);
        Mouse mouse1 = new Mouse(date, 350, Chromosome.X, Chromosome.Y, Gender.MALE,
36.8f, "Nice mouse");
        boolean expectedValue = false;
        assertEquals(expectedValue, mouse1.isPolygamous());
    } catch(MouseExceptions me) {
        System.out.println("ERROR: " + me);
    }
}

@Test
void isPolygamous(){
    try {
        LocalDate date = LocalDate.of(2020, 10, 3);
        Mouse mouse1 = new Mouse(date, 350, Chromosome.X, Chromosome.Ymut,
Gender.MALE, 36.8f, "Nice mouse");
        boolean expectedValue = true;
        assertEquals(expectedValue, mouse1.isPolygamous());
    } catch(MouseExceptions me) {

```

```

        System.out.println("ERROR: " + me);
    }
}
@Test
void isSexuallyMatureTrue(){
    LocalDate date = LocalDate.of(2024, 10, 3);
    Mouse mouse = new Mouse(Chromosome.X, Chromosome.Y, date);
    boolean expectedValue = true;
    boolean result = mouse.isSexuallyMature(90);
    assertEquals(expectedValue, result);
}
@Test
void isSexuallyMatureFalse(){
    LocalDate date = LocalDate.of(2025, 01, 6);
    Mouse mouse = new Mouse(Chromosome.X, Chromosome.Y, date);
    boolean expectedValue = false;
    boolean result = mouse.isSexuallyMature(45);
    assertEquals(expectedValue, result);
}
}
}

```

III. PopulationTest

```

package aed.Laboratory;

import aed.Exceptions.PopulationExceptions;
import org.junit.jupiter.api.Test;

import java.time.LocalDate;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

class PopulationTest {
    @Test
    void constructor631ProcreationDays(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 631));
    }
    @Test
    void constructor0ProcreationDays(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 0));
    }
    @Test
    void constructorNegativeProcreationDays(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John",
-100));
    }
    @Test
    void constructor630ProcreationDays(){
        try {
            Population p1 = new Population("Population 1", "John", 630);
            int expectedProcreationDays = 630;
            assertEquals(expectedProcreationDays, p1.getDaysProcreation());
        } catch (PopulationExceptions pe) {
            System.out.println("ERROR: " + pe);
        }
    }
    @Test
    Abril Maitena Betti

```

```

void constructorMultipleOf45ProcreationDays(){
    try {
        Population p1 = new Population("Population 1", "John", 450);
        int expectedProcreationDays = 450;
        assertEquals(expectedProcreationDays, p1.getDaysProcreation());
    } catch (PopulationExceptions pe) {
        System.out.println("ERROR: " + pe);
    }
}

@Test
void constructorNonMultipleOf45ProcreationDays(){
    assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 449));
}

@Test
void constructorResponsibleNameNumbers(){
    assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John123",
90));
}

@Test
void constructorResponsibleNameEmpty(){
    assertThrows(PopulationExceptions.class, () -> new Population("Population 1", " ", 90));
}

//public Population(String namePopulation, String nameResponsible, int daysProcreation, int
numberOfMice, int percentageMales, int percentageSterileMales, int
percentagePolygamousMales, int percentageFemales, int percentageXmutInFemales) throws
PopulationExceptions {
    @Test
    void constructorAmountMice0(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45, 0,
10, 50, 40, 10, 70));
    }

    @Test
    void constructorInvalidAmountMice(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
-10, 10, 50, 40, 10, 70));
    }

    @Test
    void constructorZeroPercentageMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 0, 50, 40, 10, 70));
    }

    @Test
    void constructorBiggerPercentageMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 100, 50, 40, 10, 70));
    }

    @Test
    void constructorZeroPercentageFemales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 40, 0, 70));
    }

    @Test
    void constructorBiggerPercentageFemales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 40, 100, 70));
    }

    @Test
    void constructorPercentageMalesAndFemalesInvalid(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 50, 50, 40, 60, 70));
    }
}

```

```

    }
    @Test
    void constructorZeroPercentageSterileMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 0, 40, 10, 70));
    }
    @Test
    void constructorBiggerPercentageSterileMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 100, 40, 10, 70));
    }
    @Test
    void constructorZeroPercentagePolygamousMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 0, 10, 70));
    }
    @Test
    void constructorBiggerPercentagePolygamousMales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 100, 10, 70));
    }
    @Test
    void constructorZeroPercentageXmutFemales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 40, 10, 0));
    }
    @Test
    void constructorBiggerPercentageXmutFemales(){
        assertThrows(PopulationExceptions.class, () -> new Population("Population 1", "John", 45,
10, 10, 50, 40, 50, 100));
    }
    @Test
    void addMiceToListCorrect(){
        Population p1 = new Population("Population 1", "John", 90);
        Mouse mouse = new Mouse(Chromosome.X, Chromosome.Y, LocalDate.now());
        p1.addMiceToList(mouse);
        List<Mouse> micelnPopulation1 = p1.getCopyOfMicelnPopulation();
        int amountMicelnPopulation = micelnPopulation1.size();
        int expectedResult = 1;
        assertEquals(expectedResult, amountMicelnPopulation);
    }
    @Test
    void addMiceToListUnsuccessfully(){
        Population p1 = new Population("Population 1", "John", 90);
        LocalDate date1 = LocalDate.of(2020, 1, 1);
        Mouse mouse = null;
        assertThrows(PopulationExceptions.class, () -> {p1.addMiceToList(mouse);});
    }
    @Test
    void identifyMouseExistingRefCode(){
        try {
            Population p1 = new Population("Population 1", "John", 90);
            LocalDate date1 = LocalDate.of(2020, 1, 1);
            LocalDate date2 = LocalDate.of(2020, 1, 2);
            LocalDate date3 = LocalDate.of(2020, 1, 3);
            Mouse mouse7 = new Mouse(date1, 350, Chromosome.X, Chromosome.Y, Gender.MALE,
39.1f, "Good mouse");
            Mouse mouse8 = new Mouse(date2, 50, Chromosome.Xmut, Chromosome.X,
Gender.FEMALE, 37.1f, "Bad mouse");

```

```

        Mouse mouse9 = new Mouse(date3, 150, Chromosome.Xmut, Chromosome.Ymut,
Gender.MALE, 35.1f, "Fat mouse");
        p1.addMiceToList(mouse7);
        p1.addMiceToList(mouse8);
        p1.addMiceToList(mouse9);
        Mouse mouse4 = p1.identifyMouse(mouse8.getRefCode());
        assertEquals(mouse8, mouse4);
    } catch (PopulationExceptions pe) {
        System.out.println("ERROR: " + pe);
    }
}

@Test
void identifyMouseNonExistingRefCode(){
    try {
        Population p1 = new Population("Population 1", "John", 90);
        LocalDate date1 = LocalDate.of(2020, 1, 1);
        LocalDate date2 = LocalDate.of(2020, 1, 2);
        LocalDate date3 = LocalDate.of(2020, 1, 3);
        Mouse mouse10 = new Mouse(date1, 350, Chromosome.Xmut, Chromosome.Y,
Gender.MALE, 39.1f, "Good mouse");
        Mouse mouse11 = new Mouse(date2, 50, Chromosome.Xmut, Chromosome.X,
Gender.FEMALE, 37.1f, "Bad mouse");
        Mouse mouse12 = new Mouse(date3, 150, Chromosome.Xmut, Chromosome.Ymut,
Gender.MALE, 35.1f, "Fat mouse");
        p1.addMiceToList(mouse10);
        p1.addMiceToList(mouse11);
        p1.addMiceToList(mouse12);
        assertThrows(PopulationExceptions.class, () -> p1.identifyMouse(150));
    } catch (PopulationExceptions pe){
        System.out.println("ERROR: " + pe);
    }
}

@Test
void deleteMouseSuccessfully(){
    Population p1 = new Population("Population 1", "John", 90);
    LocalDate date1 = LocalDate.of(2020, 1, 1);
    Mouse mouse = new Mouse(date1, 360, Chromosome.Xmut, Chromosome.Y, Gender.MALE,
39.1f, "Good mouse");
    p1.addMiceToList(mouse);
    p1.deleteMouse(mouse.getRefCode());
    int expectedValue = 0;
    assertEquals(expectedValue, p1.amountOfMiceInPopulation());
}

@Test
void deleteMouseNotFound(){
    Population p1 = new Population("Population 1", "John", 90);
    assertThrows(PopulationExceptions.class, () -> {p1.deleteMouse(300);});
}

@Test
void isListOfMiceEmptyFalse(){
    Population p1 = new Population("Population 1", "John", 90);
    LocalDate date1 = LocalDate.of(2020, 1, 1);
    Mouse mouse = new Mouse(date1, 350, Chromosome.Xmut, Chromosome.Y, Gender.MALE,
39.1f, "Good mouse");
    p1.addMiceToList(mouse);
    boolean expectedResult = false;
    assertEquals(expectedResult, p1.isListOfMiceEmpty());
}

@Test
void isListOfMiceEmptyTrue(){

```

```

    Population p1 = new Population("Population 1", "John", 90);
    boolean expectedResult = true;
    assertEquals(expectedResult, p1.isListOfMiceEmpty());
}
@Test
void amountOfMiceInPopulation(){
    Population p1 = new Population("Population 1", "John", 90);
    LocalDate date1 = LocalDate.of(2020, 1, 1);
    Mouse mouse = new Mouse(date1, 350, Chromosome.Xmut, Chromosome.Y, Gender.MALE,
39.1f, "Good mouse");
    p1.addMiceToList(mouse);
    int expectedValue = 1;
    assertEquals(expectedValue, p1.amountOfMiceInPopulation());
}
@Test
void amountOfMiceInEmptyPopulation(){
    Population p1 = new Population("Population 1", "John", 90);
    int expectedValue = 0;
    assertEquals(expectedValue, p1.amountOfMiceInPopulation());
}
@Test
void fillVirtualPopulation(){
    Population p1 = new Population("Population 1", "John", 45, 10, 50, 50, 40, 50, 70);
    p1.fillVirtualPopulation(10, 50, 50, 50, 40, 70);
    int expectedAmountOfMice = p1.amountOfMiceInPopulation();
    assertEquals(expectedAmountOfMice, 10);
}
@Test
void mouseIsInPopulation(){
    Population p1 = new Population("Population 1", "John", 45);
    Mouse mouse = new Mouse(LocalDate.now(), 332, Chromosome.X, Chromosome.Y,
Gender.MALE, 39.1f, "Good mouse");
    Mouse mouse1 = new Mouse(LocalDate.now(), 223, Chromosome.X, Chromosome.Y,
Gender.MALE, 43.4f, "Mouse");
    p1.addMiceToList(mouse);
    p1.addMiceToList(mouse1);
    boolean result = p1.isMouseInPopulation(mouse);
    assertEquals(result, true);
}
@Test
void mouseIsNotInPopulation(){
    Population p1 = new Population("Population 1", "John", 45);
    Mouse mouse = new Mouse(LocalDate.now(), 332, Chromosome.X, Chromosome.Y,
Gender.MALE, 39.1f, "Good mouse");
    Mouse mouse1 = new Mouse(LocalDate.now(), 223, Chromosome.X, Chromosome.Y,
Gender.MALE, 43.4f, "Mouse");
    p1.addMiceToList(mouse);
    boolean result = p1.isMouseInPopulation(mouse1);
    assertEquals(result, false);
}
@Test
void assignMiceToFamiliesRemoveOldMice(){
    Population p1 = new Population("Population 1", "John", 45);
    // Create a family with a mother and a father older than 2.5 years
    Mouse mother = new Mouse(LocalDate.of(2020, 01, 01), 300, Chromosome.Xmut,
Chromosome.X, Gender.FEMALE, 37.0f, "Old Mother");
    Mouse father = new Mouse(LocalDate.of(2020, 01, 01), 290, Chromosome.X,
Chromosome.Y, Gender.MALE, 39.0f, "Old Father");
    p1.assignMiceToFamilies();
    assertFalse(p1.isMouseInPopulation(mother));
}

```

```

        assertFalse(p1.isMouseInPopulation(father));
    }
    @Test
    void assignMiceToFamiliesRemoveWhenMothersIsEmpty(){
        Population p1 = new Population("Population 1", "John", 45);
        // Create a family with a mother and a father older than 2.5 years
        Mouse mother = new Mouse(LocalDate.of(2020, 01, 01), 300, Chromosome.Xmut,
Chromosome.X, Gender.FEMALE, 37.0f, "Old Mother");
        Mouse father = new Mouse(LocalDate.of(2024, 01, 01), 290, Chromosome.X,
Chromosome.Y, Gender.MALE, 39.0f, "Old Father");
        p1.assignMiceToFamilies();
        assertFalse(p1.isMouseInPopulation(mother));
    }
    @Test
    void assignMiceToFamiliesRemoveWhenFatherIsDead(){
        Population p1 = new Population("Population 1", "John", 45);
        // Create a family with a mother and a father older than 2.5 years
        Mouse mother = new Mouse(LocalDate.of(2024, 01, 01), 300, Chromosome.Xmut,
Chromosome.X, Gender.FEMALE, 37.0f, "Old Mother");
        Mouse father = new Mouse(LocalDate.of(2020, 01, 01), 290, Chromosome.X,
Chromosome.Y, Gender.MALE, 39.0f, "Old Father");
        p1.assignMiceToFamilies();
        assertFalse(p1.isMouseInPopulation(father));
    }
}

```

5. Conclusiones

El trabajo me resultó muy interesante y práctico ya que reúne todos los temas explicados en la asignatura, brindando la oportunidad perfecta para aplicar todo lo aprendido en un problema que se asemeja a la realidad. Este proyecto de programación en Java fue una experiencia enriquecedora que no solo fortaleció mis habilidades técnicas, sino que también amplió mi comprensión y conocimiento de la programación orientada a objetos. A lo largo del desarrollo de la práctica, aprendí a manejar diversas estructuras de datos y a implementar algoritmos eficientes, lo cual mejoró mi capacidad para resolver problemas complejos.

Este proyecto también me enseñó la importancia de la planificación y la organización en la programación, siendo la creación de diagramas de flujo y la estructuración del código aspectos fundamentales para mantener la claridad y la eficiencia en mi trabajo.

En términos generales estoy conforme y contenta con la resolución que propuse para el trabajo, aunque me hubiese gustado aceptar el desafío de incluir la parte de swing ya que creo que es muy interesante y me puede aportar habilidades muy útiles para un futuro.